

СЕМЕНОВ А.С.

**ПОРОЖДАЮЩИЕ И РАСПОЗНАЮЩИЕ ПРОГРАММНЫЕ
СИСТЕМЫ**

Москва 2025

СОДЕРЖАНИЕ

Введение	
Глава 1. Классификация моделей формальных языков	
1.1. Основные понятия и определения	
1.2. Платформа моделей формальных языков	
1.3. Классификация Хомского	
Глава 2. Автоматные грамматики и конечные автоматы	
2.1. Определение автоматных грамматик и конечных автоматов ...	
2.2. Свойства регулярных языков: лемма о накачке	
2.3. Способы задания конечных автоматов	
2.4. Эпсилон-переход: реализация по заданному регулярному выражению составного автомата, построение ДКА из НКА	
2.5. Составные конечные автоматы и регулярные языки	
2.6. Паттерн модель формального регулярного языка	
2.7. Проектирование порождающих и распознающих систем	
Глава 3. Контекстно-свободные грамматики и МП-автоматы	
3.1. Определение КС-грамматик и МП-автоматов	
3.2. Преобразование КС-грамматик	
3.3. Пример преобразования КС-грамматик к приведенной форме .	
3.4. Определение МП-автомата и расширенного МП-автомата. Алгоритмы построения МП-автоматов по КС-грамматикам.	
3.5. Способы реализации синтаксических анализаторов	
3.6. LL(k)-грамматики: алгоритм построения LL(0)-анализатора ...	
3.7. Алгоритм построения SLR(1)-анализатора ...	
3.8. Грамматики предшествования	
3.9. Алгоритм “перенос-свертка”	
Глава 4. Грамматики общего вида и машины Тьюринга	
4.1. Грамматики общего вида и машина Тьюринга	
4.2. Контекстно-зависимые грамматики и ленточные автоматы	
4.3. Соотношение между грамматиками и языками	
4.4. Способы реализации	
Приложение А. Порядок выполнения задач	
Библиографический список	

Введение

Системы программирования - это компьютерные программы предназначены для разработки программного обеспечения. К ним относят: ассемблеры осуществляющие преобразование программы в форме исходного текста на языке ассемблера в машинные команды в виде объектного кода, трансляторы программ, компиляторы переводящие текст программы на языке высокого уровня, в эквивалентную программу на машинном языке, интерпретаторы анализирующие команды или операторы программы и тут же выполняющие их;

Пособие направлено на освоение и применение моделей формальных языков для разработки систем программирования. При этом решаются следующие задачи: определение типа модели формального языка, доказательство правильности сделанного выбора модели, построение цепочек вывода и дерева разбора для модели, нахождения эквивалентных моделей, описание языка и построения модели распознавателя, программная реализация модели, перевод языка из одной модели в другую формальную модель. Отличительной особенностью пособия является практическое содержание и решение задач с применением программной платформы, реализующей основополагающие алгоритмы формальных моделей, которые используются для проведения практических и курсовых работ. Пособие служит документацией к базовым алгоритмам платформы. Алгоритмы представлены в виде теоретико-множественной нотации.

Пособие состоит из семи глав и приложения, в котором приведены основные шаги выполнения практических работ. Теоретический материал и практические задания организованы по темам, в соответствии с классификацией моделей формальных языков, приведенной в главе 1. Основной задачей классификации является изучение порождающих и распознающих моделей формальных языков [1-5].

Пособие организовано в порядке возрастания сложности: предшествующий материал используется в последующих главах. В процессе выполнения практических заданий развивается умение разрабатывать модели формальных языков, проводить сравнительный анализ моделей, определять их вид в соответствии с классификацией, а также применять объектно-ориентированный подход для реализации моделей на языке программирования C#.

Во второй главе рассматривается класс автоматных грамматик и конечных автоматов, которые широко применяются в различных программных приложениях объектно-ориентированного подхода и искусственного интеллекта.

В третьей главе рассмотрен класс контекстно-свободных грамматик и МП-автоматов, позволяющий строить оптимальным образом синтаксические анализаторы для языков программирования.

В четвертой главе рассмотрены грамматики общего вида и машины Тьюринга, приведены соотношения между языками и грамматиками.

В пятой и шестой главах рассматриваются методы перевода между различными формальными моделями.

Книга полезна для студентов, аспирантов, ученых и специалистов, занимающихся разработкой формальных моделей языков, формальными лингвистическими моделями и математическим моделированием программных систем.

Глава 1. Классификация моделей формальных языков

Языки программирования являются формальными языками, которые были разработаны для вычислений на компьютере. Формальный язык состоит из цепочек символов, которые берутся из алфавита и объединяются в языковые строки. в соответствии с выбранной моделью, которая описывает правила формирования формального языка. Модели задают строго синтаксические характеристики языков посредством структурных паттернов.

1.1. Основные понятия и определения

Определим теоретико-множественные операции и понятия, используемые для формирования моделей формальных языков.

Определение 1. *Алфавит* (или словарный состав) – конечное множество символов. *Символ* - объект, описывается атрибутами и методами, например, a_p , где p - атрибут.

Фигурные скобки используются для обозначения множеств, Например, *алфавит* $V_1 = \{1,0\}$ – обозначает множество, содержащее два символа, *алфавит* $V_2 = \{a,b,c,d\}$ – множество, содержащее четыре символа, a , b , c и d .

Пустое множество V_3 обозначается как $V_3 = \emptyset$.

Определение 2. Результат операций *объединения* $\cup$ и *пересечения* $\cap$ *множеств*, например, $A \cup B = C$, $A \cap B = D$, если $A = \{a,b,c\}$ и $B = \{c,d,e\}$ равен новому множеству $C = \{a,b,c,d,e\}$ и $D = \{c\}$:

$$\{a,b,c\} \cup \{c,d,e\} = \{a,b,c,d,e\}$$

$$\{a,b,c\} \cap \{c,d,e\} = \{c\}$$

Определение 3. Выражение $A \supseteq B$, читается как множество A включает $\supseteq$ множество B , то есть, каждый элемент B является элементом A , например $\{c,d,e\} \supseteq \{c\}$. B называют подмножеством множества A .

Определение 4. Пусть $Z = \{1,2,3,\dots\}$ бесконечное (обозначается $\dots$) множество положительных целых чисел. Множество B задается с помощью предиката $|$ - читается как "так что" или "при условии":

$B = \{x \mid x \in Z, x \text{ есть четное число}\}$ читается, как B есть множество символов x , таких что $x \in Z$ и $($, - запятая читается как и) x есть четное число. Тогда $B = \{2,4,6,\dots\}$ бесконечное множество четных чисел. Предикат $|$ от условия " $x \in Z$, x есть четное число" принимает значение *истина* или *ложь*.

Модель формального языка L можно сформировать, пользуясь заданием множества с помощью предиката, например:

$$L = \{0^n 1^n \mid n \geq 0\}$$

Язык L включает цепочки символов, состоящие из нуля или нулей и того же числа последующих единиц n . Пустая цепочка символов включается в язык, что намного проще синтаксисов большинства моделей формальных языков.

Определение 5. *Цепочкой символов α в алфавите V* называется любая конечная последовательность символов этого алфавита. Например, $\alpha = 01$. $V = \{0,1\}$

Определение 6. Пустой цепочкой символов ε называется цепочка, не содержащая ни одного символа.

Определение 7. Длина цепочки α – число составляющих ее символов, обозначается $|\alpha|$. Например, если цепочка символов $\alpha = \text{defgabck}$, то длина α равна 8, $|\alpha| = 8$. Длина ε равна 0, $|\varepsilon| = 0$.

Определение 8. Декартовым произведением $A \times B$ множеств A и B называется множество пар (a, b) записывается, как $A \times B = \{ (a, b) \mid a \in A, b \in B \}$, где пара (a, b) элемент множества.

Определение 9. Операция итерации – обозначается парой круглых скобок со звездочкой $(a)^*$. Результатом является множество, включающее цепочки всевозможной длины, построенные с использованием символа a . Например: $(a)^* = (a), (aa), \dots, (aaa\dots)$

Определение 10. Унарный оператор, замыкание К्लीни или звезда К्लीни, обозначается V^* определяется над множеством символов V , замкнут относительно операции конкатенации символов из V . Результатом замыкания К्लीни является бесконечное надмножество множества V или множество цепочек символов, включая пустую цепочку ε . Например, если конечное множество $V = \{0, 1\}$, то $V^* = \{\varepsilon, 0, 1, 00, 11, 01, 10, 000, 001, 011, \dots\}$ есть бесконечное множество цепочек символов.

Замыкание К्लीни без пустой цепочки ε , обозначается как V^+ – бесконечное множество всех цепочек алфавита V , исключая пустую цепочку ε . Следовательно, $V^* = V^+ \cup \{\varepsilon\}$.

Определение 11. Оператор импликации $\rightarrow$ (от лат. *implicatio* - «связь») $A \rightarrow B$ - бинарная логическая связка, читается "если A , то B ". Применяется в правилах продукций (порождений) грамматик $A \rightarrow B$, читается "символ A порождает символ B ".

Определение 12. Бинарный оператор вывода $\Rightarrow$ означает, что символы слева от оператора заменяются или подставляются (принцип подстановки) символами справа, в соответствии с правилом продукции грамматики. Оператор применяется в порождающих системах. Например, $A \rightarrow Bb, B \rightarrow c$, тогда $A \Rightarrow Bb \Rightarrow cb$. Полученная цепочка символов называется цепочкой вывода или выводом.

Алгоритм 1. Выполнения оператора вывода $\Rightarrow$ для КС-грамматик

Вход: начальный символ, а также параметры (номера используемых правил продукций, *using_rules*)

Выход: последовательность номеров правил продукций, используемых для вывода цепочки терминалов

α = цепочка начальных символов // Входная цепочка

Grammar = $\{T, V, P, S\}$ // КС-грамматика: V - нетерминалы, T - терминалы, P - правила, S_0 - стартовый символ

$S_\alpha = \{\alpha\}$ // Основной стек с входной цепочкой

$ST = \{\}$ // Стек для обнаруженных терминалов

$SV = \{\}$

while ($ST \neq \emptyset$):

$a = S_\alpha.\text{pop}()$ // Извлекаем верхний символ из стека

```

if (a ∈ V):      // Если символ - нетерминал
    SV = SV ∪ { a } // Записываем в стек нетерминалов
    foreach (LHS → RHS ∈ P): // Для каждого правила продукции
        foreach (xi ∈ RHS): // Добавляем символы RHS в обратном порядке в стек
            Sα = Sα ∪ { xi }
        end foreach
    break // Прекращаем обработку текущего нетерминала
else:           // Если символ - терминал
    ST = ST ∪ { a } // Записываем в выходную цепочку
end if
end while
return ST

```

Пояснение:

1. Инициализация:

- Стек основной (S_α) содержит входную цепочку символов.
- Стек для нетерминалов (SV) - пустой в начале.
- Стек терминалов (ST) – пустой в начале

2. Основной цикл:

- Пока основной стек не опустеет, извлекается верхний символ для анализа.

3. Обработка символов:

- Если символ — нетерминал:
 - Записывается в стек для нетерминалов.
 - Для него ищутся подходящие правила продукции, и символы из правой части правила добавляются в основной стек в обратном порядке (чтобы слева направо порядок сохранился при обработке).
- Если символ — терминал:
 - Добавляется в выходную цепочку.

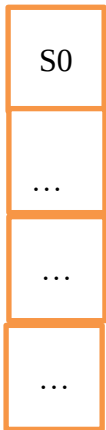
4. Завершение:

- Алгоритм завершается, когда основной стек пуст.
- Результат включает конечную цепочку и список всех обработанных нетерминалов.

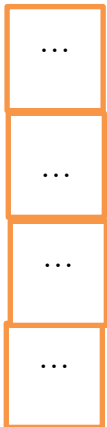
Это позволяет шаг за шагом разворачивать цепочку, фиксируя нетерминалы и строя результат.

Начальные данные: S0 - начальный символ Alpha - цепочка символ G - текущая грамматика	Правила грамматики: S0 -> aA A -> bB
SAlpha - стек, содержащий начальную цепочку SV - стек терминалов	

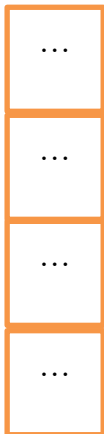
S_alpha mar 0



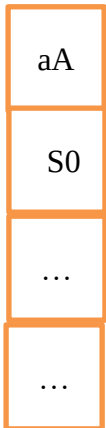
ST mar 0



SV mar 0



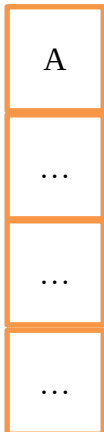
S_alpha mar 1



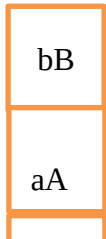
ST mar 1



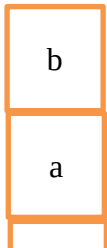
SV mar 1



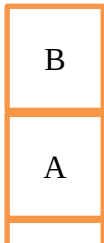
S_alpha mar 2



ST mar 2



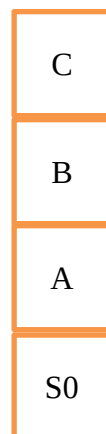
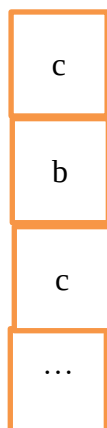
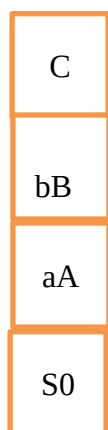
SV mar 2



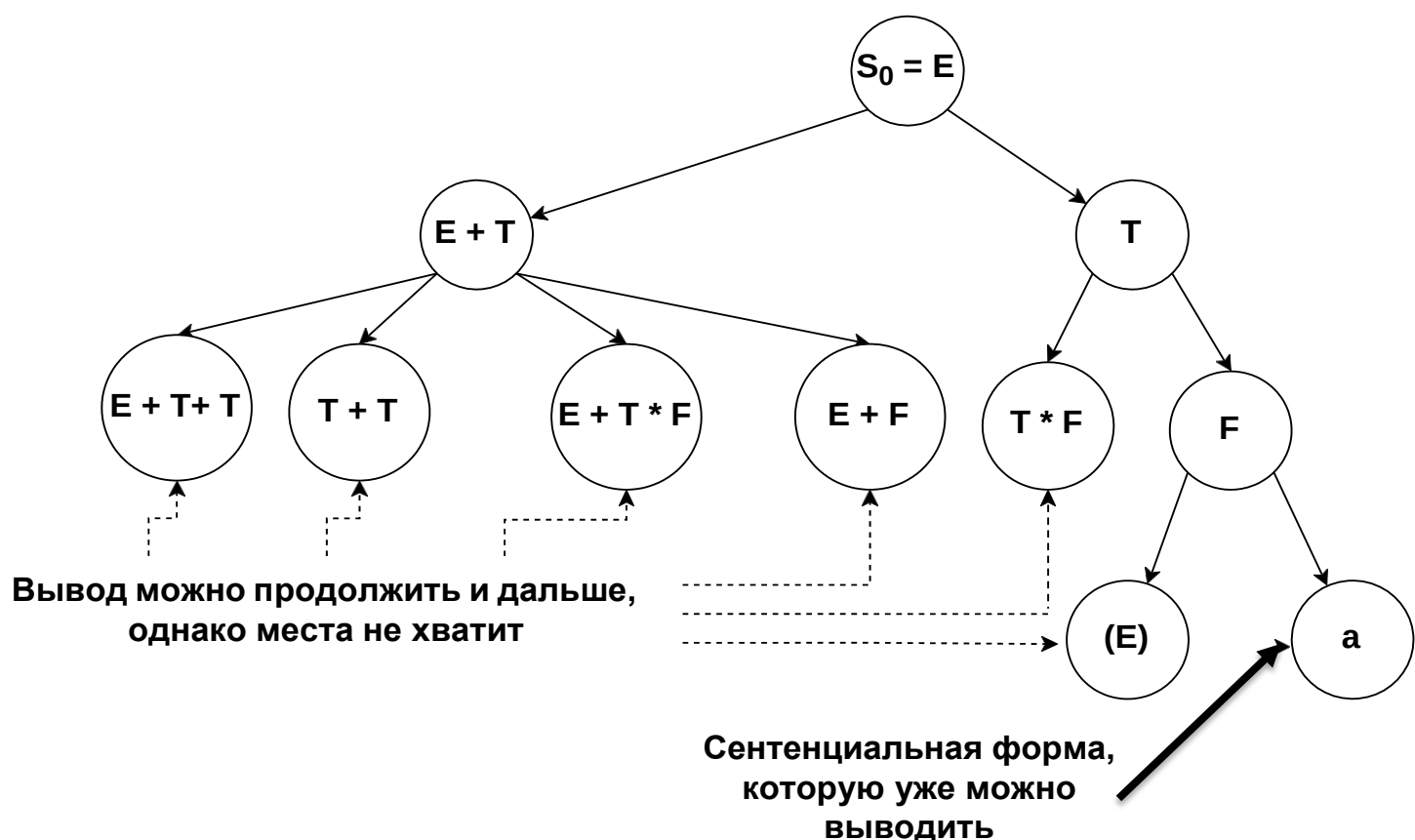
S_alpha шаг 3

ST шаг 3

SV шаг 3



Если визуализировать всё предыдущее то у нас получится



Определение 13. Операция (также правило) перехода δ или функция перехода автомата.

Определение 14. Бинарный оператор $\mid$ — означает такт срабатывания модели автомата или алгоритма в соответствии с правилом перехода δ (применяется в распознающих системах).

Замыкание Клини, оператор вывода $\Rightarrow$ и такт срабатывания $\mid$ являются основополагающими операторами для формирования порождающих и распознающих моделей формальных языков.

1.2. Платформа моделей формальных языков

Появление моделей формальных языков было обусловлено стремлением формализовать естественные языки и языки программирования. Рассмотрим основополагающие порождающие и распознающие модели описания формальных языков см. рис. 1.1.



Рис. 1.1. Платформа моделей формальных и естественных языков

Обработка естественного языка (Natural Language Processing– NLP) текстов и речи с привлечением формальных моделей и алгоритмов машинного обучения. Например, можно использовать NLP, чтобы создавать системы распознавания речи, обобщения документов, машинного перевода, выявления спама, распознавания именованных сущностей, ответов на вопросы, автокомплита, предиктивного ввода текста и т.д. [10-15]

Порождающие модели

Для описания синтаксиса языков программирования наибольшее распространение получили следующие порождающие системы: форма Бекуса-Наура и ее модификации, диаграммы Вирта [3], формальные грамматики .

В порождающих моделях правила задаются продукциями, состоящими из двух частей: левой и правой. В левой части, записывается определение, в правой части варианты, из которых может состоять определение.

Например, предложение “он смотрит кино” может быть определено синтаксическими правилами:

предложение состоит из *подлежащего*, *группа сказуемого*

подлежащее - он

группа сказуемого - *сказуемое*, *дополнение*

сказуемое - смотрит

дополнение – кино

В рассматриваемых правилах порядок элементов фиксирован. Для определения грамматического формализма, эквивалентного синтаксическому, используем правило продукций $\rightarrow$. Тогда порождающая модель, для рассматриваемого предложения, примет вид:

Предложение $\rightarrow$ *подлежащее*, *группа сказуемого*

подлежащее $\rightarrow$ он

группа сказуемого $\rightarrow$ *сказуемое*, *дополнение*

сказуемое $\rightarrow$ смотрит

дополнение $\rightarrow$ кино

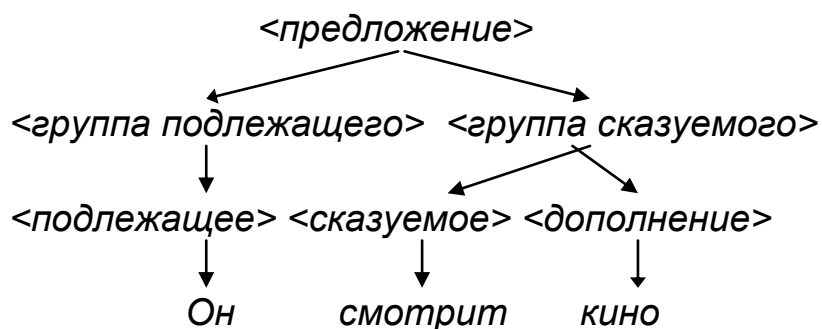


Рис. 1.2. Древовидная структура предложения

Если шаги подстановки интерпретировать как шаги построения графа дерева то получим дерево синтаксического разбора см. рис. 1.2., спускаясь по которому из вершины <предложение> получим предложение “он смотрит кино”.

Нормальная форма Бэкуса (БНФ) или форма Бэкуса-Наура была предложена Д.Бэкусом в 1959 году и впервые применена П.Науром для описания языка Алгол-60. БНФ - метаязык, который используется для описания других языков.

Обозначение правил в форме записи $\xi ::= \eta$ относится к нотации БНФ. В этой нотации используются обозначения:

$::=$ - это есть, $|$ - или, $< >$ - угловые скобки, в которых записываются металингвистическая переменная, т.е. определяемое понятие;

$[]$ - факультативный элемент (необязательная часть), то есть конструкция в скобках может присутствовать или отсутствовать во фразе языка;

{ }- множественный элемент (один из, элемент выбора), то есть во фразе языка используется один из элементов внутри скобки.

Правило вида $\alpha ::= \beta \mid \beta\gamma$ можно представить $\alpha ::= \beta [\gamma]$.

Рассматриваемый пример в БНФ:

$\langle \text{предложение} \rangle ::= \langle \text{подлежащее} \rangle \langle \text{группа сказуемого} \rangle$

$\langle \text{подлежащее} \rangle ::= \text{он} \mid \text{она}$

$\langle \text{группа сказуемого} \rangle ::= \langle \text{сказуемое} \rangle \langle \text{дополнение} \rangle$

$\langle \text{сказуемое} \rangle ::= \text{смотрит} \mid \text{обожает}$

$\langle \text{дополнение} \rangle ::= \text{кино}$

Используя множество правил, *выведем* или *породим* предложение по следующей схеме. Начнем с начального символа грамматики - $\langle \text{предложение} \rangle$, найдем правило, в котором $\langle \text{предложение} \rangle$ слева от $::=$, и подставим вместо $\langle \text{предложение} \rangle$ цепочку, которая расположена справа от $::=$, т.е.

$\langle \text{предложение} \rangle \Rightarrow \langle \text{подлежащее} \rangle \langle \text{группа сказуемого} \rangle$

Заменим синтаксическое понятие на одну из цепочек, из которых оно может состоять. Повторим процесс. Возьмем один из метасимволов в цепочке $\langle \text{подлежащее} \rangle \langle \text{группа сказуемого} \rangle$, например $\langle \text{подлежащее} \rangle$; найдем правило, где $\langle \text{подлежащее} \rangle$ находится слева от $::=$, и заменим $\langle \text{подлежащее} \rangle$ в исходной цепочке на соответствующую цепочку, которая находится справа от $::=$. Получим вывод:

$\langle \text{подлежащее} \rangle \langle \text{группа сказуемого} \rangle \Rightarrow \text{он} \langle \text{группа сказуемого} \rangle$

Полный вывод одного предложения будет таким:

$\langle \text{предложение} \rangle \Rightarrow \langle \text{подлежащее} \rangle \langle \text{группа сказуемого} \rangle \Rightarrow \text{он} \langle \text{группа сказуемого} \rangle \Rightarrow \text{он} \langle \text{сказуемое} \rangle \langle \text{дополнение} \rangle \Rightarrow \text{он} \text{смотрит} \langle \text{дополнение} \rangle \Rightarrow \text{он} \text{смотрит} \text{кино}$

Этот вывод предложения запишем сокращенно, используя символ $\Rightarrow^+$, который означает, что цепочка выводима *нетривиальным способом*:

$\langle \text{предложение} \rangle \Rightarrow^+ \text{он} \text{смотрит} \text{кино}$

Последовательность цепочек $\alpha_1, \alpha_2, \dots, \alpha_n$ в этом случае называется выводом цепочки α_n из α_1 : $\alpha_1 \Rightarrow^+ \alpha_n$.

Две последовательности связаны отношением $\Rightarrow^*$, когда вторая получается из первой применением некоторой последовательности продукции $\alpha_1 \Rightarrow^* \alpha_n$ тогда, и только тогда, когда $\alpha \Rightarrow^+ \alpha_n$. $\alpha_1 \Rightarrow^* \alpha_n$ означает, что цепочка α_n выводится из α_1 за **ноль** или более шагов.

На каждом шаге можно заменить любую метапеременную. В приведенном выше выводе всегда заменялся самый левый из них.

Если в процессе вывода цепочки правила грамматики применяются только к самому левому нетерминалу, говорят, что получен *левый вывод* цепочки. Аналогично определяется правый вывод.

Рассматриваемая модель языка определяет несколько предложений (цепочек) языка:

$\text{он} \text{смотрит} \text{кино} \quad \text{он} \text{обожает} \text{кино}$
 $\text{она} \text{смотрит} \text{кино} \quad \text{она} \text{обожает} \text{кино}$

Диаграммы Вирта позволяют визуально представить правила порождения. Каждому правилу соответствует диаграмма.

Элементы ИЛИ обозначаются разветвлениями и вершинами, элементы И последовательными вершинами графа, метасимволы {...} циклом. Вершины могут быть двух типов: терминальные – кружки, и нетерминальные – прямоугольники. Каждому правилу соответствует диаграмма см. рис. 1.3.

Правила- И, цикл, ИЛИ: $T \rightarrow \{ F \} | (E)$

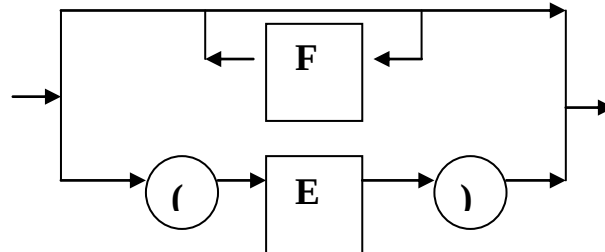


Рис. 1.3. Диаграммы Вирта

Грамматика. Грамматика позволяет осуществить грамматический анализ предложения, а значит, и явно описать его структуру. Одно из назначений грамматики – это определить бесконечное число предложений языка с помощью конечного числа правил.

Грамматика языка может быть реализована двумя способами: в виде порождающей системы и в виде устройств, называемых распознавателями.

Синтаксис и семантика. Для того чтобы убрать семантику и перейти к формальному описанию модели языков закодируем заглавными (нетерминальные) символами S,A,B,C грамматику русского языка, а символами (терминальные) a,b,c слова рассматриваемого предложения:

S	A	B
<i>Предложение</i> → <i>подлежащее, группа сказуемого</i>		
A	a	
<i>подлежащее</i> → он		
B	C	D
<i>группа сказуемого</i> → <i>сказуемое, дополнение</i>		
C	b	
<i>сказуемое</i> → смотрит		
D	c	
<i>дополнение</i> → кино		

Определение 1. Грамматика $G = (T, V, P, S_0)$,

где T - конечное множество терминальных символов (терминалов) алфавита;

V – конечное множество нетерминальных символов алфавита, не пересекающихся с T , $T \cap V = \emptyset$,

S_0 - начальный символ (или аксиома), $S_0 \in V$;

P - конечное множество правил порождения (продукций), $P = (T \cup V)^+ \times (T \cup V)^*$. Элемент (α, β) множества P называется *правилом порождения* и записывается в виде $\alpha \rightarrow \beta$.

Правила порождения с одинаковыми левыми частями $\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$, записывают сокращенно $\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$, где $\beta_i, i = 1, 2, \dots, n$ и называют *альтернативой* правил порождения из цепочки α .

Выводом (обозначим $\Rightarrow$) в грамматике G называют произвольную, конечную или бесконечную, последовательность цепочек. Цепочка $\beta \subset (T \cup V)^*$ *выводима* из цепочки $\alpha \subset (T \cup V)^+$ в грамматике G , ($\alpha \Rightarrow \beta$), если существуют цепочки $\gamma_0, \gamma_1, \dots, \gamma_n (n \geq 0)$, такие, что $\alpha = \gamma_0 \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n = \beta$. Последовательность $\gamma_0, \gamma_1, \dots, \gamma_n$ называется *выводом длины n* .

Цепочка $\beta \subset (T \cup V)^*$ *непосредственно выводима* ($\alpha \Rightarrow \beta$) из цепочки $\alpha \subset (T \cup V)^+$ в грамматике G , если $\alpha = \xi_1 \gamma \xi_2, \beta = \xi_1 \delta \xi_2$, где $\xi_1, \xi_2, \delta \subset (T \cup V)^*, \gamma \subset (T \cup V)^+$ и правило продукции $\gamma \rightarrow \delta$ содержится в P .

Для обозначения терминальных символов грамматики мы будем употреблять прописные буквы (или строки прописных букв), а для обозначения не терминалов — заглавные буквы (или строки заглавных букв).

Примеры грамматик.

Пример 1. Для примера “он смотрит кино” имеем грамматику

$G = (\{“он”, “кино”, “смотрит”\}, \{Предложение, подлежащее, группа сказуемого, сказуемое, дополнение\}, P, S_0)$, где $S_0 = Предложение$, P состоит из правил $P = \{p_1, p_2, p_3, p_4, p_5\}$;

p_1 : *Предложение* $\rightarrow$ *подлежащее, группа сказуемого*

p_2 : *подлежащее* $\rightarrow$ он

p_3 : *группа сказуемого* $\rightarrow$ *сказуемое, дополнение*

p_4 : *сказуемое* $\rightarrow$ смотрит

p_5 : *дополнение* $\rightarrow$ кино

Пример 2. $G = (T = \{c, d\}, V = \{B, S_0\}, P, S_0)$, где P состоит из правил

$S_0 \rightarrow cBd$

$cB \rightarrow ccBd$

$B \rightarrow \varepsilon$

Цепочка $ccBdd$ непосредственно выводима из cBd в грамматике G .

Цепочка $cccBddd$ в грамматике G выводима, т.к. существует вывод $S_0 \Rightarrow cBd \Rightarrow ccBdd \Rightarrow cccBddd \Rightarrow cccddd$. Длина вывода равна 4.

Определение 2. Языком, порождаемым грамматикой $G = (T, V, P, S_0)$, называется множество $L(G) = \{\alpha \subset T^* \mid S_0 \Rightarrow^* \alpha\}$.

Другими словами, $L(G)$ - это все цепочки над множеством алфавита T , которые выводимы из S_0 подстановкой правил из P .

Цепочка $\alpha \subset (T \cup V)^*$, для которой $S_0 \Rightarrow^* \alpha$ (то есть цепочка, которая может быть выведена из начального символа), называется *сентенциальной формой* в грамматике G . Язык, порождаемый грамматикой, можно определить как множество терминальных сентенциальных форм.

Например, пусть задана грамматика $G_1 = (T = \{0, 1\}, V = \{A, S_0\}, P_1, S_0)$, состоящая из правил P_1 : $S_0 \rightarrow 0A1$

$0A \rightarrow 00A1$

$A \rightarrow \varepsilon$

Тогда язык, порождаемый грамматикой G_1 , это язык $L(G_1) = \{0^n 1^n \mid n > 0\}$.

Грамматики G_1 и G_2 называются *эквивалентными*, если $L(G_1) = L(G_2)$. Например, пусть задана грамматика $G_2 = (\{0,1\}, \{S\}, P_2, S)$, где

$P_2: S \rightarrow 0S1 \mid 01$, тогда

грамматики G_1 и G_2 эквивалентны, т.к. обе порождают язык $L(G_1) = L(G_2) = \{0^n 1^n \mid n > 0\}$.

Грамматики G_2 и G_3 называются *почти эквивалентными* $L(G_2) = L(G_3)$, если $L(G_2) = L(G_3) \cup \{\varepsilon\}$. Другими словами, грамматики почти эквивалентны, если языки, ими порождаемые, отличаются не более чем на ε .

Определим грамматику $G_3 = (\{0,1\}, \{S\}, P_3, S)$ с правилами

$P_3: S \rightarrow 0S1 \mid \varepsilon$

Тогда грамматики G_2 и G_3 *почти эквивалентны*, так как $L(G_2) = \{0^n 1^n \mid n > 0\}$, а $L(G_3) = \{0^n 1^n \mid n \geq 0\}$, т.е. $L(G_3)$ состоит из всех цепочек языка $L(G_2)$ и пустой цепочки, которая в $L(G_2)$ не входит.

Распознающие модели

Формальный язык может быть распознан моделью отвечающей на вопрос, принадлежит ли данная цепочка символов языку. Например, принадлежит ли предложение “он смотрит кино” языку заданному рассмотренными грамматическими правилами? Для этого необходимо построить распознаватель.

Правила перехода δ описывают те последовательности символов, которые являются корректными, т.е. допустимыми предложениями языка.

Утверждение 1. Язык распознающей системы R_s эквивалентен языку порождающей системы G_s : $L(R_s) = L(G_s)$.

1.3. Классификация Хомского

Классификация Хомского вводит порождающие модели формальных языков. В книге классификация Хомского расширена эквивалентными распознающими моделями Таб. 1.1. Это дает представление об общности грамматик, и об эквивалентных им формальных автоматах. Приводятся примеры языков и грамматик.

Таб. 1.1. Классификация Хомского

Тип	Ограничения на правила P	Пример правил грамматики $G = (T, V, P, S_0)$	Пример языка	Автомат или абстрактная машина
0	Общего вида, не накладываемся никаких ограничений	$S_0 \rightarrow CD, C \rightarrow 0CA, C \rightarrow 1CB, AD \rightarrow 0D, BD \rightarrow 1D, A0 \rightarrow 0A, A1 \rightarrow 1A, B0 \rightarrow 0B, B1 \rightarrow 1B, C \rightarrow \varepsilon, D \rightarrow \varepsilon$	$\{\omega\omega \mid \omega \in \{0,1\}^*\}$, язык состоит из цепочек четной длины, из 0 и 1	Машина Тьюринга

1	Контекстно-зависимая (КЗ) $\alpha \rightarrow \beta, \alpha \leq \beta $ $\alpha \in (T \cup V)^*$ $\beta \in (T \cup V)^*$	$S_0 \rightarrow 0B0$ $B \rightarrow 1 \mid 0BC$ $C0 \rightarrow 100$ $C1 \rightarrow 1C$	$\{0^n 1^n 0^n \mid n \geq 1\}$	Машина Тьюринга с конечной лентой
2	Контекстно-свободная (КС) $A \rightarrow \alpha$ $\alpha \in (T \cup V)^*$	$S_0 \rightarrow AS_0B \mid AB$ $A \rightarrow 0$ $B \rightarrow 1$ (нисходящий разбор) $LL(k)$ грамматика (восходящий разбор) $LR(k)$ грамматика $k = 0, 1$	$\{0^n 1^n \mid n \geq 1\}$	МП-автомат с магазинной памятью и РМП-автомат: Распознаватели на основе управляющих таблиц: $LL(1)$ $LR(0), LR(1)$ грамматики предшествования
3	Регулярная $A \rightarrow a$ $A \rightarrow aB$ $A \rightarrow \varepsilon$	$S_0 \rightarrow 0 \mid 0B$ $A \rightarrow 0B$ $B \rightarrow 1 \mid 1A$	$\{0(01)^n \mid n \geq 0\}$	Конечный автомат (КА) $\delta(A, a) = \{q_f\}$ $\delta(A, a) = \{B\}$ $\delta(A, \varepsilon) = \{q_f \mid q_\varepsilon\}$, где q_ε - состояние для эпсилон такта

Классификация Хомского дает представление о 4 типах грамматик и об эквивалентных формальных автоматах, располагающихся по иерархии от 0 до 3 в порядке убывания их общности. Грамматики классифицируются по ограничениям накладываемым на правила вывода P, представляющие собой структурные паттерны формальной модели. В правилах вывода P большими латинскими буквами обозначают нетерминальные символы, маленькими латинскими буквами - терминальные.

Глава 2. Автоматные грамматики и конечные автоматы

2.1. Определение автоматных грамматик и конечных автоматов

Определение 3. Грамматика типа 3 или автоматная (регулярная) A -грамматика – это грамматика $G = (T, V, P, S_0)$, у которой правила порождения P имеют вид по классификации Хомского $A \rightarrow bV$ (праволинейное правило) или $V \rightarrow a$ (заключительное правило), где $A, V \in V, a, b \in T$. Каждое правило такой грамматики содержит единственный нетерминал в левой части, всегда один терминал в правой части, за которым может следовать один нетерминал. Такую грамматику также называют *праволинейной*.

Грамматика $G = (T, V, P, S_0)$ называется *леволинейной*, если каждое правило из P имеет вид по классификации Хомского $A \rightarrow Vb$ либо $V \rightarrow a$, где $A, V \in V, a, b \in T$.

Таким образом, грамматику типа 3 можно определить как праволинейную либо как леволинейную.

Выбор определения не влияет на множество языков, порождаемых грамматиками этого класса, поскольку доказано, что множество языков, порождаемых праволинейными грамматиками, совпадает с множеством языков, порождаемых леволинейными грамматиками.

Автоматной грамматике эквивалентен простейший распознаватель – недетерминированный конечный автомат. Автомат представляет собой устройство, у которого отсутствует вспомогательная память.

Определение 4. Конечный автомат (КА) – это пятерка объектов

$KA = (Q, \Sigma, \delta, q_0, F)$, где

Q - конечное множество состояний;

Σ - конечный алфавит входных символов, например, $\Sigma = \{0,1\}$;

δ - конечное множество функций (правил) переходов. Функция перехода задается отображением

$$\delta: Q \times \Sigma \rightarrow Q,$$

где Q - конечное множество подмножеств множества Q ;

q_0 - начальное состояние автомата, $q_0 \in Q$;

$F \subseteq Q$ - множество заключительных состояний.

В каждый момент времени КА находится в некотором состоянии $q \in Q$ и читает поэлементно последовательность символов $a_k \in \Sigma$, записанную на конечной ленте. При этом либо читающая головка машины движется в одном направлении (слева направо), либо лента перемещается (справа налево) рис.2.1. при неподвижной читающей головке.



Рис. 2.1. Схема конечного автомата
Входная лента: Входные символы $a_i \in \Sigma$

Если автомат в состоянии q_i читает символ a_k и определена функция перехода $\delta(q_i, a_k) = q_j$, то автомат воспринимает символ a_k и переходит в состояние q_j для обработки следующего символа.

Определение 5. Конфигурация КА – это пара множества $(q, \omega) \in Q \times \Sigma^*$, где $q \in Q$, $\omega \in \Sigma^*$. Конфигурация (q_0, ω) называется *начальной*, а (q, ε) , где $q \in F$, – *заключительной*.

Определим бинарное отношение $\vdash$ на конфигурациях, соответствующее одному такту работы КА. Если $q' \in \delta(q, a)$, то $(q, a\omega) \vdash (q', \omega)$ для всех $\omega \in \Sigma^*$.

Пусть $\{C\}$ – множество конфигураций.

1. $C \vdash^0 C'$ означает, что $C = C'$
2. $C_0 \vdash^k C_k$, если существует последовательность конфигураций $C_1, C_2, \dots, C_{k-1}$, для $k \geq 1$, в которых $C_i \vdash C_{i+1}$ для $0 \leq i < k$.
3. $C \vdash^+ C'$ означает, что $C \vdash^k C_k$ для некоторого $k \geq 1$, а $C \vdash C'$ означает, что $C \vdash^k C'$ для $k \geq 1$.

Конечный автомат $КА = (Q, \Sigma, \delta, q_0, F)$ распознает входную цепочку $\omega \in \Sigma^*$, если $(q_0, \omega) \vdash^k (q_f, \varepsilon)$ для $q_f \in F$.

Языком $L(КА)$, распознаваемым КА называется множество входных цепочек,

$$L(КА) = \{\omega \in \Sigma^* \mid (q_0, \omega) \vdash^k (q_f, \varepsilon), \text{ где } q_f \in F\}$$

КА называется *недетерминированным*, если для каждой конфигурации существует конечное множество всевозможных следующих шагов, любой из которых КА может сделать, исходя из этой конфигурации.

КА называется *детерминированным*, если для каждой конфигурации существует не более одного следующего шага.

Для любого конечного недетерминированного автомата можно построить ему эквивалентный детерминированный автомат.

Класс языков, распознаваемый конечными автоматными, совпадает с классом языков, порождаемых автоматными грамматиками и наоборот.

Утверждение 2. Пусть задана автоматная грамматика $G = (T, V, P, S_0)$, тогда существует такой (недетерминированный) конечный автомат $КА = (Q, \Sigma, \delta, q_0, F)$, что $L(КА) = L(G)$. КА строится следующим образом:

1. Входные алфавиты конечного автомата Σ и автоматной грамматики T совпадают, $\Sigma = T$.

2. $Q = V \cup \{q_f\}$, где q_f – заключительное состояние конечного автомата.

3. $q_0 := S_0$.

4. Функция переходов δ КА определяется следующим образом:

1. для каждого правила порождения вида $A \rightarrow a \in P$, $A \in V$, $a \in T$, построить функцию перехода $\delta(A, a) = \{q_f\}$, $A \in Q$, $a \in \Sigma$;
2. для каждого правила порождения вида $A \rightarrow aB \in P$, построить функцию перехода $\delta(A, a) = \{B\}$, $A, B \in Q$, $a \in \Sigma$;
3. для каждого правила порождения вида $A \rightarrow \varepsilon \in P$, $F = \{A, q_f\}$, $A \in Q$, в противном случае $F = \{q_f\}$;

Пример. Пусть задан регулярный язык $L = \{0(10)^n \mid n \geq 0\}$. Построить автоматную грамматику $G = (T, V, P, S_0)$ для заданного языка L и привести пример вывода строки. Используя грамматику G , построить КА $(\Sigma, Q, \delta, q_0, F)$ и привести пример конфигурации КА.

1. Построение грамматики. $L = \{0(10)^n \mid n \geq 0\}$, то 0, 010, 01010 и т.д. - этот язык порожден регулярной грамматикой $G = (T, V, P, S_0)$, где $T = \{0, 1\}$, $V = \{S_0, A, B\}$, $P = \{S_0 \rightarrow 0, S_0 \rightarrow 0A, A \rightarrow 1B, B \rightarrow 0, B \rightarrow 0A\}$. Пример вывода цепочки

$S_0 \Rightarrow 0A \Rightarrow 01B \Rightarrow 010A \Rightarrow 0101B \Rightarrow 01010$.

2. Построение КА. Воспользуемся утверждением 1, тогда

$KA = (\{0, 1\}, \{S_0, A, B, q_f\}, \delta, S_0, q_f)$

1. $\delta(S_0, 0) = \{A, q_f\}$

2. $\delta(A, 1) = \{B\}$

3. $\delta(B, 0) = \{A, q_f\}$

Пример конфигурации КА: $(S_0, 01010) \xrightarrow{1} (A, 1010) \xrightarrow{2} (B, 010) \xrightarrow{3} (A, 10) \xrightarrow{2} (B, 0) \xrightarrow{3} (q_f, \varepsilon)$

2.2. Способы задания конечных автоматов

На рис. 2.2. приведена диаграмма переходов КА для рассматриваемого примера.

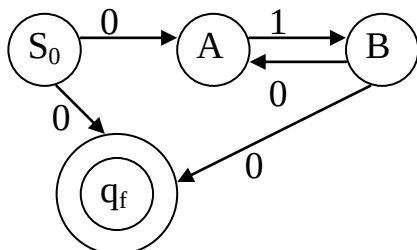


Рис. 2.2. Диаграмма переходов КА

Конечный автомат можно задать в виде таблицы переходов и диаграммы (графа) переходов.

В таблице переходов двум аргументам ставится в соответствие одно значение. В таб. 2.1 приведена таблица переходов КА для рассматриваемого примера.

Таб. 2.1. Таблица переходов для КА

	0	1
S_0	{ A, q_f }	
A		{B}
B	{A, q_f }	
q_f		

Диаграммой переходов КА называется неупорядоченный граф, удовлетворяющий условиям:

- каждому состоянию q соответствует некоторая вершина, отмеченная его именем;
- диаграмма переходов содержит дугу из состояния q_k в состояние q_n , отмеченную символом a , если $q_k \in \delta(q_n, a)$. Дуга может быть помечена множеством символов, переводящих автомат из состояния q_k в состояние q_n ;
- вершины, соответствующие заключительным состояниям $q_f \in F$, отмечаются двойным кружком.

2.3. Свойства регулярных языков: лемма о накачке

Теорема, называемая “лемма о накачке” утверждает, что все достаточно длинные цепочки регулярного языка можно *накачать*, то есть **повторить** внутреннюю часть цепочки символов сколько угодно раз, производя новую подцепочку, также принадлежащее языку.

Все конечные языки являются автоматными, эту проверку имеет смысл делать только для бесконечных языков.

Лемма описывает существенное свойство всех регулярных языков и служит инструментом для доказательства нерегулярности некоторых языков.

Формальное утверждение. Длина накачки p принимается более длины самой длинной цепочки языка L , такое что цепочка w из L длины по меньшей мере p может быть записана как $w = xyz$, где y – это подцепочка, которую можно накачать (удалить или повторить произвольное число раз, так что результат останется в L).

$$\begin{aligned}
 \forall L \subseteq \Sigma^* (\text{regular}(L) \Rightarrow & \quad 1. \text{любой язык } L \subseteq \Sigma^* \text{ регулярный если} \\
 (\exists p \geq 1 & \quad 2. \text{существует целое } p \geq 1 \text{ такое что} \\
 (\forall w \in L (|w| \geq p) \Rightarrow & \quad 3. \text{для любой цепочки языка } |w| \geq p \\
 (\exists x, y, z \in \Sigma^* (w = xyz \Rightarrow & \quad 4. \text{найдется } w = xyz \text{ такое что} \\
 (|y| \geq 1 \wedge |xy| \leq p \wedge i \geq 0 \wedge (xy^iz \in L)) & \quad 5. y \text{ повторяется } i \text{ раз} \\
)) &
 \end{aligned}$$

))
)
), на x и z ограничений не накладывается.

$$\forall L \subseteq \Sigma^* (\text{regular}(L) \Rightarrow (\exists p \geq 1 (\forall w \in L ((|w| \geq p) \Rightarrow (\exists x, y, z \in \Sigma^* (w = xyz \Rightarrow (|y| \geq 1 \wedge |xy| \leq p \wedge \forall i \geq 0 (xy^i z \in L))))))))))$$

Доказательство леммы о накачке

Пусть автоматный язык L содержит бесконечное число цепочек и предположим, что L распознаётся детерминированным конечным автоматом КА с n состояниями. Для проверки заключения леммы выберем произвольную цепочку α этого языка, которая имеет длину n .

Если конечный автомат КА распознаёт L , то цепочка α допускается этим автоматом, то есть в автомате КА существует путь длины n из начального в одно из заключительных состояний, помеченный символами цепочки α . Путь этот не может быть простым, он должен проходить ровно через $n+1$ состояние, в то время как автомат КА имеет n состояний. Это значит, что этот путь проходит по меньшей мере два раза через одно и то же состояние автомата КА, то есть на этом пути есть цикл с повторяющимся состоянием. Пусть это повторяющееся состояние q_k .

Разделим цепочку α на три части, так что $\alpha = xuz$, где u — подцепочка, переводящая КА из состояния q_k опять в состояние q_k , и z — подцепочка, переводящая КА из состояния q_k в заключительное состояние. Заметим, что как x , так и z могут быть пустыми, но подцепочка u не может быть пустой. Но тогда очевидно, что автомат КА должен допускать также и цепочку $xuuz$, поскольку повторяющаяся подцепочка u снова проходит по циклическому пути из q_k в q_k , а также и цепочку $xuuuz$, и любую вида $xu...uz$.

Это рассуждение и составляет доказательство леммы о накачке.

Для доказательства регулярности языка:

1. выделить цепочку y^i для всех $i \geq 0$, $xy^iz \in L$, на x и z ограничений не накладывается (то есть, нет цепочки символов y^i , то не регулярный язык).
2. построить распознаватель для заданного языка - конечный автомат;

Свойство замкнутости регулярных языков, позволяет минимизировать построение автомата:

1. строить распознаватели для одних языков, построенных из других с помощью операций;
2. определить, что два различных автомата определяют один язык.

Пример 2. Рассмотрим язык $L_{01} = \{0^n 1^n \mid n \geq 1\}$, состоящий из всех цепочек вида 01, 0011, 000111 и так далее, содержащий один или несколько нулей, за которыми следует такое же количество единиц.

Утверждается, что язык L_{01} нерегулярен.

Если бы L_{01} был регулярным языком, то допускался некоторым ДКА, имеющим какое-то число состояний k . Пусть на вход ДКА поступает k нулей. Он находится в некотором состоянии после чтения каждого из $k+1$ префиксов

входной цепочки, т.е. $\varepsilon, 0, 00, \dots, 0^k$. Поскольку есть только k различных состояний, например, $0^i, 0^j$, автомат должен находиться в одном и том же состоянии.

Прочитав i или j нулей ДКА получает на вход 1. По прочтении i единиц он должен допустить вход, если ранее получил i нулей, и отвергнуть его, получив j нулей. Но в момент поступления 1, автомат не способен вспомнить какое число нулей i, j было принято. Следовательно, он может работать неправильно.

Пример 3. Нерегулярность языка $L = \{a^n b^n \mid n \geq 0\}$ над алфавитом $\Sigma = \{a, b\}$ можно показать следующим образом.

Пусть w, x, y, z, p , и i заданы соответственно формулировке леммы выше. Пусть w из L задаётся как $w = a^p b^p$. По лемме о накачке, существует разбиение $w = xyz$, где $|xy| \leq p$, $|y| \geq 1$, такое что $xy^i z$ принадлежит L для любого $i \geq 0$. Если допустить, что $|xy| = p$, а $|z| = p$, то xy — это первая часть w , состоящая из p последовательных экземпляров символа a .

Поскольку $|y| \geq 1$, она содержит по меньшей мере одну букву a , а $xy^2 z$ содержит больше букв a чем b . Следовательно, $xy^2 z$ не в языке L (заметим, что любое значение $i \neq 1$ даст противоречие). Достигнуто противоречие, поскольку в этом случае накачанное слово не принадлежит языку L . Предположение о регулярности L неверно и L — не регулярный язык.

Пример 1. Пусть КА (см. рис.) распознает строку **abcbcd**. Поскольку длина её превышает число состояний, существуют повторяющиеся состояния: q_1 и q_2 .

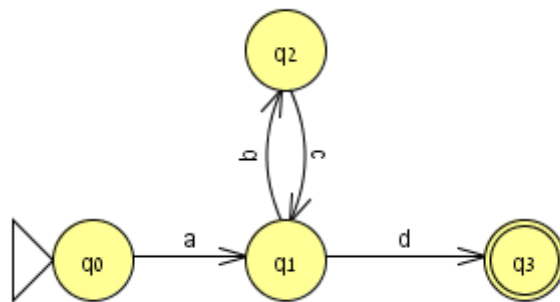


Рис. 2.3. Диаграмма переходов КА

Поскольку подстрока **bcbc** строки **abcbcd** проводит автомат по переходам из состояния q_1 обратно в q_1 , эту строку можно повторять сколько угодно раз, и КА всё равно будет её принимать, например строки **abcbcbcbcd** и **ad**.

В терминах леммы о накачке, строка **abcbcd** разбивается на часть $x = a$, часть $y = bc$ и часть $z = bcd$.

Заметим, что можно разбить её различными способами, например $x = a$, $y = bcbs$, $z = d$, и все условия будут выполнены, кроме $|xy| \leq p$, где p — длина накачки.

2.4. Эпсилон-переход: реализация по заданному регулярному выражению составного автомат, построение ДКА из НКА

Конфигурация автомата изменяется при такте срабатывания и выполнении функции перехода δ , при этом считывается символ с входной ленты. По *эпсилон-переходу* символ не считывается с входной ленты, но конфигурация изменяется.

Определение. Функция перехода δ без чтения символа называется *ε -переход* (*эпсилон-переход* или *эпсилон-такт*).

Определение. *ε -замыканием* состояния s_i называется множество состояний НКА, в которые из s_i можно попасть по цепочке ε -переходов. Как минимум, в это множество входит само s_i .

Определение. $move(Q, a)$ - функция переходов, **возвращает состояние**, в которое происходит переход из состояния $q \in Q$ при входном символе a .

ε -переход позволяет объединить нескольких автоматов в один. Так, например, два автомата, представленных на рис.2.5, можно соединить с помощью ε -переходов. В итоге получим автомат, представленный на рис.2.6.

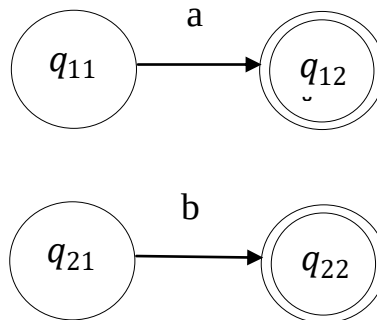


Рис. 2.5

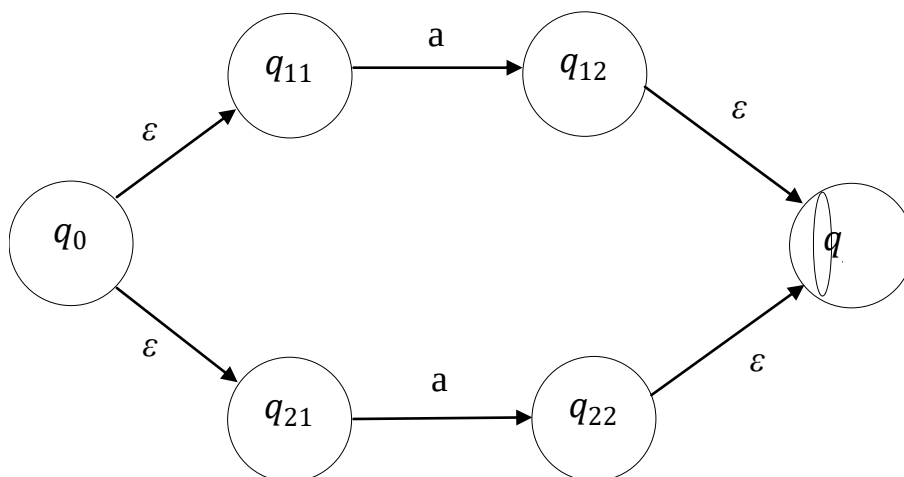


Рис. 2.6

Алгоритм. Построение ДКА из НКА: на основе таблицы переходов и очереди.

Таблица переходов:

В таблице переходов НКА каждая запись представляет собой множество состояний; в таблице переходов ДКА - единственное состояние.

Общая идея преобразования НКА в ДКА: каждому состоянию ДКА соответствует множество состояний НКА. ДКА использует свои состояния для отслеживания всех возможных состояний, в которых НКА может находиться после чтения очередного входного символа.

После чтения входной цепочки $a_1a_2..a$ ДКА находится в состоянии, подмножества состояний НКА, достижимых из начального состояния НКА по пути $a_1a_2..a$. Наихудший случай, встречающийся крайне редко, когда количество состояний ДКА экспоненциально зависит от количества состояний НКА.

Вход: НКА $N = (Q, \Sigma, \delta, q_0, F)$

Выход: ДКА $D = (Q', \Sigma, \delta', q_0', F')$, допускающий тот же язык.

Построим таблицу переходов $Dtran$ для ДКА. Каждое состояние ДКА является множеством состояний НКА, и мы строим $Dtran$ так, чтобы ДКА “параллельно” моделировал все возможные перемещения НКА по данной входной строке.

Для отслеживания множеств состояний НКА используем операции, приведенные на рис.2.7 (q представляет состояние НКА, а Q - множество состояний НКА).

Операция	Описание
$\varepsilon\text{-closure}(q)$ ($\varepsilon\text{-замыкание}(q)$)	Множество состояний НКА, достижимых из состояния q только по ε -переходам
$\varepsilon\text{-closure}(Q)$ ($\varepsilon\text{-замыкание}(Q)$)	Множество состояний НКА, достижимых из какого-либо состояния $q \in Q$ только по ε -переходам
$move(Q, a)$	Множество состояний НКА, в которые имеется переход из какого-либо состояния $q \in Q$ по входному символу a

Рис. 2.7. Операции над состояниями НКА

НКА может быть в любом из состояний множества $\varepsilon\text{-closure}(q_0)$, где q_0 - начальное состояние НКА. Предположим, что состояния множества Q , и только они, достижимы из q_0 после считывания входных символов, a - следующий входной символ. НКА может перейти в любое состояние из множества $move(Q, a)$. Совершив из этих состояний все возможные ε -переходы, НКА после обработки a может быть в любом состоянии из $\varepsilon\text{-closure}(move(Q, a))$.

Алгоритм. Определить множество состояний $Dstates$ ДКА и таблица переходов $Dtran$:

1. Каждое состояние ДКА соответствует множеству состояний НКА, в которых может находиться НКА после считывания входных символов, включая все возможные ε -переходы до и после считанных символов.

2. $q_0 = \varepsilon\text{-closure}(q_0)$.

3. Состояния и переходы добавляются в ДКА согласно следующему алгоритму:

Вход: НКА = $(Q, \Sigma, \delta, q_0, F)$.

Выход: ДКА = $(Q', \Sigma, \delta', q_0', F')$.

$Q' = \emptyset$

$\delta' = \emptyset$

$q_0' = q_0$

$\text{newStates} = \emptyset$ – множество состояний, достижимых из текущих состояний по терминальному символу, ε -переходам.

$\text{currStates} = \varepsilon\text{-closure}(q_0)$ – множество текущих состояний.

$Q' = Q' \cup \text{currStates}$

foreach ($a \in \Sigma$)

$\text{newStates} = \varepsilon\text{-closure}(\text{move}(\text{currStates}, a))$

if ($\text{newStates} \not\subseteq \emptyset$)

$\delta' = \delta' \cup \delta(\text{currStates}, a)$

if ($\text{newStates} \not\subseteq Q'$)

$Q' = Q' \cup \text{newStates}$

$\text{Dtran}[\text{currStates}, a] = \text{newStates}$

end

end

end foreach

4. Состояние ДКА является допустимым, если оно представляет собой множество состояний НКА, содержащих как минимум одно допускающее состояние НКА.

Вычисление $\varepsilon\text{-closure}(Q)$ является типичным процессом поиска графа для узлов, достижимых из данного множества узлов. В этом случае состояния Q представляют данное множество узлов, а граф состоит только из дуг НКА, помеченных ε .

Алгоритм вычисления $\varepsilon\text{-closure}(Q)$ использует множество nextStates для хранения состояний, дуги которых не были проверены на наличие ε -переходов.

ReachableStates – множество достижимых состояний Q' .

Вход: НКА $(Q, \Sigma, \delta, q_0, F)$, currStates

Выход: ReachableStates

$\text{ReachableStates} = \emptyset$

$\varepsilon\text{-closure}(\text{currStates}, \text{ReachableStates})$

$\text{nextStates} = \emptyset$

foreach ($q \in \text{currState}$)

foreach ($\delta'(q, \varepsilon) = \{q'\} \in \delta$)

```

    nextStates = nextStates  $\cup$  q'
  end foreach
  if (q  $\notin$  ReachableStates)
    ReachableStates = ReachableStates  $\cup$  q
  end
  if (nextStates  $\neq \emptyset$ )
    foreach (n  $\in$  nextStates)
      ReachableStates = ReachableStates  $\cup$  n
    end foreach
  end
end
if (nextStates  $\neq \emptyset$ )
   $\epsilon$ -closure(nextStates, ReachableStates)
end

```

Пример. Дан конечный автомат с $Q = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

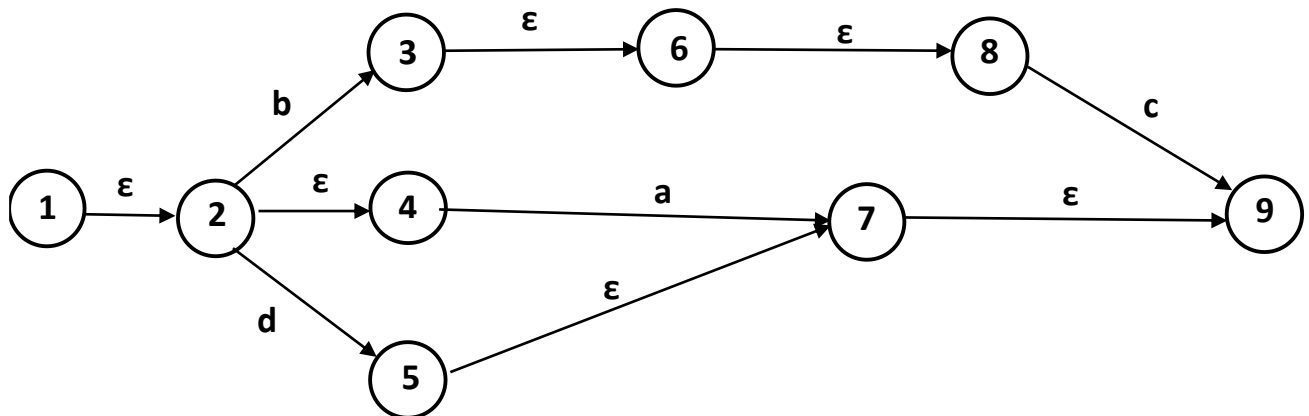


рис. 2.8

Получим ϵ -closure(Q) = {1, 2, 4, 6, 7, 8, 9}.

Определение заключительных состояний ДКА

Вход: Q' , F .

Выход: F'

$F' = \emptyset$

```

foreach (f  $\in$  F)
  foreach (q'  $\in$  Q')
    if (q'  $\in$  f)
       $F' = F' \cup f$ 
    end
  end foreach
end foreach

```

Пример. Задан недетерминированный конечный автомат $\Sigma = \{0, 1, 2, \}$:

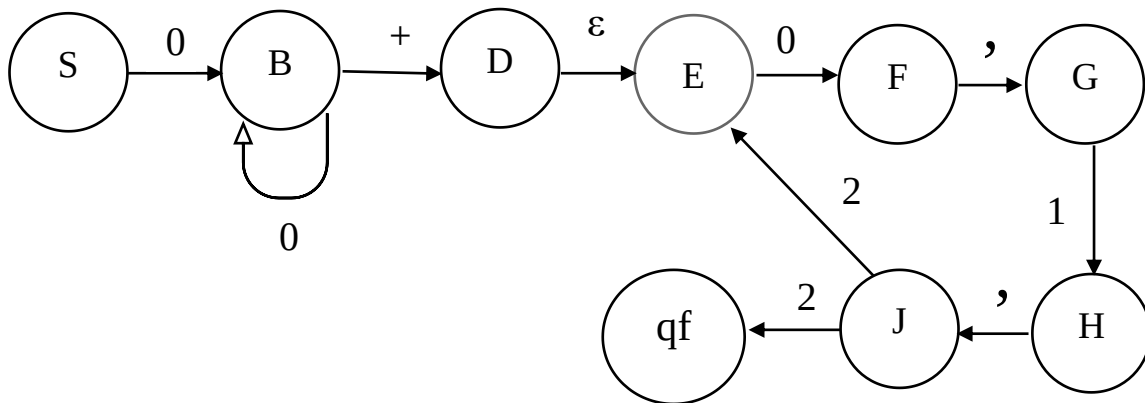


Рис. 2.9

q_0 эквивалентного ДКА $= \varepsilon\text{-closure}(S)$, т.е. $S' = \{S\}$, именно эти состояния достижимы из состояния S путями, в которых каждая дуга помечена ε . Заметим, что путь может и не иметь дуги, так что состояние S также достигается из состояния S , а значит, входит в искомое множество.

Вычислим $\varepsilon\text{-closure}(\text{move}(S', 0))$. $\text{move}(S', 0)$, множество состояний, имеющих переходы по символу 0 для элементов множества S' . Из состояний S только S имеет такой переход к состоянию B , $\varepsilon\text{-closure}(\text{move}(\{S\}, 0)) = \varepsilon\text{-closure}(\{B\}) = \{B\}$, $D\text{tran}[S', 0] = B'$.

Для множества B : $\varepsilon\text{-closure}(\text{move}(B', 0))$. Вычислим $\text{move}(B', 0)$, множество состояний, имеющих переходы по символу 0 для элементов множества B' . Из состояний B только B имеет такой переход к состоянию B ,

$$\varepsilon\text{-closure}(\text{move}(\{B\}, 0)) = \varepsilon\text{-closure}(\{B\}) = \{B\}, D\text{tran}[B', 0] = B'$$

$$\varepsilon\text{-closure}(\text{move}(B', +))$$

Вычислим $\text{move}(B', +)$, множество состояний, имеющих переходы по символу $+$ для элементов множества B' . Из состояний B только B имеет такой переход к состоянию D ,

$$\varepsilon\text{-closure}(\text{move}(\{B\}, +)) = \varepsilon\text{-closure}(\{D\}) = \{D, E\}, D\text{tran}[B', +] = D' E'$$

$\varepsilon\text{-closure}(\text{move}(D' E', 0))$. Вычислим $\text{move}(D' E', 0)$, множество состояний, имеющих переходы по символу 0 для элементов множества $D' E'$. $\varepsilon\text{-closure}(\text{move}(\{D, E\}, 0)) = \varepsilon\text{-closure}(\{F\}) = \{F\}$. Таким образом, $D\text{tran}[D' E', 0] = F$

Продолжая этот процесс, в конечном итоге все множества-состояния ДКА окажутся помеченными. Имеем восемь множеств состояний:

$S' = \{S\}$ $B' = \{B\}$ $D' E' = \{D, E\}$ $F' = \{F\}$ $G' = \{G\}$ $H' = \{H\}$ $J' = \{J\}$
 $E' qf' = \{qf, E\}$, где S' начальное состояние, qf' - единственное заключительное состояние.

Таблица переходов $D\text{tran}$

Состояние	Входной символ				
	0	1	2	,	+
S'	B'				

B'	B'				D'E'
D'E'	F'				
F'				G'	
G'		H'			
H'				J'	
J'			qf'		
E'qf'	F'				

На основе очереди:

Шаг 1. Помещаем в очередь Q множество, состоящее только из стартовой вершины.

Шаг 2. Пока очередь не пуста выполняем следующие действия:

- Берем из очереди множество q
- Для всех $c \in \Sigma$ посмотрим в какое состояние ведет переход по символу c из каждого состояния в q . Полученное множество состояний положим в очередь Q если его не было ранее в очереди. Каждое такое множество в итоговом ДКА будет отдельной вершиной, в которую будут вести переходы по соответствующим символам.
- Если в множестве q хотя бы одна из вершин была терминальной в НКА, то соответствующая данному множеству вершина в ДКА также будет терминальной.

Вход: НКА = $(Q, \Sigma, \delta, q_0, F)$.

Выход: ДКА = $(Q', \Sigma, \delta', q_0', F')$.

$Q' = \emptyset$

$\delta' = \emptyset$

$q_0' = q_0$

Queue = $\emptyset$ // Очередь для обхода уникальных состояний

currStates = $\emptyset$ // Мн-во состояний из очереди

newStates = $\emptyset$ // промежуточное мн-во состояний из текущего по символу

Queue = Queue $\cup$ q_0

while (Queue $\neq \emptyset$)

currStates = Queue.pop()

currStates = currStates $\cup$ ϵ -closure(currStates)

foreach ($a \in \Sigma$)

newStates = move (currStates, a)

if (ϵ -closure(newStates) $\notin Q'$)

Queue = Queue $\cup$ newStates

$Q' = Q' \cup \epsilon$ -closure(newStates)

end

if (newStates $\neq \emptyset$)

$\delta' = \delta' \cup \delta$ (currStates, a)

end

end foreach

end while

Пример. Задан недетерминированный конечный автомат $\Sigma = \{0, 1, 2, ,\}$ (рис.2.9).

Шаги алгоритма для данного примера:

delta δ' – добавленный переход

Шаг итерации: 0

Q': $\emptyset$

Queue: S

Cur: S

delta δ' : $\delta(S, 0)$

Шаг итерации: 1

Q': S

Queue: B

Cur: B

delta δ' : $\delta(B, +), \delta(B, 0)$

Шаг итерации: 2

Q': S, B

Queue: D

Cur: DE

delta δ' : $\delta(DE, 0)$

Шаг итерации: 3

Q': S, B, DE

Queue: F

Cur: F

delta δ' : $\delta(F, ,)$

Шаг итерации: 4

Q': S, B, DE, F

Queue: G

Cur: G

delta δ' : $\delta(G, 1)$

Шаг итерации: 5

Q': S, B, DE, F, G

Queue: H

Cur: H

delta δ' : $\delta(H, ,)$

Шаг итерации: 6

Q': S, B, DE, F, G, H

Queue: J

Cur: J

delta δ' : $\delta(J, 2)$ – переход в состояние qfE

Шаг итерации: 7

Q' : S, B, DE, F, G, H, J, qfE

Queue: qfE

Cur: F

delta δ' : $\delta(qfE, 0)$ – переход в состояние F

Шаг итерации: 8 // F не добавляем, так как оно уже есть в мно-ве

Q' : S, B, DE, F, G, H, J, qfE

Queue: $\emptyset$

Cur: $\emptyset$

delta δ' : -

Очередь пуста. Алгоритм завершён.

Результат:

Q' : S, B, DE, F, G, H, J, qfE

δ' :

$\delta(S, 0)$

$\delta(B, 0)$

$\delta(B, +)$

$\delta(DE, 0)$

$\delta(F, ,)$

$\delta(G, 1)$

$\delta(H, ,)$

$\delta(J, 2)$

$\delta(qfE, 0)$

F' : qfE

Оба алгоритма выдали одинаковый результат.

Граф переходов ДКА имеет вид:

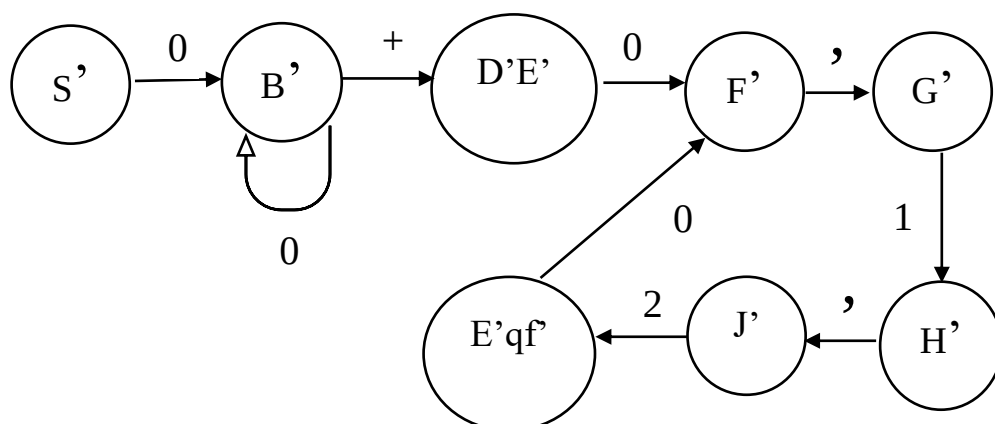
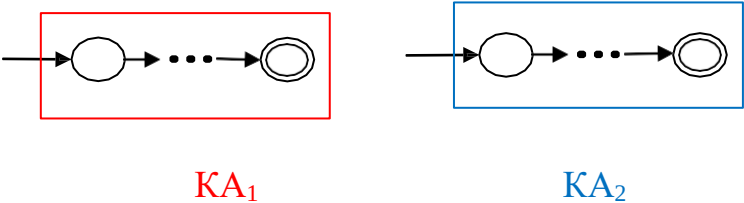


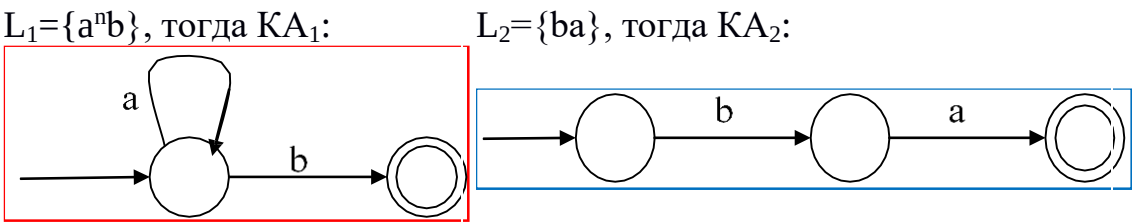
рис. 2.10

2.5. Составные конечные автоматы и регулярные языки

Пусть L_1 и L_2 регулярные языки, тогда $L(KA_1) = L_1$ и $L(KA_2) = L_2$.



Рассмотрим для примера:



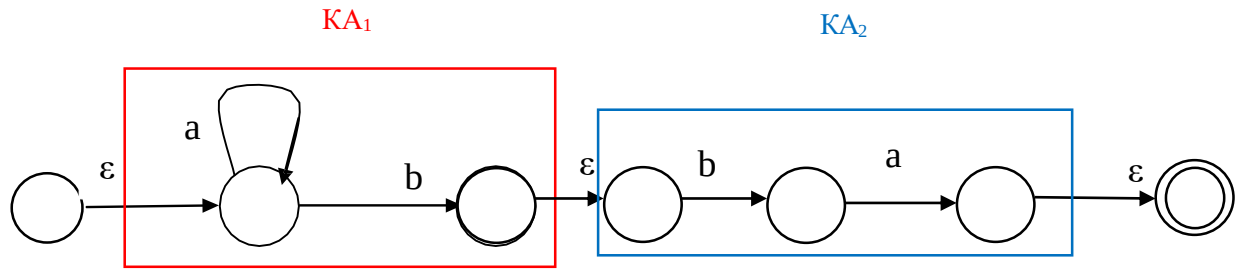
Для двух регулярных языков L_1 и L_2 справедливо следующее:

Операция	Доказательство замкнутости относительно операции
Объединение: $L_1 \cup L_2$	$L_1 \cup L_2$ является регулярным по определению регулярных языков.

Недетерминированный конечный автомат $L_1 \cup L_2 = \{a^n b\} \cup \{ba\}$

Конкатенация: $L_1 + L_2$

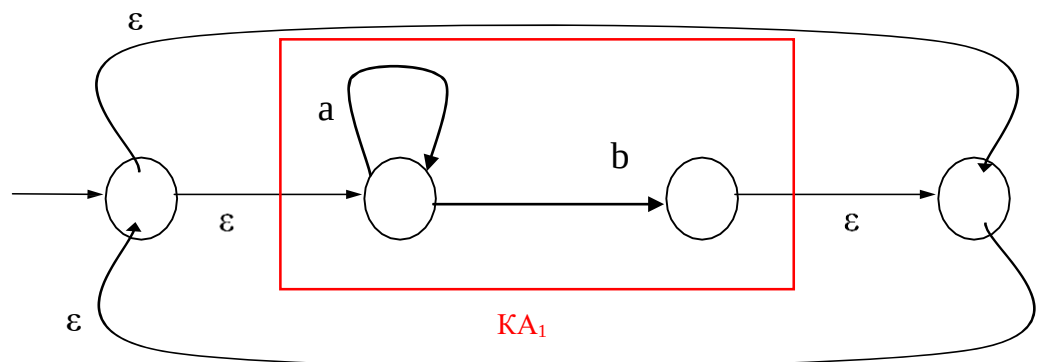
$L_1 + L_2$ является регулярным по определению **регулярных языков**.



Недетерминированный конечный автомат $L_1 + L_2 = \{a^n b\} + \{ba\} = \{a^n bba\}$

Замыкание Клини: L_1^*

L_1^* является регулярным по определению **регулярных языков**

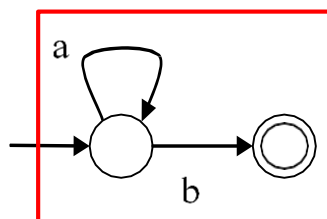


Недетерминированный конечный автомат $L_1^* = \{a^n b\}^*$

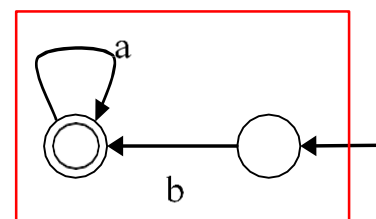
Реверс: L_1^R

Рассмотрим НКА с ϵ -переходами $A = \langle \Sigma, Q_1, s', \{s_1\}, \delta'_1 \rangle$, где $\delta'_1(v, c) = \{u \mid \delta_1(u, c) = v\}$; $\delta'_1(s', \epsilon) = \{T_1\}$. Если в исходном автомате путь по α из s_1 приводил в терминальное состояние, то в новом автомате существует путь по α из этого терминального состояния в s_1 (и наоборот). Следовательно, этот автомат распознает в точности развернутые слова языка L_1 . Тогда язык L_1^R — регулярный.

1. Отменить все переходы
2. Сделать начальное состояние конечным и наоборот



KA_1



KA_1^R

Недетерминированный конечный автомат $L_1 = \{a^n b\}$, $L_1^R = \{b a^n\}$

Дополнение: $\underline{L}_1$	Рассмотрим автомат $A = \langle \Sigma, Q_1, s_1, Q_1 \setminus T_1, \delta_1 \rangle$, то есть автомат A, в котором терминальные и нетерминальные состояния инвертированы (при таком построении следует помнить, что, если в исходном автомате было опущено состояние, его нужно явно добавить и сделать допускающим.) Очевидно, A допускает те и только те слова, которые не допускает автомат A, а значит, задаёт язык $\underline{L}_1$. Таким образом, $\underline{L}_1$ — регулярный.
1. Пусть KA_1 допускает L_1 2. Сделать конечные состояния не конечными и наоборот KA_1	
Недетерминированный конечный автомат $\bar{L}_1 = \{a, b\}^* - \{a^n b\}$	
Пересечение: $L_1 \cap L_2$	$L_1 \cap L_2 =$ дополнение $(L_1 \cup L_1)$. Тогда $L_1 \cap L_2$ — регулярный. Автомат для пересечения языков строится явно произведением автоматов.

Результат операций есть регулярный язык. Регулярные языки замкнуты над этими операциями.

Алгоритмы для операций над регулярными языками

Алгоритм 1. Конкатенация двух автоматов.

Вход: $KA_1 = (Q_1, \Sigma_1, \Delta_1, q_{01}, F_1 = \{qf_{1.1}, \dots, qf_{1.n}\})$,
 $KA_2 = (Q_2, \Sigma_2, \Delta_2, q_{02}, F_2 = \{qf_{2.1}, \dots, qf_{2.n}\})$
Выход: $KA = KA_1 + KA_2 = Q_1 \cup Q_2$
 $\Sigma = \Sigma_1 \cup \Sigma_2$
 $F = F_2$
 $q_0 = q_{01}$
 $\Delta = \emptyset$
foreach ($\sigma \in \Delta_1$)
 $\Delta = \Delta \cup \sigma$
end foreach
foreach ($\sigma(q, a) = A \in \Delta_2$)
 $\Delta = \Delta \cup \sigma(q, a) = A$
 if ($q = q_{02}$)
 foreach ($qf_1 \in F_1$)
 $\Delta = \Delta \cup \sigma(qf_1, a) = A$
 end foreach
 end foreach
end foreach

Алгоритм 2. Конкатенация n конечных автоматов.

Вход: $FS = \{KA_1, KA_2, \dots, KA_n\}$
Выход: $KA = KA_1 + KA_2 + \dots + KA_n$
 $KA = KA_1$
foreach ($KA_i \in FS; i > 1$)
 $KA = KA + KA_i$ // применяем алгоритм конкатенации 2х автоматов
end foreach

Алгоритм 3. Объединение 2х автоматов

Вход: $KA_1 = (Q_1, \Sigma_1, \Delta_1, q_{01}, F_1)$, $KA_2 = (Q_2, \Sigma_2, \Delta_2, q_{02}, F_2)$
Выход: $KA = KA_1 \cup KA_2 = (Q, \Sigma, \Delta, q_0, F)$
 $q_0 = S$ // S - новый начальный нетерминал
 $Q = Q_1 \cup Q_2 \cup q_0$
 $\Sigma = \Sigma_1 \cup \Sigma_2, F = F_1 \cup F_2, \Delta = \emptyset$
 $\Delta = \Delta \cup \{\sigma(q_0, \varepsilon) = q_{01}, \sigma(q_0, \varepsilon) = q_{02}\}$
 $\Delta = \Delta \cup \Delta_1 \cup \Delta_2$
 $KA = (Q, \Sigma, \Delta, q_0, F)$

Алгоритм 4. Объединение n конечных автоматов

Вход: $FS = \{KA_1, KA_2, \dots, KA_n\}$
 Выход: $KA = KA_1 \cup KA_2 \cup \dots \cup KA_n$
 $KA = KA_1$
 foreach($KA_i \in FS; i > 1$)
 $KA = KA \cup KA_i$ // применяем алгоритм объединения 2х автоматов
 end foreach

Примеры использования составных автоматов

Даны регулярные языки:

$L = \{0(000)^*(0+1) \omega_1 \mid \omega_1 \in \{0,1\}^*\},$

$L_1 = \{0(000)^*\},$ где $(000)^* = \{000\ 000000\ 0000000000\ \dots\}$

$L_{1.1} \subset L_1 = \{0000, 00000000, 00000000000, 0000000000000\}$

$L_2 = \{(0+1)\}$

$L_3 = \{\omega_1 \in \{0,1\}^*\},$ где $\{0,1\}^* = \{\{\epsilon\}, \{0\}, \{1\}, \{00\}, \{10\}, \{01\}, \{11\}, \dots\}$

$L_{3.1} \subset L_3 = \{\epsilon, 0, 1, 01\}$

Правила грамматики языка L_1 :

1. $S_{01} \rightarrow 0A_1$

2. $A_1 \rightarrow 0B_1$

3. $B_1 \rightarrow 0C_1$

4. $C_1 \rightarrow 0A_1$

5. $C_1 \rightarrow 0$

$G_1 = (\{0\}, \{S_{01}, A_1, B_1, C_1\}, P = \{p_1, p_2, p_3, p_4, p_5\}, S_{01})$

Правила грамматики языка L_2 :

1. $S_{02} \rightarrow (A_2$

2. $A_2 \rightarrow 0B_2$

3. $B_2 \rightarrow +C_2$

4. $C_2 \rightarrow 1D_2$

5. $D_2 \rightarrow 0$

$G_2 = (\{0, 1, +, (,)\}, \{S_{02}, A_2, B_2, C_2, D_2\}, P = \{p_1, p_2, p_3, p_4, p_5\}, S_{02})$

Правила грамматики языка L_3 (замыкание Клини):

1. $S_{03} \rightarrow \epsilon$

2. $S_{03} \rightarrow 0$

3. $S_{03} \rightarrow 1$

4. $S_{03} \rightarrow 0S_{03}$

5. $S_{03} \rightarrow 1S_{03}$

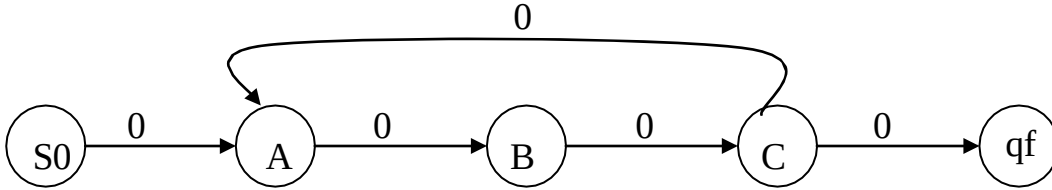
$G_3 = (\{0,1\}, \{S_{03}\}, P = \{p_1, p_2, p_3, p_4, p_5\}, S_{03})$

Построим конечные автоматы, таблицы и схемы к ним:

$$\mathbf{KA}_1 = (\{S_{01}, A_1, B_1, C_1\}, \{0\}, \sigma, S_{01}, \{q_{f1}\})$$

1. $\sigma_1(S_{01}, 0) = \{A_1\}$
2. $\sigma_1(A_1, 0) = \{B_1\}$
3. $\sigma_1(B_1, 0) = \{C_1\}$
4. $\sigma_1(C_1, 0) = \{A_1, q_{f1}\}$

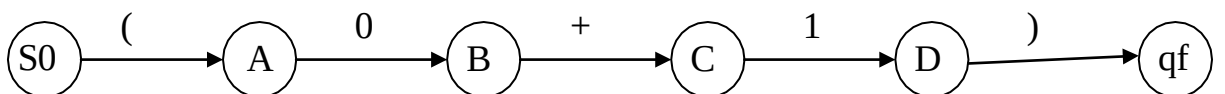
σ_1	0
S01	{A ₁ }
A ₁	{B ₁ }
B ₁	{C ₁ }
C ₁	{B ₁ , q _{f1} }
qf1	



$$\mathbf{KA}_2 = (\{S_{02}, A_2, B_2, C_2, D_2\}, \{0,1,(,),+\}, \sigma, S_{02}, \{q_{f2}\})$$

1. $\sigma_2(S_{02}, () = \{A_2\}$
2. $\sigma_2(A_2, 0) = \{B_2\}$
3. $\sigma_2(B_2, +) = \{C_2\}$
4. $\sigma_2(C_2, 1) = \{D_2\}$
5. $\sigma_2(D_2,)) = \{q_{f2}\}$

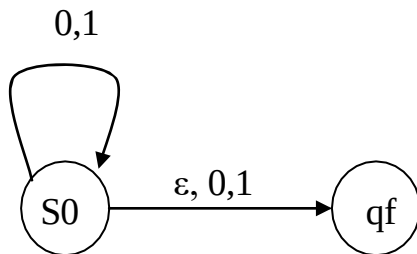
σ_2	0	1	(	)	+
S02			{A ₂ }		
A ₂	{B ₂ }				
B ₂					{C ₂ }
C ₂		{D ₂ }			
D ₂				{q _{f2} }	
qf2					



$$KA_3 = (\{S_{03}\}, \{0,1\}, \sigma, S_{03}, \{q_{f3}\})$$

1. $\sigma_3(S_{03}, \varepsilon) = \{q_{f3}\}$
2. $\sigma_3(S_{03}, 0) = \{S_{03}, q_{f3}\}$
3. $\sigma_3(S_{03}, 1) = \{S_{03}, q_{f3}\}$

σ_2	0	1	ε
S 03	{S03, qf3}	{S03, qf3}	{ qf3}
qf 3			

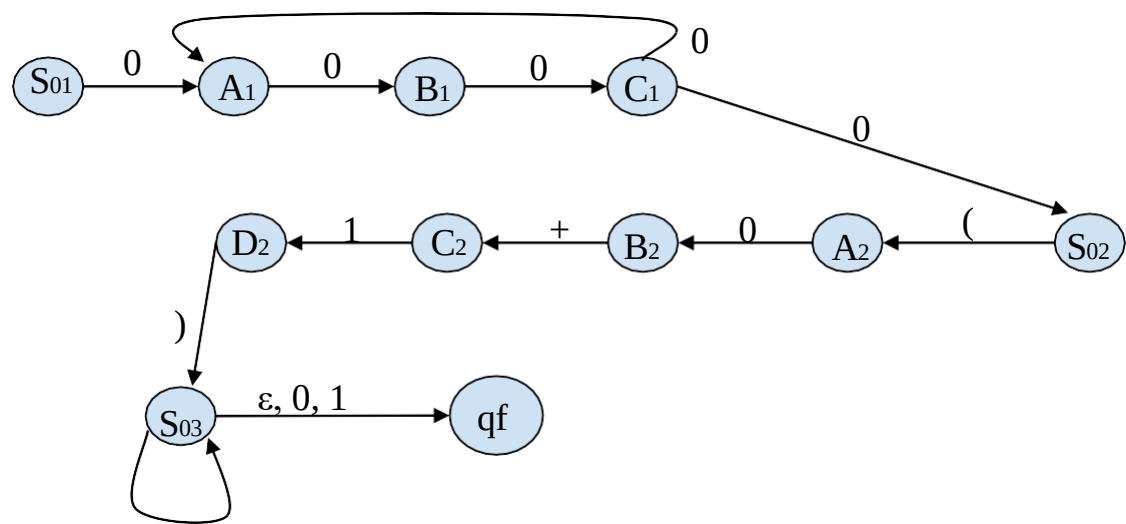


Применим рассмотренные операции к языкам:

1) Конкатенация $L_1+L_2+L_3$

$$KA_{L_1+L_2+L_3} = KA_L = (\{S_{01}, A_1, B_1, C_1, S_{02}, A_2, B_2, C_2, D_2, S_{03}\}, \{0,1,(,),+\}, \sigma, S_{01}, \{q_{f3}\})$$

	σ	0	1	$($	$)$	$+$	ε
L₁	S ₀₁	{ A ₁ }					
	A ₁	{ B ₁ }					
	B ₁	{ C ₁ }					
	C ₁	{ A ₁ , S ₀₂ }					
L₂	S ₀₂			{ A ₂ }			
	A ₂	{B ₂ }					
	B ₂					{C ₂ }	
	C ₂		{D ₂ }				
	D ₂				{S ₀₃ }		
L₃	S ₀₃	{S ₀₃ , q _{f3} }	{S ₀₃ , q _{f3} }				{q _{f3} }
	q _{f3}						

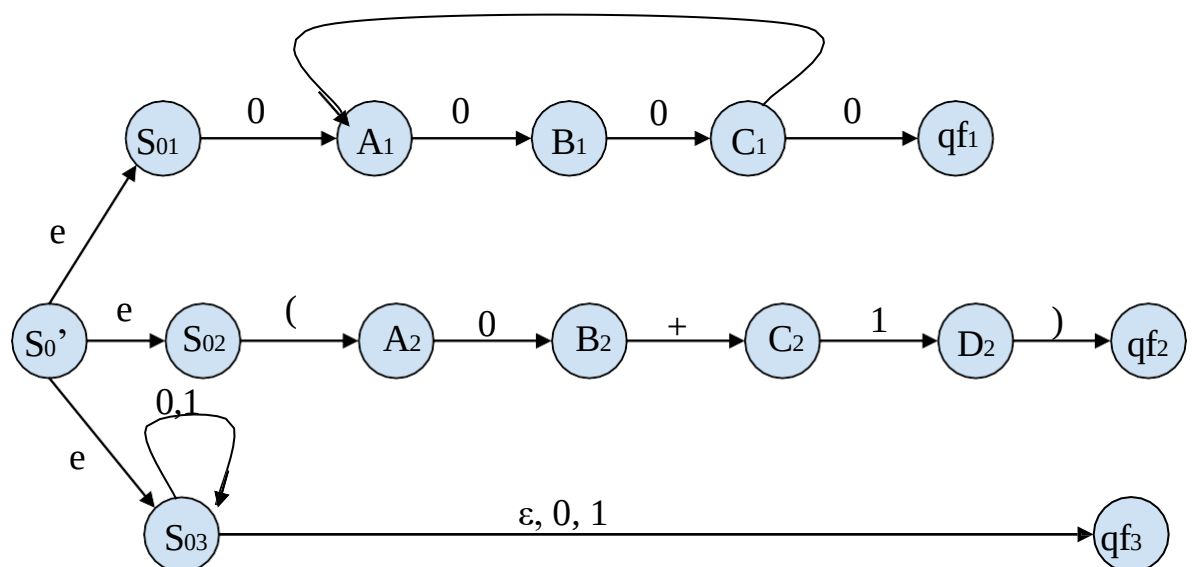


2) Объединение $L_1 \cup L_2 \cup L_3$

$KA_{L_1 \cup L_2 \cup L_3} = (\{S_0, S_{01}, A_1, B_1, C_1, S_{02}, A_2, B_2, C_2, D_2, S_{03}\}, \{0, 1, (, +\}, \sigma, S_0, \{q_{f1}, q_{f2}, q_{f3}\})$

1.

	Σ	0	1	$($	$)$	$+$	ε
	S_0						$\{S_{01}, S_{02}, S_{03}\}$
L_1	S_{01}	$\{A_1\}$					
	A_1	$\{B_1\}$					
	B_1	$\{C_1\}$					
	C_1	$\{A_1, q_{f1}\}$					
	q_{f1}						
L_2	S_{02}			$\{A_2\}$			
	A_2	$\{B_2\}$					
	B_2					$\{C_2\}$	
	C_2		$\{D_2\}$				
	D_2				$\{q_{f2}\}$		
	q_{f2}						
L_3	S_{03}	$\{S_{03}, q_{f3}\}$	$\{S_{03}, q_{f3}\}$				$\{q_{f3}\}$
	q_{f3}						



2.6. Язык регулярных выражений на основе паттернов

Язык регулярных выражений на основе паттернов - это узкоспециализированный язык программирования, например для обработки текста. Примерами такого языка являются язык паттернов встроенный в C#.NET [1], Python [2] и другие языки программирования. Паттерны регулярных выражений определяются специальным синтаксисом или языком, который совместим с регулярными выражениями Perl 5 [3] и добавляет некоторые дополнительные функции. Этот язык относительно невелик и ограничен, поэтому не все возможные задачи обработки строк могут быть выполнены с использованием регулярных выражений.

Паттерн язык предоставляет мощный, гибкий и эффективный метод обработки текста. Нотация сопоставления текста с паттерном позволяет быстро анализировать большие объемы текста, чтобы:

- Найти определенные паттерны символов.
- Проверить текст, чтобы убедиться, что он соответствует предварительно определенному паттерну (например, адресу электронной почты).
- Извлечь, редактировать, заменить или удалить текстовые подстроки.
- Добавить извлеченные строки в коллекцию, чтобы сгенерировать отчет.

Примеры регулярных выражений

Поддерживается синтаксис немного отличный от PCRE (Perl Compatible Regular Expressions) [3]. Регулярные выражения по умолчанию чувствительны к регистру.

Следующие примеры иллюстрируют использование и построение простых регулярных выражений. Каждый пример включает тип текста для сопоставления, одно или несколько регулярных выражений, соответствующих этому тексту, и примечания, поясняющие использование **специальных** символов и форматирование.

Предметные области применения паттернов



Перечисление операционных символов

Опера ционн ый симво л	Паттерн	Описание
{a ₁ }	\W	Проверка на соответствие любому символу, который не является буквой, цифрой или символом подчеркивания.
{a ₂ }	p	Проверка, что символ равен 'p'
{a ₃ }	o	Проверка, что символ равен 'o'
{a ₄ }	[#-]	Проверка, что символ равен '#' или '-'
{a ₅ }	\s	Проверка, что символ пробельный
{a ₆ }	\d	Проверка, что символ является цифрой
{a ₇ }	[s-]	Проверка, что символ пробельный или равен '-'
{a ₈ }	s	Проверка, что символ равен 's'
{a ₉ }	t	Проверка, что символ равен 't'
{a ₁₀ }	c	Проверка, что символ равен 'c'
{a ₁₁ }	k	Проверка, что символ равен 'k'
	i	Проверка, что символ равен 'i'
{a ₁₂ }	\b	Поиск границы слова
{a ₁₃ }	M	Проверка, что символ равен 'M'
{a ₁₄ }	\w	Проверка, что символ является буквой, цифрой или равен '_'
{a ₁₅ }	(?i)(baloney darn drat fooey gosh\s darnit heck)	Проверка, что цепочка начинается со слова из множества {baloney, darn, drat, fooey, gosh, darnit, heck} независимо от регистра
{a ₁₆ }	g	Проверка, что символ равен 'g'
{a ₁₇ }	[i!1]	Проверка, что символ принадлежит множеству {I, !, 1}
{a ₁₈ }	[r@]	Проверка, что символ принадлежит множеству {r, @}
{a ₁₉ }	a	Проверка, что символ равен 'a'
{a ₂₀ }	[vi@a]	Проверка, что символ принадлежит множеству {v, I, @, a}
{a ₂₁ }	[\w.\-]	Проверка, что символ является буквой, цифрой или равен '_', '.' или '-'
{a ₂₂ }	@	Проверка, что символ равен '@'
{a ₂₃ }	(yahoo hotmail gmail)	Проверка, что цепочка начинается со слова из множества {yahoo, hotmail, gmail}
{a ₂₄ }	.	Проверка, что символ равен '.'

{a ₂₅ }	m	Проверка, что символ равен 'm'
{a ₂₆ }	((\w+([-+.\w+)*@\w+([-.\w+)*\.\w+([-.\w+)*]\s*[;]{0,1}\s*))	Проверка, что слово начинается с одного или нескольких слов, разделенных символами +, - или ., затем символ @, затем одно или несколько слов, разделенных символом ;, затем символ ., и наконец одно или несколько слов, разделенных символом ;
{a ₂₇ }	^	Проверка, что символ равен '^'
{a ₂₈ }	\$	Проверка, что символ равен '\$'
{a ₂₉ }	((\w+([-+.\w+)*@\w+([-.\w+)*\.\w+([-.\w+)*]\s*[;]{0,1}\s*))	Проверка, что слово начинается с одного или нескольких слов, разделенных символами +, - или ., затем символ @, затем одно или несколько слов, разделенных символами - или ., затем символ ., и наконец одно или несколько слов, разделенных символами - или ..
{a ₃₀ }	((\w+([-+.\w+)*@\w+([-.\w+)*\.\w+([-.\w+)*]\s*[;,]{0,1}\s*))	Проверка, что слово начинается с одного или нескольких слов, разделенных символами +, - или ., затем символ @, затем одно или несколько слов, разделенных символами - или ., затем символ ., и наконец одно или несколько слов, разделенных символами - или ..
{a ₃₁ }	1	Проверка, что символ равен '1'
{a ₃₂ }	9	Проверка, что символ равен '9'
{a ₃₃ }	2	Проверка, что символ равен '2'
{a ₃₄ }	6	Проверка, что символ равен '6'
{a ₃₅ }	8	Проверка, что символ равен '8'
{a ₃₆ }	\\	Проверка, что символ равен '\'
{a ₃₇ }	[^0-9a-zA-Z]	Проверка, что символ является буквой (независимо от регистра) или цифрой
{a ₃₈ }	[a-z]	Проверка, что символ является буквой в нижнем регистре
{a ₃₉ }	\+	Проверка, что символ равен '+'
{a ₄₀ }	\-	Проверка, что символ равен '-'
{a ₄₁ }	[A-Z]	Проверка, что символ является буквой в верхнем регистре
{a ₄₂ }	[a-zA-Z]	Проверка, что символ является буквой (независимо от регистра)

{a ₄₃ }	\s[a-zA-Z]+	Проверка, что цепочка начинается с пробельного символа и буквы. При этом количество повторений может быть больше одного
{a ₄₄ }	\s[azA-Z]+	Проверка, что цепочка начинается с пробельного символа и буквы верхнего регистра или 'a', 'z'. При этом количество повторений может быть больше одного
{a ₄₅ }	\D	Проверка, что символ не является цифрой

1. Точное совпадение с фразой

Входная строка	Операции	Набор операционных символов
stock tips	{a ₁ }, {a ₂ }, {a ₈ }, {a ₉ }, {a ₃ }, {a ₁₀ }, {a ₁₁ }, {a ₅ }, {a ₁₂ }	{a ₁ }{a ₈ }{a ₉ }{a ₃ }{a ₁₀ }{a ₁₁ }{a ₅ }{a ₉ }{a ₁₂ }{a ₂ }{a ₈ }{a ₁ }
stock tips	{a ₁ }, {a ₈ }, {a ₉ }, {a ₃ }, {a ₁₀ }, {a ₁₁ }, {a ₅ }, {a ₁₂ }, {a ₂ }	{a ₁ }{a ₈ }{a ₉ }{a ₃ }{a ₁₀ }{a ₁₁ }{a ₅ {0,3}}{a ₉ }{a ₁₂ }{a ₂ }{a ₈ }{a ₁ }
stock tip	{a ₁ }, {a ₈ }, {a ₉ }, {a ₃ }, {a ₁₀ }, {a ₁₁ }, {a ₅ }, {a ₁₂ }, {a ₂ }	{a ₁ }{a ₈ }{a ₉ }{a ₃ }{a ₁₀ }{a ₁₁ }{a ₅ {0,3}}{a ₉ }{a ₁₂ }{a ₂ }{a ₈ {0,1}}{a ₁ }
Mabbbbbbb	{a ₁₃ }, {a ₁₄ }, {a ₁₅ }	{a ₁₃ }{a ₁₄ }{a ₁₅ {1}}

Пример	Совпадение с фразой "stocks tips"
Pattern 1	{a ₁ }stock\stips(\W \$)
Pattern 2	{a ₁ }stock\s{0,3}tips(\W \$)
Pattern 3	(\W ^)stock\s{0,3}tip(s){0,1}(\W \$)
Pattern 4	\b[M]\w+
Примечания	\W соответствует любому символу, который не является буквой, цифрой или символом подчеркивания. Это предотвращает регулярное выражение из совпадающих символов до или после фразы. - В примере 2 \s соответствует символу пробела, а {0,3} указывает, что от 0 до 3 между словами stock и tip могут быть пробелы. ^ соответствует звездочке новой строки. Позволяет регулярному выражению соответствовать фразе, если она появляется в начале строки без символов перед ним. \$ соответствует концу строки. Позволяет регулярному выражению соответствовать фразе, если она появляется в конце строки без символов после него. - В примере 3 (s) соответствует букве s, а {0,1} указывает, что буква может встречаться 0 или 1 раз после слова tip. Следовательно, регулярное выражение соответствует stock tip и stock tips. - Кроме того, вы можете использовать символ ? вместо {0,1}

2. Сопоставьте слово или фразу в списке

Входная строка	Операции	Набор операционных символов
baloney	{a ₁ }, {a ₁₆ }	{a ₁ }{a ₁₆ }{a ₁ }

Pattern 1	(?i)(\W ^)(baloney darn drat foeey gosh\s darnit heck)(\W \$)
Пример	Сопоставьте слово или фразу из следующего списка: • baloney • darn • drat • foeey • gosh darnit • heck

3. Искажение слова: сопоставление слов с разными вариантами написания, включая специальные символы

Входная строка	Операции	Набор операционных символов
gra@via	{a ₁₇ }, {a ₁₈ }, {a ₂₀ }, {a ₂₁ }	{a ₁₇ }{a ₁₈ }{a ₂₀ }{a ₂₁ }

Pattern 1	g[i!1][r@]a[vi@a]
Пример	Слово "гравий" искажено. Искажение используют спамеры. Например: • gra@via • grav1ia • gr@av1ia • gra@via
Примечание	• \W не включен, так что другие символы могут стоять до или после любого из вариантов "гравий". Например, регулярное выражение по-прежнему соответствует "гравий" в следующем тексте: gravia!! or ***gravia***

4. Совпадение с адресом электронной почты из определенного домена. Проверка нескольких писем

Входная строка	Операции	Набор операционных символов
example.email@gmail.com	{a ₁ }, {a ₂₂ }, {a ₂₃ }, {a ₂₄ }, {a ₂₅ }, {a ₁₀ }, {a ₃ }, {a ₂₆ }	{a ₁ }{a ₂₂ {0,25}}{a ₂₃ }{a ₂₄ }{a ₂₅ }{a ₁₀ }{a ₃ }{a ₂₆ }{a ₁ }
example.email@gmail.com; another.email@yahoo.com	{a ₂₈ }, {a ₂₇ }, {a ₂₉ }	{a ₂₈ }{a ₂₇ {1}}{a ₂₉ }
example.email@gmail.com, another.email@yahoo.com	{a ₂₈ }, {a ₃₀ }, {a ₂₉ }	{a ₂₈ }{a ₃₀ {1}}{a ₂₉ }
example.email@gmail.com; another.email@yahoo.com, third.email@hotmail.com	{a ₂₈ }, {a ₃₁ }, {a ₂₉ }	{a ₂₈ }{a ₃₁ {1}}{a ₂₉ }

Pattern 1	(\W ^)[\w.\-]{0,25}@(yahoo hotmail gmail)\.com(\W \$)
Pattern 2	^((\w+([-+.\w+)*@\w+([-.\w+)*\.\w+([-.\w+)*\s*[,;]{0,1}\s*)+\$
Pattern 3	^((\w+([-+.\w+)*@\w+([-.\w+)*\.\w+([-.\w+)*\s*[,;]{0,1}\s*)+\$
Pattern 4	^((\w+([-+.\w+)*@\w+([-.\w+)*\.\w+([-.\w+)*\s*[,;.]{0,1}\s*)+\$
Пример	Совпадение с адресом электронной почты из доменов yahoo.com, hotmail.com и gmail.com
Примечания	• \W соответствует любому символу, который не является буквой, цифрой или символом подчеркивания. Это предотвращает регулярное выражение из совпадающих символов до или после адреса электронной почты. • ^ соответствует новой строке. Позволяет регулярному выражению сопоставляться с адресом, если он появляется в начале строки без символов перед ним. • \$ соответствует концу строки. Позволяет регулярному выражению сопоставляться с адресом, если он находится в конце строки без символов после него. • [\w.\-] соответствует любому символу слова (a-z, A-Z, 0-9 или знак подчеркивания), точке или дефису. Это наиболее часто используемые допустимые символы в первой части адреса электронной почты. Символ \- (обозначающий дефис) должен стоять последним в списке символов в квадратных скобках. • Знак \ перед тире и точкой «экранирует» эти символы, т. е. указывает, что тире и точка сами по себе не являются специальными символами регулярных выражений. Нет необходимости экранировать точку в квадратных скобках. • {0,25} указывает, что от 0 до 25 символов в предыдущем наборе символов могут стоять перед символом @. Настройка электронной почты соответствия содержания поддерживает сопоставление до 25 символов для каждого набора символов в регулярном выражении. • Форматирование (...) группирует домены, а символ, который их разделяет, указывает на «или».

5. Совпадение с IP-адресом в диапазоне

Входная строка	Операции	Набор операционных символов
192.168.1.	{a32}, {a33}, {a34}, {a25}, {a35}, {a36}	{a32}{a33}{a34}{a25}{a32}{a35}{a36}{a25}{a32}{a25}
192.168.1.176	{a33}, {a34}, {a32}, {a35}, {a36}, {a25}, {a6}	{a32}{a33}{a34}{a25}{a32}{a35}{a36}{a25}{a32}{a25}{a6{1,3}}

Pattern 1	192\.168\.1\.
Pattern 2	192\.168\.1\. \d{1,3}
Пример	Соответствует IP-адресу в диапазоне от 192.168.1.0 до 192.168.1.255.

Примечания	• Знак \ перед каждой точкой «убегает» от точки, т. е. указывает, что период сам по себе не является специальным символом регулярного выражения. • В примере 1 за последней точкой не следует никаких символов, поэтому регулярное выражение соответствует любому IP-адресу, начинающемуся с 192.168.1., независимо от следующего за ним числа. • В примере 2 \d соответствует любой цифре от 0 до 9 после последней точки, а {1,3} указывает, что цифры от 1 до 3 могут появляться после этой последней точки. В этом случае регулярное выражение соответствует любому полному IP-адресу, начинающемуся с 192.168.1. Это регулярное выражение также соответствует недопустимым IP-адресам, таким как 192.168.1.999.
------------	--

6. Соответствие буквенно-цифровому формату. Замена нескольких пробелов, специальные символы. Разделить строку по символу.

Входная строка	Операции	Набор операционных символов
\$po#11-3434\$	{a ₁ }, {a ₂ }, {a ₃ }, {a ₄ }, {a ₅ }, {a ₆ }, {a ₇ }, {a ₈ }	{a ₁ }{a ₂ }{a ₃ }{a ₄ }{a ₅ }{a ₆ }{a ₇ }{a ₈ }{a ₁ }
\s\s\s\s	{a ₃₇ }{a ₈ }	{a ₃₇ }{a ₈ {1}}
!@#\$\$%^&*()	{a ₃₈ }	{a ₃₈ {1}}
aaaza	{a ₃₉ }	{a ₃₉ {1}}

1 pattern	(\W ^)PO[#-]{0,1}\s{0,1}\d{2}[\s-]{0,1}\d{4}(\W \$)
Пример	Сопоставьте номера заказов на покупку. Номер заказа имеет различные форматы, например: • PO nn-nnnnnn • PO nn-nnnn • PO# nn nnnn • PO#nn-nnnn • PO nnnnnn
2 pattern	\\s+
Пример	<pre>string badString = "a string with ton of white space."; string CleanedString = Regex.Replace(badString, "\\s+", " "); Console.WriteLine(\$"Cleaned String: {CleanedString}");</pre>
3 pattern	[0-9a-zA-Z]+
Примечание	Код <code>Regex.Replace(your String, @"[^0-9a-zA-Z]+", "")</code> удаляет все специальные символы, но если вы не хотите удалять некоторые специальные символы, например, запятая "," и двоеточие ":" то: <pre>Regex.Replace(Your String, @"[^0-9a-zA-Z:,]+", "")</pre>
4 pattern	[a-z]+
Пример	<pre>string azpattern = "[a-z]+"; string str = "Asd2323b0900c1234Def5678Ghi9012Jklm"; string[] result = Regex.Split(str, azpattern, RegexOptions.IgnoreCase, TimeSpan.FromMilliseconds(500)); for (int i = 0; i < result.Length; i++){ Console.WriteLine(\$"'{0}'", result[i]); if (i < result.Length - 1) Console.Write(", "); }</pre>

7. Допустимый номер мобильного телефона

Входная строка	Операции	Набор операционных символов
+91-12345-67890	{a ₂₈ }, {a ₄₀ }, {a ₆ }, {a ₄₁ }	{a ₂₈ }{a ₄₀ {0,1}}{a ₆ {0,2}}{a ₄₁ {0,1}}{a ₆ {4,5}} {a ₄₁ {0,1}}{a ₆ {5,6}}

1 pattern	<code>^\+?\d{0,2}\-?\d{4,5}\-?\d{5,6}</code>
Пример	Сопоставить номер мобильного телефона шаблону, например: <code>string[] str = { "+91-9678967101", "9678967101", "+91-9678-967101", "+91-96789-67101", "+919678967101"};</code>
Примечания	<pre>using System; using System.Collections.Generic; using System.Linq; using System.Text; using System.Text.RegularExpressions; namespace RegularExpression1 { class Program { static void Main(string[] args) { Regex r = new Regex(@"^\+?\d{0,2}\-?\d{4,5}\-?\d{5,6}"); string[] str = { "+91-9678967101", "9678967101", "+91-9678-967101", "+91-96789-67101", "+919678967101" }; // Введите строки для совпадения с // действительным номером // мобильного телефона. foreach (string s in str) { Console.WriteLine(" {0} {1} a valid mobile number.", s, r.IsMatch(s) ? "is" : "is not"); } //The IsMatch method is used to validate a string or //to ensure that a string conforms to a particular part } // foreach } // Main } // class } // namespace</pre>

8. Проверка ввода символьных данных пользователя.

Входная строка	Операции	Набор операционных символов
Word	{a ₂₈ }, {a ₄₂ }, {a ₄₃ }, {a ₂₉ }	{a ₂₈ }{a ₄₂ }{a ₄₃ {0}}{a ₂₉ }
Two Words	{a ₂₈ }, {a ₄₃ }, {a ₄₄ }, {a ₂₉ }	{a ₂₈ }{a ₄₃ {1}}{a ₄₄ {0,1}}{a ₂₉ }
Two ZAa	{a ₂₈ }, {a ₄₃ }, {a ₄₅ }, {a ₂₉ }	{a ₂₈ }{a ₄₃ {1}}{a ₄₅ {0,1}}{a ₂₉ }
12345	{a ₂₈ }, {a ₆ }, {a ₂₉ }	{a ₂₈ }{a ₆ {5,5}}{a ₂₉ }
123-456-7890	{a ₂₈ }, {a ₄₁ }, {a ₆ }, {a ₂₉ }	{a ₂₈ }{a ₆ {3,3}}{a ₄₁ }{a ₆ {3,3}}{a ₄₁ }{a ₆ {4,4}}{a ₂₉ }

Pattern 1	<code>^[A-Z][a-zA-Z]*\$</code>
Пример	<code>if(!Regex.Match(firstNameTextBox.Text, "^[A-Z][a-zA-Z]*\$").Success) {...}</code>
Pattern 2	<code>^([a-zA-Z]+ [a-zA-Z]+\s[a-zA-Z]+)\$</code>
Пример	<code>if(!Regex.Match(cityTextBox.Text, @"^([a-zA-Z]+ [a-zA-Z]+\s[a-zA-Z]+)\$").Success) {...}</code>
Pattern 3	<code>^([a-zA-Z]+ [a-zA-Z]+\s[a-zA-Z]+)\$</code>
Пример	<code>if(!Regex.Match(stateTextBox.Text, @"^([a-zA-Z]+ [a-zA-Z]+\s[a-zA-Z]+)\$").Success) {...}</code>
Pattern 4	<code>^\d{5}\$</code>
Пример	<code>if(!Regex.Match(zipCodeTextBox.Text, @"^\d{5}\$").Success) {...}</code>
Pattern 5	<code>^[1-9]\d{2}-[1-9]\d{2}-\d{4}\$</code>
Пример	<code>if(!Regex.Match(phoneTextBox.Text, @"^[1-9]\d{2}-[1-9]\d{2}-\d{4}\$").Success) {...}</code>

9. Выбор слов из текста на основе паттерна

Входная строка	Операции	Набор операционных символов
abc!@#	{a45}	{a45{1}}

Pattern 1	<code>\D+</code>
Пример	<code>string Text = "1 One, 2 Two, 3 Three is good.";</code>
Примечания	<pre>string[] digits = Regex.Split(Text, @"\D+"); foreach (string value in digits) { int number; if (int.TryParse(value, out number)) { Console.WriteLine(value); } }</pre>

Пример грамматики с операционными символами

Ниже представлена грамматика с операционными символами, соответствующая паттерну для сопоставления с текстом.

Паттерн: `\wо[#-]{0,1}\s{0,1}\d{2}[\s-]{0,1}\d{4}\W`

Грамматика: $G = (T, V, P, S, T^*, OP, D)$, где

T - множество терминальных символов

T^* - замыкание Клини над множеством терминальных символов

V - множество нетерминальных символов

$S_0 = S$ - начальный нетерминал

D - словарь с операциями

$OP = \{\{a_1\}, \{a_2\}, \dots, \{a_8\}\}$ - множество операционных символов

$\{a_1\}: \{\backslash W\{1, 1\}\}$

$\{a_2\}: \{p\{1, 1\}\}$

$\{a_3\}: \{o\{1, 1\}\}$

$\{a_4\}: \{[\#-]\{\emptyset, 1\}\}$

$\{a_5\}: \{\backslash s\{\emptyset, 1\}\}$

$\{a_6\}: \{\backslash d\{2, 2\}\}$

$\{a_7\}: \{[\backslash s-]\{\emptyset, 1\}\}$

$\{a_8\}: \{\backslash d\{4, 4\}\}$

$P = \{p_0, \dots, p_8\}$ - множество правил продукций:

$p_1: S \rightarrow \{a_1\}A$

$p_2: A \rightarrow \{a_2\}B$

$p_3: B \rightarrow \{a_3\}C$

$p_4: C \rightarrow \{a_4\}D$

$p_5: D \rightarrow \{a_5\}E$

$p_6: E \rightarrow \{a_6\}G$

$p_7: G \rightarrow \{a_7\}F$

$p_8: F \rightarrow \{a_8\}$

Примеры продукций:

$S \Rightarrow^1 \{a_1\}A \Rightarrow^2 \{a_1\}\{a_2\}B \Rightarrow^3 \{a_1\}\{a_2\}\{a_3\}C \Rightarrow^4 \{a_1\}\{a_2\}\{a_3\}\{a_4\}D \Rightarrow^5$
 $\{a_1\}\{a_2\}\{a_3\}\{a_4\}\{a_5\}E \Rightarrow^6 \{a_1\}\{a_2\}\{a_3\}\{a_4\}\{a_5\}\{a_6\}G \Rightarrow^7$
 $\{a_1\}\{a_2\}\{a_3\}\{a_4\}\{a_5\}\{a_6\}\{a_7\}F \Rightarrow^8 \{a_1\}\{a_2\}\{a_3\}\{a_4\}\{a_5\}\{a_6\}\{a_7\}\{a_8\}$

$L(G) = \{\{a_1\}\{a_2\}\{a_3\}\{a_4\}\{a_5\}\{a_6\}\{a_7\}\{a_8\}\}$

Конечный автомат с операционными символами

Конечный автомат с операционными символами представляет собой следующий кортеж:

$$KA = (\Sigma, \text{tape}, q_0, V, D, \delta, f(\text{tape}))$$

Σ - множество символов входной цепочки

tape - лента

Q - множество состояний

F - множество конечных состояний

δ - множество дельта правил

q_0 - начальное состояние

tape(D) - операция записи на ленту результирующей цепочки операции

D - отношение “операционный символ” - “операция”

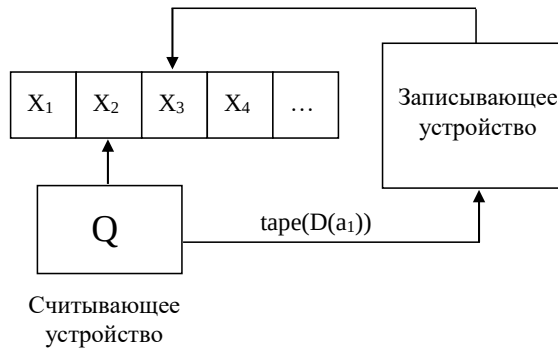
Определение 1

Дельта-правилом конечного автомата называется отношение следующего вида

$$\delta(A, \beta_i, \text{tape}(D(\{a\}))) = \{B, \beta_i + 1\}$$

Конечный автомат работает по следующему принципу:

1. Считывающее устройство считывает всю входную цепочку β_i
2. Исходя из текущего состояния A выполняется операция $D(\{a\})$. При этом, на вход такой операции приходит считанная цепочка β
3. На выходе операция $D(\{a\})$ возвращает цепочку $\beta_i + 1$, которая передается операции tape . В результате на ленте находится цепочка $\beta_i + 1$



Определение 2

Конфигурацией конечного автомата с операционными символами называется следующая пара

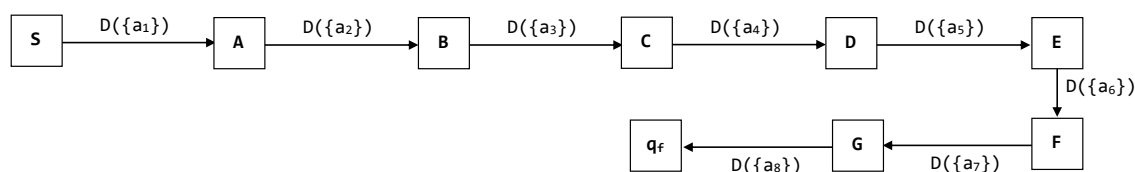
$$(Q, \alpha)$$

где Q - текущее состояние автомата, а α - цепочка, находящаяся на ленте.

Пример

Построим дельта правила на основе введенных операций, помеченных операционными символами $\{a_1\}, \dots, \{a_j\}$

$$\begin{aligned} \delta_1(S, \alpha_1, \text{tape}(D(\{a_1\}))) &= \{A, \alpha_2\} \\ \delta_2(A, \alpha_2, \text{tape}(D(\{a_2\}))) &= \{B, \alpha_3\} \\ \delta_3(B, \alpha_3, \text{tape}(D(\{a_3\}))) &= \{C, \alpha_4\} \\ \delta_4(C, \alpha_4, \text{tape}(D(\{a_4\}))) &= \{D, \alpha_5\} \\ \delta_5(D, \alpha_5, \text{tape}(D(\{a_5\}))) &= \{E, \alpha_6\} \\ \delta_6(E, \alpha_6, \text{tape}(D(\{a_6\}))) &= \{F, \alpha_7\} \\ \delta_7(F, \alpha_7, \text{tape}(D(\{a_7\}))) &= \{G, \alpha_8\} \\ \delta_8(G, \alpha_8, \text{tape}(D(\{a_8\}))) &= \{q_f, \alpha_9\} \end{aligned}$$



Пример разбора:

$(S, \$ро\# \sqcup 4444) \vdash^1 (A, ро\# \sqcup 4444) \vdash^2 (B, о\# \sqcup 4444) \vdash^3 (C, \# \sqcup 4444) \vdash^4 (D, \sqcup 4444) \vdash^5 (E, 4444) \vdash^6 (F, 44) \vdash^7 (G, 44) \vdash$ **ОШИБКА**

$(S, \$ро\#22-1234) \vdash^1 (A, ро\#22-1234) \vdash^2 (B, о\#22-1234) \vdash^3 (C, \#22-1234) \vdash^4 (D, 22-1234) \vdash^5 (D, 22-1234) \vdash^6 (E, -1234) \vdash^7 (G, 1234) \vdash^7 (q_f, \epsilon)$

2.7. Проектирование порождающих и распознающих систем

1. Постановка задачи: $L = L(G) = L(KA)$.

Задан язык $L = \{c\omega n \mid \omega \in \{+, d, -, k, f\}^*\}$, где ω обозначает множество всех цепочек, составленных из символов $\{+, d, -, k, f\}^*$. Цепочки начинаются с символа c , заканчиваются символом n .

Спроектировать грамматику $G = (T, V, P, S_0)$ для заданного языка L , привести пример вывода цепочки.

Используя грамматику G , построить $KA = (Q, \Sigma, \delta, q_0, F)$ и привести пример конфигурации KA . Построить диаграмму переходов KA . Определить свойства KA , если это НКА реализовать алгоритм преобразования НДКА в ДКА.

1.a. Определить свойства языка. Применить лемму о накачке.

1.b. Спроектировать грамматику $G = (T, V, P, S_0)$, где

$T = \{c, d, k, f, n, +, -\}$ множество терминальных символов

$V = \{S_0, Q\}$ множество нетерминальных символов

$P = \{S_0 \rightarrow Qn, Q \rightarrow c \mid Q+ \mid Q- \mid Qd \mid Qk \mid Qf\}$

1.c. Определить свойства грамматики. Грамматика является левوليнейной, бесконечной. Пример вывода цепочки $S_0 \Rightarrow Qn \Rightarrow Qf-n \Rightarrow Qkf-n \Rightarrow Qdkf-n \Rightarrow Q-dkf-n \Rightarrow Q+d-dkf-n \Rightarrow c+d-dkf-n$.

1.d. Используя грамматику G построить KA .

$KA = (\{S_0, Q, q_f\}, \{c, d, k, f, n, +, -\}, \delta, S_0, q_f)$

$\delta(S_0, c) = \{Q\};$

$\delta(Q, d) = \{Q\}; \delta(Q, k) = \{Q\}; \delta(Q, f) = \{Q\}; \delta(Q, +) = \{Q\}; \delta(Q, -) = \{Q\};$

$\delta(Q, n) = \{q_f\};$

Пример конфигурации КА: $S_0c+d-dkf-n \mid Q+d-dkf-n \mid Qd-dkf-n \mid Q-dkf-n \mid Qdkf-n \mid Qkf-n \mid Qf-n \mid Q-n \mid Qn \mid q_f$

1.e. Определить свойства конечного автомата. Конечный автомат является недетерминированным (НДКА). Преобразовать НДКА в ДКА.

1.f. Построить диаграмму переходов ДКА рис. 2.6.:

$\{d, k, f, +, -\}$

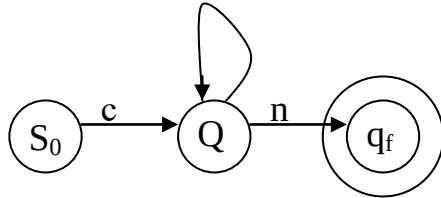


Рис. 2.6. Диаграмма переходов КА

Глава 3. Контекстно-свободные грамматики и МП-автоматы

3.1. Определение КС-грамматик и МП-автоматов

Из четырех типов грамматик иерархии Хомского класс контекстно-свободных грамматик наиболее важен с точки зрения приложений к языкам программирования и компиляции. С помощью этого типа грамматик определяется большая часть синтаксических структур языков программирования.

Определение 6. Контекстно-свободная грамматика типа 2:

Грамматика $G = (T, V, P, S_0)$ называется *контекстно-свободной* (КС) (или *бесконтекстной*), если каждое правило из P имеет вид $A \rightarrow \alpha$, где $A \in V$, $\alpha \in (T \cup V)^*$.

Заметим, что, согласно определению каждая праволинейная грамматика – КС. Языки, порождаемые КС-грамматиками, называются КС-языками.

Пример, КС-грамматики, порождающей скобочные арифметические выражения: $G_1 = (\{v, +, *, (,)\}, \{S_0, F, L\}, P, S_0)$, где $P = \{p_1: S_0 \rightarrow S_0 + F, p_2: S_0 \rightarrow F, p_3: F \rightarrow F * L, p_4: F \rightarrow L, p_5: L \rightarrow v, p_6: L \rightarrow (S_0)\}$.

Пример вывода, заменяя самый левый нетерминал (левосторонний вывод) сентенциальной формы: $S_0 \Rightarrow^1 S_0 + F \Rightarrow^2 F + F \Rightarrow^4 L + F \Rightarrow^5 v + F \Rightarrow^3 v + F * L \Rightarrow^4 v + L * L \Rightarrow^5 v + v * L \Rightarrow^5 v + v * v$.

3.2. Преобразование КС-грамматик

Для построения синтаксических анализаторов необходимо, чтобы КС-грамматика была в *приведенной* форме см. рис. 3.1.

Если применить к КС-грамматике алгоритмы:

- 1). устранения бесполезных символов: *непроизводящих* и *недостижимых*;
- 2). устранения левой рекурсии;
- 3). преобразования в грамматику без ϵ -правил;
- 4). устранения цепных правил. то получим *приведенную* КС-грамматику.

Определение. Символ $x \in (T \cup V)$ называется *недостижимым* в КС-грамматике G , если он не может появиться ни в одной из ее сентенциальных форм. Т.е. символ *недостижимый*, если он не участвует ни в одной цепочке вывода из начального символа грамматики. Очевидно, что такой символ грамматике не нужен.

Определение. Нетерминальный символ $A \in V$ называется *производящим*, если из него можно вывести терминальную цепочку, т.е. если существует вывод $A \Rightarrow^+ \alpha$, где $\alpha \in T^+$. В противном случае символ называется *непроизводящим*.

Определение. Символ $a \in T \cup V$ называется *бесполезным* в КС-грамматике G , если он *непроизводящий* или *недостижимый*.

Рассмотрим алгоритмы и их применение.



Рис.3.1 Метод преобразования грамматики к *приведенной* форме

Алгоритм устранения бесполезных символов

Устранение *бесполезных* символов. Вначале исключить *непроизводящие* нетерминалы, а затем *недостижимые* символы. Выполнить:

1. Алгоритм устранения непроизводящих символов и

2. Алгоритм устранения недостижимых символов.

1. Алгоритм устранения непроизводящих символов

Утверждение. Если все символы цепочки α из правой части правила вывода $A \Rightarrow^+ \alpha$ являются производящими, то нетерминал $A \in V$ в левой части правила вывода также должен быть производящим: $V_p = \{A \mid A \Rightarrow^+ \alpha, A \in V, \alpha \in T^+\}$

Вход: КС $G = (T, V, P, S)$

Выход: $V_p = \{A \mid A \Rightarrow^+ \alpha, A \in V, \alpha \in T^+\}, S'$

1. Составить множество V_p нетерминалов, для которых найдется хотя бы одно правило, правая часть которого не содержит нетерминалов.

2. Если найдено такое правило, что все нетерминалы, стоящие в его правой части уже занесены в V_p , то добавить в V_p нетерминал, стоящий в его левой части.

3. Если на шаге 2, V_p больше не пополняется, то V_p содержит все производящие нетерминалы грамматики, а все нетерминалы не попавшие в него являются непроизводящими.

$i = 0$

$V_p^i = \emptyset$

do

$i = i + 1;$

foreach $(A \rightarrow \alpha \in P, \alpha \in (T \cup V^{i-1}_p)^+)$

if $(A \notin V^{i-1}_p)$

$V_p^i = V^{i-1}_p \cup \{A\}$ // $V_p^i = \{A \mid (A \rightarrow \alpha) \in P, \alpha \in (T \cup V^{i-1}_p)^+\}$

end

end foreach

while $(V_p^i \neq V^{i-1}_p)$

if $(S \notin V_p^i)$ S - непроизводящий символ, то определить $S \in V_p^i$

$n = 0$

foreach $(A \in V_p^i)$

if $|\alpha| \geq n, (A \rightarrow \alpha \in P)$ при $n = 0$ если есть $A \rightarrow \varepsilon$

$n = |\alpha|$

$S = A$

end

end foreach

end

1. Добавлять в P' только правила в правой части которых только производящие символы

Вход: КС $G = (T, V, P, S), V_p^i$

Выход: $G_1 = (T, V_p^i, P', S')$

$P' := \emptyset$
 foreach $(A \in V_p^i, (A \rightarrow \alpha) \in P)$
 из цепочки символов α построить множество нетерминалов V_α
 $V_\alpha = \{x \mid x \in \alpha, x \in V\}$ для $V_\alpha = \emptyset, V_\alpha - V_p^i = \emptyset$
 if $(V_\alpha - V_p^i = \emptyset)$ в V_α только производящие символы из V_p^i (- разность множеств)
 $P' = P' \cup (A \rightarrow \alpha) \in P$
 end
 end foreach

2. Алгоритм устранения недостижимых символов

VT_r - множество достижимых символов (нетерминальных и терминальных).

Утверждение. Если нетерминал A в левой части правила грамматики G является достижимым, то достижимы и все символы α правой части этого правила $A \rightarrow \alpha$. $VT_r = \{x \mid A \rightarrow \alpha, x \in \alpha, \alpha \in (V \cup T)^*\}$

Вход: КС грамматика $G_1 = (T, V_p^i, P', S')$

Выход: $G' = (T', V', P'', S)$, у которой $L(G') \neq \emptyset$ и в T', V' нет бесполезных символов.

Шаг 1. Построить множество VT_r^i - достижимых терминалов и не терминалов.

$i = 0;$
 $VT_r^i = \{S'\}$
 do
 $i = i + 1;$
 foreach $((A \rightarrow \alpha) \in P', A \in VT_r^{i-1}, \alpha \in (V \cup T)^*)$
 foreach $x \in \alpha$
 if $x \notin VT_r^{i-1}$
 $VT_r^i = VT_r^{i-1} \cup \{x\}$
 end foreach
 end foreach
 end foreach
 while $(VT_r^i \neq VT_r^{i-1})$

Шаг 2. Построить P'', T', V' .

$P'' = \emptyset$
 foreach $(A \rightarrow \{X_1, X_2, \dots, X_n\}, A, X_1, X_2, \dots, X_n \in VT_r^i) \in P$
 $P'' := P'' \cup (A \rightarrow \{X_1, X_2, \dots, X_n\})$
 end foreach

$T' := T \cap VT_r^i$
 $V' := V \cap VT_r^i$

Пример: Устранить из грамматики G бесполезные символы. $G = (T, V, P, S)$, где $V = \{A, B, C, S\}$, $T = \{a, b, c\}$, $P = \{S \rightarrow aC, S \rightarrow A, A \rightarrow cAB, B \rightarrow b, C \rightarrow a\}$.

Алгоритм устранения непродуцирующих символов.

множество $V_p^1 = \{B, C\}$, $V_p^2 = \{S, B, C\}$

$V - V_p = \{A\}$ – *непроизводящий символ*

$G_1 = (\{B, C, S\}, \{a, b\}, P, S)$, где $P = \{S \rightarrow aC, B \rightarrow b, C \rightarrow a\}$

Алгоритм устранения недостижимых символов.

$VT_r^1 = \{S\}$

$VT_r^2 = \{S, C, a\}$

$VT_r^3 = \{S, C, a\}$, B – *недостижим*

$G' = (\{C, S\}, \{a\}, P, S)$, где $P = \{S \rightarrow aC, C \rightarrow a\}$, $L(G') = \{aa\}$

Алгоритм устранения эпсилон-правил

КС-грамматика называется *неукорачивающей* КС-грамматикой (НКС-грамматикой, КС-грамматикой без ε -правил) при условии, что P не содержит $S \rightarrow \varepsilon$ и S не встречается в правах частей остальных правил.

В грамматике с правилами вида $A \rightarrow \varepsilon$ длина выводимой цепочки при переходе от k -го шага к $(k+1)$ -му уменьшается. Поэтому грамматики с правилами вида $A \rightarrow \varepsilon$ называются укорачивающими.

Восходящий синтаксический разбор в укорачивающих грамматиках сложнее по сравнению с разбором в неукорачивающих грамматиках, т.к. при редукции необходимо отыскать такой фрагмент входной цепочки, в которую можно вставить пустой символ.

Для произвольной КС-грамматики, порождающей язык без пустой цепочки, можно построить эквивалентную неукорачивающую КС-грамматику.

Построение множества V_ε укорачивающих (обнуляемых) нетерминалов.

1. Алгоритм построение множества укорачивающих нетерминалов

Утверждение. Нетерминал A - укорачивающий (обнуляемый) если $A \Rightarrow^* \varepsilon$.

1. A - укорачивающий для правила $A \rightarrow \varepsilon$ (см. Шаг 1.).

2. A - укорачивающий для правила $A \rightarrow B_1 B_2 \dots B_k$, если каждый нетерминал B_1 в правиле укорачивающий (см. Шаг 2.).

Вход: КС грамматика $G = (T, V, P, S)$.

Выход: $V_\varepsilon = \{A \mid A \Rightarrow^+ \varepsilon, A \in V\}$

Шаг 1. Построить множество V_ε^i - укорачивающих нетерминалов для правил вида $A \rightarrow \varepsilon$.

$i = 0$

$V_\varepsilon^i = \emptyset$

foreach ($A \in V$)

if ($(A \rightarrow \varepsilon) \in P$ // $A \rightarrow \varepsilon$ правило укорачивающееся

$V_\varepsilon^i := V_\varepsilon^i \cup \{A\}$

end

end foreach

Шаг 2. Построить множество V_ε^i - укорачивающих нетерминалов для правил вида $A \rightarrow \varepsilon$. $A \rightarrow B_1 B_2 \dots B_k$, если каждый нетерминал B_1 в правиле укорачивающий. $V_\varepsilon^i = V_\varepsilon^{i-1} \cup \{A \mid (A \rightarrow \alpha) \in P \text{ и } \alpha \in (V_\varepsilon^{i-1})^+\}$. $\alpha \in (V_\varepsilon^{i-1})^+$ - рассматриваются только правила, содержащие в правых частях только нетерминальные укорачивающие символы: $A \rightarrow \alpha$, например $A \rightarrow CDF$.

```

do
  i = i + 1
   $V_{\varepsilon}^{i-1} = V_{\varepsilon}^i$ 
  foreach  $((A \rightarrow \alpha) \in P$  и  $\alpha \in (V_{\varepsilon}^{i-1})^+$ )
     $V_{\varepsilon}^i := V_{\varepsilon}^{i-1} \cup A$ 
  end
while( $V_{\varepsilon}^{i-1} \neq V_{\varepsilon}^i$ )

```

2. Алгоритм устранения эpsilon-правил

Алгоритм устранения ε -правил в КС-грамматике основан на использовании множества укорачивающих нетерминалов. Алгоритм преобразует КС-грамматику с ε -правилами в эквивалентную КС-грамматику. Пусть $X_i \in \{X \mid (T \cup V) \cup \{\varepsilon\}, 0 < i \leq k\}$. Построить из $\{X_1, X_2, \dots, X_k\}$ множество цепочек в которых, либо присутствует, либо устранён каждый из нетерминалов V_{ε} .

Вход: $G' = (T, V_{\varepsilon}, P, S)$

Выход: $G'' = (T, V', P', S')$

$P' = \emptyset$

foreach $(A \rightarrow X_1, X_2, \dots, X_k \in P, X_1 \neq \varepsilon)$

$V'_{\varepsilon} = \{ [X_1, X_2, \dots, X_k] \cap V_{\varepsilon} \mid X_i < X_m \}$ V'_{ε} - упорядоченное мн-во

if $(V'_{\varepsilon} = \emptyset)$

$P' = P' \cup A \rightarrow X_1, X_2, \dots, X_k$

else построить $A \rightarrow \{Y_1, Y_2, \dots, Y_k\}$

$Y = \emptyset$

foreach $a \in \{\alpha = \{X_i, \dots, X_m\} \subset V'_{\varepsilon}{}^+ \mid |\alpha| \leq |V'_{\varepsilon}| \text{ и } X_i < X_m\}$

foreach $v \in a$

foreach $(X_i \in \{X_1, X_2, \dots, X_k\}, A \neq X_i)$ правило $A \rightarrow A$ бессмысленно

if $X_i = v$

$Y_i = \varepsilon$

end

$Y = Y \cup Y_i$

end foreach

$P' = P' \cup (A \rightarrow \{Y_1, Y_2, \dots, Y_k\})$

end foreach

end foreach

end

end foreach

if $(S \in V_{\varepsilon})$

$V' = V' \cup \{S'\}$

$P' = P' \cup (S' \rightarrow S \mid \varepsilon)$

end

Положить $G'' = (T, V', P', S')$.

Пример 1. Устранить из грамматика G ε -правила. Преобразовать грамматику $G = (T, V, P, S)$, где $V = \{A, S\}$, $T = \{b, c\}$, $P = \{S \rightarrow cA, S \rightarrow \varepsilon, A \rightarrow cA, A \rightarrow bA, A \rightarrow \varepsilon\}$, в эквивалентную НКС-грамматику.

Шаг 1. Применяя алгоритм построения множества неукорачивающих нетерминалов, получаем $V^0_\varepsilon = \{S, A\}$, $V^1_\varepsilon = \{S, A\}$, значит $V^0_\varepsilon = V^1_\varepsilon = \{S, A\}$.

Шаг 2. Положить $P' = \emptyset$.

Рассмотрим правило $S \rightarrow cA$.

Для него в новое множество правил грамматики P' добавляем правила $S \rightarrow cA$ и $S \rightarrow c$.

Для правила $A \rightarrow cA$, добавляем в P' правила $A \rightarrow cA$ и $A \rightarrow c$.

Для правила $A \rightarrow bA$, добавляем в P' $A \rightarrow bA$ и $A \rightarrow b$, тогда $P' = \{S \rightarrow cA, S \rightarrow c, A \rightarrow cA, A \rightarrow c, A \rightarrow bA, A \rightarrow b\}$.

Шаг 3. $S \in V_\varepsilon$, то в V' добавляем новый нетерминал S' , а в P' два правила $S' \rightarrow S$ и $S' \rightarrow \varepsilon$.

$G' = (\{b, c\}, \{S', S, A\}, \{S' \rightarrow S, S' \rightarrow \varepsilon, S \rightarrow cA, S \rightarrow c, A \rightarrow cA, A \rightarrow c, A \rightarrow bA, A \rightarrow b\}, S')$.

Пример 2. Устранить из грамматика G ε -правила. Преобразовать грамматику $G = (\{a, b, c\}, \{A, B, C, S\}, S, P)$

$P: S \rightarrow AaB \mid aB \mid cC$

$A \rightarrow AB \mid a \mid b \mid B$

$B \rightarrow Ba \mid \varepsilon$

$C \rightarrow AB \mid c$

Алгоритм построения множества неукорачивающих нетерминалов V^i_ε :

1. $V^0_\varepsilon = \{B\}$, $i = 1$ (шаг 1)

2. $V^1_\varepsilon = \{B, A\}$, $V^1_\varepsilon \neq V^0_\varepsilon$, $i = 2$ (шаги 2 и 3)

3. $V^2_\varepsilon = \{B, A, C\}$, $V^2_\varepsilon \neq V^1_\varepsilon$, $i = 3$ (шаги 2 и 3)

4. $V^3_\varepsilon = \{B, A, C\}$, $V^3_\varepsilon = V^2_\varepsilon$, $i = 3$ (шаги 2 и 3)

Алгоритм устранения эпсилон-правил:

$V = \{A, B, C, S\}$, $T = \{a, b, c\}$, , строим множество P'

Добавить в P' все правила P за исключением ε -правил

P' $p_1: S \rightarrow AaB \mid aB \mid cC$

$p_2: A \rightarrow AB \mid a \mid b \mid B$

$p_3: B \rightarrow Ba$

$p_4: C \rightarrow AB \mid c$

Рассмотрим все правила множества из P' (шаг. 5):

$p_1: S \rightarrow AaB \mid aB \mid cC$ исключаем все комбинации $V^3_\varepsilon = \{B, A, C\}$ получим $S \rightarrow Aa \mid aB \mid a \mid a \mid c$ добавим в P' без дублирования

$P' = P' \cup (S \rightarrow Aa \mid aB \mid a \mid a \mid c)$

$p_2: A \rightarrow AB \mid a \mid b \mid B$ исключаем все комбинации $V^3_\varepsilon = \{B, A, C\}$ получим $A \rightarrow A \mid B$ их добавлять не надо $A \rightarrow A$ бессмысленно а $A \rightarrow B$ уже есть

$p_3: B \rightarrow Ba$ исключаем все комбинации $V^3_\varepsilon = \{B, A, C\}$ получим

$B \rightarrow a$ добавим в P' без дублирования
 $P' = P' \cup (B \rightarrow a)$ получим $B \rightarrow Ba \mid a$

$p_4: C \rightarrow AB \mid c$ исключаем все комбинации $V_\varepsilon^3 = \{B, A, C\}$ получим
 $C \rightarrow A \mid B \mid c$ добавим в P' без дублирования
 $P' = P' \cup (C \rightarrow A \mid B \mid c)$ получим $C \rightarrow AB \mid A \mid B \mid c$

$S \notin V_\varepsilon^3$, поэтому в грамматику G не надо добавлять S' . В итоге получим грамматику.

$G' = (\{a,b,c\}, \{A,B,C,S\}, S, P')$
 $P': p_1: S \rightarrow AaB \mid aB \mid cC \mid Aa \mid a \mid c$
 $p_2: A \rightarrow AB \mid a \mid b \mid B$
 $p_3: B \rightarrow Ba \mid a$
 $p_4: C \rightarrow AB \mid A \mid B \mid c$

Алгоритм устранения цепных правил

Правило вида $A \rightarrow B$, где A и B - нетерминалы называется цепным.

Вход: КС грамматика $G = (T, V, P, S)$.

Выход: Эквивалентная КС грамматика $G' = (T, V, P', S)$ без цепных правил.

Шаг 1. Для каждого правила $X \rightarrow A \in P$ построить множество всех цепных символов V_X^i .

```
foreach ( $X \in V$ )  построить множество  $V_X = \{A \mid X \Rightarrow^+ A\}$ 
  i = 0
   $V_X^i = \{X\}$ 
  do
    i++
    foreach ( $X \in V^{i-1}_X, X \rightarrow A \in P, A \in V$ )
       $V_X^i = V^{i-1}_X \cup \{A\}$ 
    end
  while ( $V_X^i \neq V^{i-1}_X$ )
end
```

Шаг 2. Для каждого цепного правила $A \rightarrow R \in P$, Построить правила $A \rightarrow Y\rho$ без цепных символов $Y \notin V_A^i$. Добавить в P' не цепные правила.

```
 $P' = \emptyset$ 
foreach ( $A \rightarrow \alpha R \beta \in P$ )
  if ( $\alpha = \varepsilon, \beta = \varepsilon, R \in V_A^i$ )  цепное правило
    foreach ( $R \in V_A^i$ )  найти правую не цепную часть правила
      if ( $R \rightarrow Y\rho \in P, Y \notin V_A^i$ )
         $P' = P' \cup \{A \rightarrow Y\rho\}$ 
      end
    end
  else
     $P' = P' \cup \{A \rightarrow \alpha R \beta\}$ 
```

end
end

Пример. Устранить из КС грамматики G цепные правила, $G = (\{S, F, L\}, \{v, +, *, (,)\}, P, S)$, где P состоит из правил

$S \rightarrow S + F \mid F, F \rightarrow F * L \mid L, L \rightarrow v \mid (S)$

Шаг 1. Получаем $V_S = \{S, F, L\}$, $V_F = \{F, L\}$, $V_L = \{L\}$.

Шаг 2. Для каждого цепного правила $A \rightarrow R \in P$, Построить правила $A \rightarrow Y$ без цепных символов $Y \notin V_A^i$. Добавить в P' не цепные правила.

Для $V_S = \{S, F, L\}$ построим $\{S \rightarrow S + F \mid F * L \mid (S) \mid v\}$.

Включаем эти правила в $P' = \{S \rightarrow S + F \mid F * L \mid (S) \mid v\}$.

Тоже для V_F, V_L .

В результате $G' = (T, V, P', S)$, где $P' = \{S \rightarrow S + F \mid F * L \mid (S) \mid v, F \rightarrow F * L \mid (S) \mid v, L \rightarrow v \mid (S)\}$

Алгоритм устранения левой рекурсии

Определение. Нетерминал КС-грамматики называется *рекурсивным*, если $A \Rightarrow^+ \alpha A \beta$, для некоторых α и β . Если $\alpha = \varepsilon$, то A называется *леворекурсивным*, если $\beta = \varepsilon$, то A называется *праворекурсивным*. Грамматика, имеющая хотя бы один леворекурсивный нетерминал, называется *леворекурсивной*. Грамматика, имеющая хотя бы один праворекурсивный нетерминал, называется *праворекурсивной*.

При нисходящем синтаксическом анализе требуется, чтобы приведенная грамматика рассматриваемого языка не содержала **левой рекурсии**.

Непосредственной левой рекурсией считается такая рекурсия, при которой существует продукция вида $A \rightarrow Aa$. Общим случаем устранения левой рекурсии обозначают устранение левой рекурсии, вызванной двумя и более шагами порождения.

Частный случай не леворекурсивной грамматики – грамматика в нормальной форме Шейлы Грейбах.

Определение 7. КС грамматика $G = (T, V, P, S)$ называется грамматикой в нормальной форме Грейбах, если в ней нет ε -правил, т.е. правил вида $A \rightarrow \varepsilon$, и каждое правило из P отличное от $S \rightarrow \varepsilon$, имеет вид $A \rightarrow a\alpha$, где $a \in T$, $\alpha \in V^*$.

Также полезно представлять грамматику в нормальной форме Хомского, что позволяет упростить рассмотрение ее свойств.

Определение 8. КС грамматика $G = (T, V, P, S)$ называется грамматикой в нормальной форме Хомского, если каждое правило из P имеет один из следующих видов:

1. $A \rightarrow BC$, где $A, B, C \in V$;
2. $A \rightarrow a$, где $a \in T$;
3. $S \rightarrow \varepsilon$, если $\varepsilon \in L(G)$, причем S не встречается в правых частях правил.

Утверждение. Для любой КС-грамматики существует эквивалентная грамматика без левой рекурсии. Упорядочив нетерминалы по возрастанию индексов, и исключив правила вида $A_i \rightarrow A_j \alpha$, где $j > i$ для всех A_i , получим что нет вывода $A_i \Rightarrow^* A_i$ и в грамматике нет левой рекурсии.

Алгоритм. Устранение непосредственной левой рекурсии [1] Ахо “Компиляторы: принципы, технологии и инструментарий”:

Вход: КС грамматика $G = (T, V, P, S_0)$, без ε -правил (вида $A \rightarrow \varepsilon$).

Выход: Эквивалентная приведенная КС грамматика $G' = (T, V, P', S'_0)$.

1. Упорядочить нетерминалы V по возрастанию $V = [A_1, A_2, \dots, A_n]$. Преобразовать правила $A_i \rightarrow \alpha$ так, чтобы цепочка α начиналась либо с терминала, либо с такого A_j , что $j > i$. $i = 1$.

foreach i от 1 до n

foreach j от 1 до $i - 1$

2. Множество A_i правил – это $A_i \rightarrow A_i\alpha_1 | \dots | A_i\alpha_m | \beta_1 | \dots | \beta_p$, где ни одна цепочка β_j не начинается с A_k , если $k \leq i$. Заменяем A_i - правила правилами:

$A_i \rightarrow \beta_1 | \dots | \beta_p | \beta_1 A_i' | \dots | \beta_p A_i'$

$A_i' \rightarrow \alpha_1 | \dots | \alpha_m | \alpha_1 A_i' | \dots | \alpha_m A_i'$

где A_i' – новый символ. Правые части всех A_i - правил начинаются теперь с терминала или с A_k для некоторого $k > i$.

3. Если $i = n$, то останов и получена грамматика G' , иначе $j = i$, $i = i + 1$.

4. Заменить каждое правило вида $A_i \rightarrow A_j\alpha$ правилами $A_i \rightarrow \beta_1\alpha | \dots | \beta_m\alpha$, где $A_j \rightarrow \beta_1 | \dots | \beta_m$ – все A_j - правила.

Так как правая часть каждого A_j – правила начинается уже с терминала или с A_k для $k > j$, то и правая часть каждого A_i – правила будет обладать этим же свойством.

5. Если $j = i - 1$, перейти к шагу 2, иначе $j = j + 1$ и перейти к шагу 4.

Если ε присутствовал в языке исходной грамматики, добавим новый начальный символ S' и правила $S' \rightarrow S | \varepsilon$.

Корректность алгоритма на i итерации внешнего цикла:

1. В любой продукции вида $A_k \rightarrow A_l\alpha$, $k < i$, должно быть $l > k$. При следующей итерации внутреннего цикла растет нижний предел m всех продукций вида $A_i \rightarrow A_m\alpha$ до тех пор, пока не будет достигнуто $i \leq m$.

2. Будут только правила вида $A_i \rightarrow A_j\alpha$, где $j > i$. Неравенство становится строгим только после применения алгоритма устранения непосредственной левой рекурсии. При этом добавляются новые нетерминалы.

3. Созданный нетерминал A_i' имеет номер A_{-i} (номер меньший всех имеющихся на данный момент нетерминалов). Нет правила вида $\dots \rightarrow A_i'$, где A_i' самый левый нетерминал, а значит новые нетерминалы можно не рассматривать во внешнем цикле.

4. Все правила вида $A_i \rightarrow A_j\gamma$ где $j < i$ заменяются на $A_i \rightarrow \alpha_1\gamma | \dots | \alpha_k\gamma$ где $A_j \rightarrow \alpha_1 | \dots | \alpha_k$.

5. Итерация не меняет язык, а значит язык, получившийся в итоге преобразования правил, совпадает с исходным.

Пример. Алгоритм устранения левой рекурсии из КС – грамматики $G = (\{v, +, *, ()\}, \{S, F, L\}, P, S)$, где P состоит из правил:

$$p_1: S \rightarrow S + F \mid F$$

$$p_2: F \rightarrow F * L \mid L$$

$$p_3: L \rightarrow v \mid (S)$$

Шаг 1. Пусть $S_1 = S$, $F_2 = F$, $L_3 = L$. $V = [S_1, F_2, L_3]$.

Шаг 2. Для $i = 1$ преобразуем правила $p_1: S \rightarrow S + F \mid F$, $\alpha = + F$, $\beta = F$. Заменим S - правила правилами: $S \rightarrow F \mid FS'$, и добавим в грамматику правило для нового нетерминала $S' \rightarrow + F \mid + FS'$.

V	S	F	L
i	1		
j			

Шаг 3. $i = 1$ и $j = 2$.

V	S	F	L
i	1		
j		2	

Шаг 4. Для $i = 1$, $j = 2$ правила вида $A_i \rightarrow A_j \alpha$ отсутствуют.

Шаг 2. Для $i = 2$ преобразуем правила $p_2: F \rightarrow F * L \mid L$, $\alpha = * L$, $\beta = L$. Заменим F - правила правилами: $F \rightarrow L \mid LF'$, и добавим в грамматику правило для нового нетерминала $F' \rightarrow * L \mid LF'$.

V	S	F	L
i		2	
j			

Шаг 3. $i = 2$, $j = 2$.

V	S	F	L
i		2	
j		2	

Шаг 4. Для $i = 2$, $j = 2$ правила вида $A_i \rightarrow A_j \alpha$ отсутствуют.

Шаг 4. Для $i = 3$, $j = 1$ правила вида $A_i \rightarrow A_j \alpha$ отсутствуют.

V	S	F	L
i			3
j	1		

Обошли все нетерминалы

Получили правила новой грамматики G' :

$S \rightarrow F \mid FS', S' \rightarrow + F \mid + FS', F \rightarrow L \mid LF', F' \rightarrow * L \mid LF', L \rightarrow v \mid (S).$

Алгоритм 2. Устранение косвенной левой рекурсии

Вход: приведенная КС-грамматика $G = (T, V, P, S)$, то есть грамматика не должна содержать цепных и ε – правил.

Выход: эквивалентная КС-грамматика без левой рекурсии.

$P' = \emptyset$ // множество конечных правил

$Pdel = \emptyset$ // хранение удаляемых правил

$V_1 = V$ // хранение дополняем нетерминалы со штрихами

foreach ($A_i \in V \mid 1 \leq i \leq n$)

//при начальном нетерминале следующие 3 вложенных цикла не выполняются

// но при $i = 1$ удаляется непосредственная левая рекурсия

foreach ($A_j \in V \mid 1 \leq j < i$)

foreach ($p \in P \mid A_i \rightarrow A_j \gamma$)

foreach ($p \in P \mid \{ A_j \rightarrow \alpha_1 \mid \dots \mid \alpha_k \} \notin Pdel$)

//добавление правил вида $A_i \rightarrow x\gamma$

$P = P \cup A_i \rightarrow x\gamma$

//добавление неиспользуемых правил

$Pdel = Pdel \cup A_i \rightarrow A_j \gamma$

end

end

end

$V_r = \emptyset$ //множество нетерминалов, имеющих левую рекурсию

//добавление в множество нетерминалов, имеющих левую рекурсию

foreach ($p \in P \mid A_i \rightarrow A_i \gamma$)

$V_r = V_r \cup A_i$

end

if ($A_i \in V_r$)

$V_1 = V_1 \cup A_i'$

end if

//добавление правил $A' \rightarrow \alpha_1 \mid \dots \mid \alpha_k \mid \alpha_1 A' \mid \dots \mid \alpha_k A'$ в конечное множество

foreach ($A_i \rightarrow A_i \alpha_1 \mid \dots \mid A_i \alpha_k \in P \mid A_i \rightarrow A_i \alpha_1 \mid \dots \mid A_i \alpha_k \notin Pdel$)

$P' = P' \cup A_i' \rightarrow \alpha_1 \mid \dots \mid \alpha_k \mid \alpha_1 A_i' \mid \dots \mid \alpha_k A_i'$

$Pdel = Pdel \cup A_i \rightarrow A_i \alpha_1 \mid \dots \mid A_i \alpha_k$

end

//добавление правил $A \rightarrow \beta_1 \mid \dots \mid \beta_m \mid \beta_1 A' \mid \dots \mid \beta_m A'$ в конечное множество

foreach ($A_i \rightarrow \beta_1 \mid \dots \mid \beta_m \in P \mid A_i \rightarrow \beta_1 \mid \dots \mid \beta_m \notin Pdel$) {

$P' = P' \cup A_i \rightarrow \beta_1 \mid \dots \mid \beta_m \mid \beta_1 A_i' \mid \dots \mid \beta_m A_i'$

```

 $P = P \cup A_i \rightarrow \beta_1 | \dots | \beta_m | \beta_1 A_i' | \dots | \beta_m A_i'$ 
end
//добавление правил, если не было рекурсии
if ( $A_i \notin V_r$ )
  foreach ( $R \rightarrow x_m \in \{ A_i \rightarrow \alpha_1 | \dots | \alpha_k \} \mid R \rightarrow x_i \notin Pdel$ )
     $P' = P' \cup A_i \rightarrow x_m$ 
  end
end if
end foreach

```

Пример 1.

$G = (\{a, b, d\}, \{S, C\}, P, S)$, где $P = \{S \rightarrow Sb, S \rightarrow Ca, S \rightarrow a, C \rightarrow d\}$

Расположить нетерминалы в некотором порядке:

$V = \{S, C\}$

1. $i = 1$

устранить непосредственную левую рекурсию среди A_i – продукции:

добавляем правила $A' \rightarrow \alpha_1 | \dots | \alpha_k | \alpha_1 A' | \dots | \alpha_k A'$:

$P' = \{S' \rightarrow bS', S' \rightarrow b\}$

добавляем правила $A \rightarrow \beta_1 | \dots | \beta_m | \beta_1 A' | \dots | \beta_m A'$ в P и P' :

$P' = \{S \rightarrow Ca, S \rightarrow a, S \rightarrow CaS', S \rightarrow aS', S' \rightarrow bS', S' \rightarrow b\}$

$P = \{S \rightarrow Sb, S \rightarrow Ca, S \rightarrow a, C \rightarrow d, S \rightarrow Ca, S \rightarrow a, S \rightarrow CaS', S \rightarrow aS'\}$

2. $i = 2, j = 1$

Правила отсутствуют. Рекурсия отсутствует.

Добавляем правила для C в конечное множество:

$P' = \{S \rightarrow CaS', S \rightarrow aS', S \rightarrow bS, S \rightarrow b, S \rightarrow Ca, S \rightarrow a, C \rightarrow d\}$

Получим: $G' = (\{a, b, d\}, \{S, S', C\}, P', S)$, где

$P' = \{S \rightarrow CaS, S \rightarrow aS, S \rightarrow bS, S \rightarrow b, S \rightarrow Ca, S \rightarrow a, C \rightarrow d\}$

Пример 2 : $G = (\{S, A\}, \{b, c, d\}, P, S)$ $P = \{S \rightarrow b | cA | c, A \rightarrow Ab | b | d\}$

Расположить нетерминалы в порядке : S, A

1. $i = 1$

V	S	A
i	1	
j		

$V_r = \emptyset$

рекурсия отсутствует

$P = \{S \rightarrow b, S \rightarrow cA, S \rightarrow c, A \rightarrow Ab, A \rightarrow b, A \rightarrow d\}$

$P' = \{S \rightarrow b, S \rightarrow cA, S \rightarrow c\}$

2. $i = 2, j = 1$

V	S	A
i		2
j	1	

Правила отсутствуют.

$$V_r = \{A\}$$

Устраняем рекурсию:

добавляем правила вида $A' \rightarrow \alpha_1 | \dots | \alpha_k | \alpha_1 A' | \dots | \alpha_k A'$:

$$P' = \{ S \rightarrow b, S \rightarrow cA, S \rightarrow c, A' \rightarrow bA', A' \rightarrow b \}$$

$$P_{del} = \{ A \rightarrow Ab \}$$

добавляем правила вида $A \rightarrow \beta_1 | \dots | \beta_m | \beta_1 A' | \dots | \beta_m A'$ в P и P' :

$$P' = \{ S \rightarrow b, S \rightarrow cA, S \rightarrow c, A' \rightarrow bA', A' \rightarrow b, A \rightarrow b, A \rightarrow d, A \rightarrow bA', A \rightarrow dA' \}$$

$$P = \{ S \rightarrow b, S \rightarrow cA, S \rightarrow c, A \rightarrow Ab, A \rightarrow b, A \rightarrow d, A \rightarrow b, A \rightarrow d, A \rightarrow bA', A \rightarrow dA' \}$$

Получим $G' = (\{a, b, d\}, \{S, S, C\}, P', S)$

где $P' = \{ S \rightarrow b, S \rightarrow cA, S \rightarrow c, A \rightarrow b, A \rightarrow d, A \rightarrow bA', A \rightarrow dA', A' \rightarrow bA', A' \rightarrow b \}$

Пример 3: $G = \{\{S, A\}, \{a, b, c, d\}, P, S\}$ $P = \{ S \rightarrow Aa \mid b, A \rightarrow Sd \mid Ac \}$

при $i = 1$, $V_r = \emptyset$ - рекурсии нет.

$$P = \{ S \rightarrow Aa, S \rightarrow b, A \rightarrow Sd, A \rightarrow Ac \}$$

$$P' = \{ S \rightarrow Aa, S \rightarrow b \}$$

при $i = 2, j = 1$

$$P = \{ S \rightarrow Aa, S \rightarrow b, A \rightarrow Sd, A \rightarrow Ac, A \rightarrow Aad, A \rightarrow bd \}$$

$$P_{del} = \{ A \rightarrow Sd \}$$

$$V_r = \{A\}$$

Устраняем рекурсию:

добавляем правила вида $A' \rightarrow \alpha_1 | \dots | \alpha_k | \alpha_1 A' | \dots | \alpha_k A'$:

$$P' = \{ S \rightarrow Aa, S \rightarrow b, A' \rightarrow cA', A' \rightarrow adA', A' \rightarrow c, A' \rightarrow ad \}$$

$$P_{del} = \{ A \rightarrow Sd, A \rightarrow Ac, A \rightarrow Aad \}$$

добавляем правила вида $A \rightarrow \beta_1 | \dots | \beta_m | \beta_1 A' | \dots | \beta_m A'$ в P и P' :

$$P' = \{ S \rightarrow Aa, S \rightarrow b, A' \rightarrow cA', A' \rightarrow adA', A' \rightarrow c, A' \rightarrow ad, A \rightarrow bd, A \rightarrow bdA' \}$$

$$P = \{ S \rightarrow Aa, S \rightarrow b, A \rightarrow Sd, A \rightarrow Ac, A \rightarrow Aad, A \rightarrow bd, A \rightarrow bdA' \}$$

Пример 4: $G = \{\{S, A, B\}, \{a, b, c, d\}, P, S\}$

$$P = \{ S \rightarrow Bc \mid Ad, A \rightarrow Sa \mid AbB \mid c, B \rightarrow Sc \mid b \}$$

при $i = 1$

V	S	A	B
i	1		
j			

$V_r = \emptyset$, значит левой рекурсии нет, добавляем правила в конечное множество

$$P' = \{ S \rightarrow Bc, S \rightarrow cA \}$$

$$P = \{ S \rightarrow Bc, S \rightarrow Ad, A \rightarrow Sa, A \rightarrow AbB, A \rightarrow c, B \rightarrow Sc, AB \rightarrow b \}$$

при $i = 2, j = 1$

V	S	A	B
i		2	
j	1		

такое правило есть: $A \rightarrow Sa$

добавляем в P правила $A \rightarrow Bca, A \rightarrow Ada$

$$P_{del} = \{A \rightarrow Sa\}$$

$$Vr = \{A\}$$

Устраняем рекурсию:

добавляем правила вида $A' \rightarrow \alpha_1 | \dots | \alpha_k | \alpha_1 A' | \dots | \alpha_k A'$:

$$P' = \{S \rightarrow Bc, S \rightarrow cA, A' \rightarrow da, A' \rightarrow daA'\}$$

$$P_{del} = \{A \rightarrow Ada, A \rightarrow AbB, A \rightarrow Sa\}$$

добавляем правила вида $A \rightarrow \beta_1 | \dots | \beta_m | \beta_1 A' | \dots | \beta_m A'$ в P и P' :

$$P' = \{S \rightarrow Bc, S \rightarrow cA, A' \rightarrow da, A' \rightarrow daA', A \rightarrow BcaA', A \rightarrow Bca, A \rightarrow c, A \rightarrow cA'\}$$

$$P = \{S \rightarrow Bc, S \rightarrow Ad, A \rightarrow Sa, A \rightarrow AbB, A \rightarrow c, B \rightarrow Sc, B \rightarrow b, A \rightarrow BcaA', A \rightarrow Bca, A \rightarrow c, A \rightarrow cA'\}$$

$$i = 3, j = 1$$

V	S	A	B
i			3
j	1		

такое правило есть: $B \rightarrow Sc$ добавляем в P правила: $B \rightarrow Bcc, B \rightarrow Adc$

$$P = \{S \rightarrow Bc, S \rightarrow Ad, A \rightarrow Sa, A \rightarrow AbB, A \rightarrow c, B \rightarrow Sc, B \rightarrow b, A \rightarrow BcaA', A \rightarrow Bca, A \rightarrow c, A \rightarrow cA', B \rightarrow Bcc, B \rightarrow Adc\}$$

$$P_{del} = \{A \rightarrow Ada, A \rightarrow AbB, A \rightarrow Sa, B \rightarrow Sc\}$$

$$i = 3, j = 2$$

V	S	A	B
I			3
J		2	

Находим правила $B \rightarrow Ay$: $B \rightarrow Adc$

Добавляем новые правила:

$$P = \{S \rightarrow Bc, S \rightarrow Ad, A \rightarrow Sa, A \rightarrow AbB, A \rightarrow c, B \rightarrow Sc, B \rightarrow b, A \rightarrow BcaA', A \rightarrow Bca, A \rightarrow c, A \rightarrow cA', B \rightarrow Bcc, B \rightarrow Adc, B \rightarrow cdc, B \rightarrow cA'dc, B \rightarrow BcaA'dc, B \rightarrow Bcadc\}$$

$$P_{del} = \{A \rightarrow Ada, A \rightarrow AbB, A \rightarrow Sa, B \rightarrow Sc, B \rightarrow Adc\}$$

$$Vr = \{B\}$$

Устраняем рекурсию:

Добавляем правила вида $A' \rightarrow \alpha_1 | \dots | \alpha_k | \alpha_1 A' | \dots | \alpha_k A'$:

$$(B \rightarrow BcaA'dc, B \rightarrow Bcadc, B \rightarrow Bcc)$$

$$P' = \{S \rightarrow Bc, S \rightarrow cA, A' \rightarrow da, A' \rightarrow daA', A \rightarrow BcaA', A \rightarrow Bca, A \rightarrow c, A \rightarrow cA', B' \rightarrow caA'dcB', B' \rightarrow caA'dc, B' \rightarrow cadcB', B' \rightarrow cadc, B' \rightarrow ccB', B \rightarrow cc\}$$

$$P_{del} = \{A \rightarrow Ada, A \rightarrow AbB, A \rightarrow Sa, B \rightarrow Sc, B \rightarrow Adc, B \rightarrow BcaA'dc, B \rightarrow Bcadc, B \rightarrow Bcc\}$$

Добавляем правила вида $A \rightarrow \beta_1 | \dots | \beta_m | \beta_1 A' | \dots | \beta_m A'$ в P и P' :

$$(B \rightarrow b, B \rightarrow cdc, B \rightarrow cA'dc)$$

$P' = \{S \rightarrow Bc, S \rightarrow cA, A' \rightarrow da, A' \rightarrow daA', A \rightarrow BcaA', A \rightarrow Bca, A \rightarrow c, A \rightarrow cA', B' \rightarrow caA'dcB', B' \rightarrow caA'dc, B' \rightarrow cadcB', B' \rightarrow cadc, B' \rightarrow ccB', B \rightarrow cc, B \rightarrow bB', B \rightarrow b, B \rightarrow cdcB', B \rightarrow cdc, B \rightarrow cA'dcB', B \rightarrow cA'dc\}$

$P = \{S \rightarrow Bc, S \rightarrow Ad, A \rightarrow Sa, A \rightarrow AbB, A \rightarrow c, B \rightarrow Sc, B \rightarrow b, A \rightarrow BcaA', A \rightarrow Bca, A \rightarrow c, A \rightarrow cA', B \rightarrow Bcc, B \rightarrow Adc, B \rightarrow cdc, B \rightarrow cA'dc, B \rightarrow BcaA'dc, B \rightarrow Bcadc, B \rightarrow bB', B \rightarrow b, B \rightarrow cdcB', B \rightarrow cdc, B \rightarrow cA'dcB', B \rightarrow cA'dc\}$

Алгоритмы приведения грамматик к нормальной форме Хомского и Грейбах

Алгоритм 1. Приведение грамматики к нормальной форме Хомского

1) Удаление длинных правил

Для каждого длинного правила $A \rightarrow \alpha_1\alpha_2...\alpha_k, k > 2, \alpha_i \in V \cup T$:

1. Добавляем в грамматику $k-2$ новых нетерминала $A_1, \dots, A_{k-2}$

2. Добавляем в грамматику $k-1$ новое правило:

$A \rightarrow \alpha_1A_1$

$A_1 \rightarrow \alpha_2A_2$

$A_2 \rightarrow \alpha_3A_3$

...

$A_{k-2} \rightarrow \alpha_{k-1}\alpha_k$

3. Удаляем из грамматики правило $A \rightarrow \alpha_1\alpha_2...\alpha_k$

Вход: КС $G = (T, V, P, S)$

Выход: $G' = (T, V', P', S)$

$V' = V$

$P' = \emptyset$

foreach ($A \rightarrow \alpha \in P$)

if ($A \rightarrow \alpha_1\alpha_2...\alpha_k, k > 2$)

$V' = V' \cup \{A_1, \dots, A_{k-2}\}$ // создание $k-2$ нетерминалов

$P' = P' \cup A \rightarrow \alpha_1A_1$ // создание правила $A \rightarrow \alpha_1A_1$

for ($i = 2 \dots k-2$) // создание правил вида $A_i \rightarrow \alpha_{i+1}A_{i+1}$

$P' = P' \cup A_{i-1} \rightarrow \alpha_iA_i$

end for

$P' = P' \cup A_{k-2} \rightarrow \alpha_{k-1}\alpha_k$ // создание правила $A_{k-2} \rightarrow \alpha_{k-1}\alpha_k$

else

$P' = P' \cup A \rightarrow \alpha$

end if

end foreach

- 2) Удаление бесполезных символов
- 3) Удаление ε -правил
- 4) Если начальный нетерминал встречается в правых частях правил, нужно убрать правила такого вида. Заведём новый стартовый символ S' и добавим правило $S' \rightarrow S$, а также $S' \rightarrow \varepsilon$ вместо $S \rightarrow \varepsilon$, если оно было.

Алгоритм:

$V' = V$

$P' = P$

foreach($A \rightarrow \alpha \in P$)

if ($S \in \alpha$)

$V' = V' \cup \{S'\}$

$P' = P' \cup \{S' \rightarrow S\}$

break

end if

end

- 5) Удаление цепных правил

- 6) Удаление ситуаций, в которых встречаются несколько терминалов

Алгоритм:

Для всех правил вида $A \rightarrow \alpha_1 \alpha_2$, α_i - терминал или нетерминал, заменяем все терминалы α_i на новые нетерминалы U_i и добавляем правила $U_i \rightarrow \alpha_i$.

Вход: КС $G = (T, V, P, S)$

Выход: $G' = (T, V', P', S)$

$V' = V$

$P' = \emptyset$

foreach ($A \rightarrow \alpha_1 \alpha_2 \in P$)

// если оба символа - терминалы

if ($A \rightarrow \alpha_1 \alpha_2, \alpha_1, \alpha_2 \in T$)

/* выполняем проверку на существование правил $Y \rightarrow \alpha_2$ и $X \rightarrow \alpha_1$ в P и P' , чтобы избежать дублирования правил */

// если правил вида $Y \rightarrow \alpha_2$ и $X \rightarrow \alpha_1$ нет

if ($(X \rightarrow \alpha_1 \notin P' \ \&\& \ X \rightarrow \alpha_1 \notin P \ \&\& \ Y \rightarrow \alpha_2 \notin P' \ \&\& \ Y \rightarrow \alpha_2 \notin P)$)

$V' = V \cup \{A_1, A_2\}$

$P' = P' \cup \{A \rightarrow A_1 A_2, A_1 \rightarrow \alpha_1, A_2 \rightarrow \alpha_2\}$

// если нет только правила вида $X \rightarrow \alpha_1$

```

    elif ( $X \rightarrow \alpha_1 \notin P'$  &&  $X \rightarrow \alpha_1 \notin P$ )
         $V' = V \cup \{A_1\}$ 
         $P' = P' \cup \{A \rightarrow A_1 Y, A_1 \rightarrow \alpha_1\}$ 
//  $Y \rightarrow \alpha_2$ , уже есть в P или P', поэтому в P' такое правило точно будет
// Аналогично дальше
    elif ( $Y \rightarrow \alpha_2 \notin P'$  &&  $Y \rightarrow \alpha_2 \notin P$ )
         $V' = V \cup \{A_2\}$ 
         $P' = P' \cup \{A \rightarrow X A_2, A_2 \rightarrow \alpha_2\}$ 
    else
         $P' = P' \cup \{A \rightarrow X Y, A_1 \rightarrow \alpha_1\}$ 
    end if
// если только первый символ - терминал
    elif ( $A \rightarrow \alpha_1 \alpha_2, \alpha_1 \in T$ )
        if ( $X \rightarrow \alpha_1 \notin P'$  &&  $X \rightarrow \alpha_1 \notin P$ )
             $P' = P' \cup \{A \rightarrow A_1 \alpha_2, A_1 \rightarrow \alpha_1\}$ 
             $V' = V' \cup \{A_1\}$ 
        else
             $P' = P' \cup \{A \rightarrow X \alpha_2\}$ 
        end if
    elif ( $A \rightarrow \alpha_1 \alpha_2, \alpha_2 \in T$ )
         $V' = V \cup \{A_1, A_2\}$ 
         $P' = P' \cup \{A \rightarrow \alpha_1 A_2, A_2 \rightarrow \alpha_2\}$ 
        if ( $X \rightarrow \alpha_2 \notin P'$  &&  $X \rightarrow \alpha_2 \notin P$ )
             $P' = P' \cup \{A \rightarrow \alpha_1 A_2, A_2 \rightarrow \alpha_2\}$ 
             $V' = V' \cup \{A_2\}$ 
        else
             $P' = P' \cup \{A \rightarrow \alpha_1 X\}$ 
        end if
    else
         $P' = P' \cup \{A \rightarrow \alpha_1 \alpha_2\}$ 
    end if
end foreach

```

Пример 1.

Дано: $P = \{ S \rightarrow S + F, S \rightarrow F, F \rightarrow F * L, F \rightarrow L, L \rightarrow (S), L \rightarrow i, K \rightarrow i \}$

Шаг 1.

$V' = V = \{S, F, L, K\}$

$P' = \emptyset$

1. $S \rightarrow S + F, k=3 \Rightarrow$

Добавляем в V' 1 новый нетерминал S_1 .

Добавляем в P' правила $S \rightarrow SS_1, S_1 \rightarrow +F$.

2. $S \rightarrow F, k=1 \Rightarrow$

Добавляем в P' правило $S \rightarrow F$.

Аналогично дальше:

3. $F \rightarrow F*L, k=3 \Rightarrow V' = V' \cup \{F_1\}, P' = P' \cup \{F \rightarrow FF_1, F_1 \rightarrow *L\}$

4. $F \rightarrow L, k=1 \Rightarrow P' = P' \cup \{F \rightarrow L\}$

5. $L \rightarrow (S), k=3 \Rightarrow V' = V' \cup \{L_1\}, P' = P' \cup \{L \rightarrow (L_1, L_1 \rightarrow S)\}$

6. $L \rightarrow i, k=1 \Rightarrow P' = P' \cup \{L \rightarrow i\}$

7. $K \rightarrow i, k=2 \Rightarrow P' = P' \cup \{K \rightarrow i\}$

Результат:

$V' = \{S, S_1, F, F_1, L, L_1, K\}$

$P' = \{S \rightarrow SS_1, S_1 \rightarrow +F, S \rightarrow F, F \rightarrow FF_1, F_1 \rightarrow *L, F \rightarrow L, L \rightarrow (L_1, L_1 \rightarrow S), L \rightarrow i, K \rightarrow i\}$

Шаг 2.

Удаление непроеизводящих символов.

$V_p^1 = \{K, L\}, V_p^2 = \{F_1, F, L, K\}, V_p^3 = \{S, S_1, F_1, F, L, K\}, V_p^4 = \{S, S_1, F_1, F, L, L_1, K\} \Rightarrow$ непроеизводящих символов нет

Удаление недостижимых символов

$VT_r^1 = \{S\}$

$VT_r^2 = \{S, S_1, +, F\}$

$VT_r^3 = \{S, S_1, +, F, F_1, *, L\}$

$VT_r^4 = \{S, S_1, +, F, F_1, *, L, L_1, ()\}$

$VT_r^5 = \{S, S_1, +, F, F_1, *, L, L_1, (,), i\}$

$\Rightarrow K$ - недостижимый символ $\Rightarrow$

$V'' = \{S, S_1, F, F_1, L, L_1\}, P'' = \{S \rightarrow SS_1, S_1 \rightarrow +F, S \rightarrow F, F \rightarrow FF_1, F_1 \rightarrow *L, F \rightarrow L, L \rightarrow (L_1, L_1 \rightarrow S), L \rightarrow i\}$

Шаг 3.

$P'' = \{S \rightarrow SS_1, S_1 \rightarrow +F, S \rightarrow F, F \rightarrow FF_1, F_1 \rightarrow *L, F \rightarrow L, L \rightarrow (L_1, L_1 \rightarrow S), L \rightarrow i\}$

ε -правил нет.

Шаг 4.

Вводим новый стартовый символ S' , добавляем новое правило $S' \rightarrow S$

$V''' = \{S', S, S_1, F, F_1, L, L_1\}, S'$ - начальный символ

$P''' = \{S' \rightarrow S, S \rightarrow SS_1, S_1 \rightarrow +F, S \rightarrow F, F \rightarrow FF_1, F_1 \rightarrow *L, F \rightarrow L, L \rightarrow (L_1, L_1 \rightarrow S), L \rightarrow i\}$

Шаг 5.

$V_{S'} = \{S', S, F, L\}$, $V_S = \{S, F, L\}$, $V_{S_1} = \{S_1\}$, $V_F = \{F, L\}$, $V_{F_1} = \{F_1\}$, $V_L = \{L\}$,
 $V_{L_1} = \{L_1\}$.
 $V_{S'} = \{S', S, F, L\}$ построим $\{S' \rightarrow SS_1 \mid FF_1 \mid (L_1 \mid i)\}$
 $V_S = \{S, F, L\}$ построим $\{S \rightarrow SS_1 \mid FF_1 \mid (L_1 \mid i)\}$.
 $V_{S_1} = \{S_1\} - \{S_1 \rightarrow +F\}$
 $V_F = \{F, L\} - \{F \rightarrow FF_1 \mid (L_1 \mid i)\}$
 $V_{F_1} = \{F_1\} - \{F_1 \rightarrow *L\}$
 $V_L = \{L\} - \{L \rightarrow (L_1 \mid i)\}$
 $V_{L_1} = \{L_1\} - \{L_1 \rightarrow S\}$

$P''' = \{S' \rightarrow SS_1 \mid FF_1 \mid (L_1 \mid i, S \rightarrow SS_1 \mid FF_1 \mid (L_1 \mid i, S_1 \rightarrow +F, F \rightarrow FF_1 \mid (L_1 \mid i, F_1 \rightarrow *L, L \rightarrow (L_1 \mid i, L_1 \rightarrow S))\}$

Шаг 6.

$P''' = \{S' \rightarrow SS_1 \mid FF_1 \mid (L_1 \mid i, S \rightarrow SS_1 \mid FF_1 \mid (L_1 \mid i, S_1 \rightarrow +F, F \rightarrow FF_1 \mid (L_1 \mid i, F_1 \rightarrow *L, L \rightarrow (L_1 \mid i, L_1 \rightarrow S))\}$

1. $S' \rightarrow SS_1 \mid FF_1 \mid (L_1 \mid i \Rightarrow S' \rightarrow SS_1 \mid FF_1 \mid S'_2 L_1 \mid i \ S'_2 \rightarrow (S \rightarrow SS_1 \mid FF_1 \mid (L_1 \mid i \Rightarrow$

$S \rightarrow SS_1 \mid FF_1 \mid S'_2 L_1 \mid i \ S'_2 \rightarrow ($

2. $S_1 \rightarrow +F \Rightarrow S_1 \rightarrow S_2 F \ S_2 \rightarrow +$

3. $F \rightarrow FF_1 \mid (L_1 \mid i \Rightarrow F \rightarrow FF_1 \mid S'_2 L_1 \mid i$

4. $F_1 \rightarrow *L \Rightarrow F_1 \rightarrow F_2 L \ F_2 \rightarrow *$

5. $L \rightarrow (L_1 \mid i \Rightarrow L \rightarrow S'_2 L_1 \mid i \ L_1 \rightarrow S) \Rightarrow L_1 \rightarrow S L_2 \ L_2 \rightarrow)$

Результат:

$V = \{S', S, S_1, S'_2, S_2, F, F_1, F_2, L, L_1, L_2\}$

P:

$S' \rightarrow SS_1 \mid FF_1 \mid S'_2 L_1 \mid i$

$S'_2 \rightarrow ($

$S \rightarrow SS_1 \mid FF_1 \mid S'_2 L_1 \mid i$

$S_1 \rightarrow S_2 F, S_2 \rightarrow +$

$F \rightarrow FF_1 \mid S'_2 L_1 \mid i$

$F_1 \rightarrow F_2 L, F_2 \rightarrow *$

$L \rightarrow S'_2 L_1 \mid i$

$L_1 \rightarrow S L_2$

$L_2 \rightarrow)$

3.3. Свойства КС языков: лемма о накачке

Лемма о накачке КС языков. Пусть L - контекстно-свободный язык над алфавитом Σ , тогда существует такое p , что для любой цепочки $\omega \in L$, длины

не меньше p найдется $w = uxvyz$ такое что $|xy| \geq 1$ and $|xvy| \leq p$ и для любого $i \geq 0, j \geq 0, ux^i v y^j z \in L$.

Алгоритм. Определение накачки регулярных и КС языков

foreach($\omega \in L = \{a \mid S_0 \Rightarrow^* a, a \in T^*\}$)

$\Lambda = \emptyset$

foreach ($p \geq 1$ and $p \leq |\omega|, p++$)

foreach($|xy| \geq 1$ and $|xvy| \leq p \mid uxvyz = \omega$)

foreach($i \geq 1$ and $i \leq |x|$ and $j \geq 1$ and $j \leq |y|, i++, j++$)

if ($x = a^i$ or $y = b^j \mid a, b \in T$)

Цепочка принадлежит регулярному языку

$\lambda = \{ux^i v y^j z\}$

$\Lambda = \Lambda \cup \lambda$

end if

if ($x = a^i$ and $y = b^j \mid a, b \in T$)

Цепочка принадлежит КС-языку

$\lambda = \{ux^i v y^j z\}$

$\Lambda = \Lambda \cup \lambda$

end if

end foreach

end foreach

end foreach

if ($\Lambda \neq \emptyset$)

В Λ содержатся все повторения

end if

end foreach

Рассмотрим КС язык в нормальной форме Хомского.

$S \rightarrow AB \quad A \rightarrow DE \quad B \rightarrow FD \quad E \rightarrow FG$

$G \rightarrow DB \mid g \quad F \rightarrow HG \quad D \rightarrow d \quad H \rightarrow h$

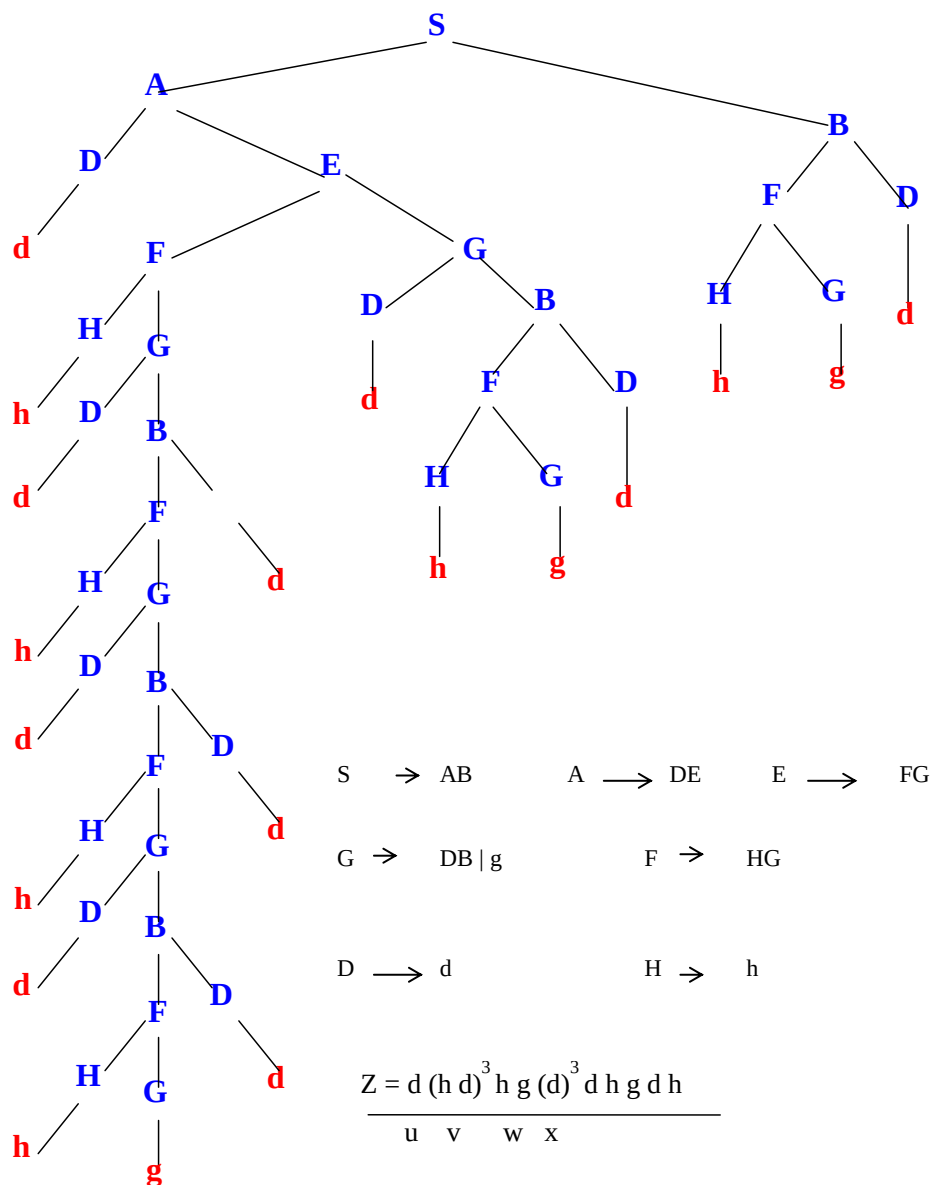


Рис. 2.6. Дерево разбора КС грамматики

Дерево вывода для строки $z = d(hd)^3hg(d)^3dghdghd$ приведено на рис.2.6. Паттерн нетерминальной последовательности FGB-FGB-FGB встречается на самом длинном стволе дерева синтаксического анализа. Он имеет повторяющийся шаблон FGB, с листьями “hd” слева и “d” справа. Можно “вырастить” стембель, добавляя или удаляя повторяющиеся шаблоны F-G-B без ограничений.

Полученное дерево является деревом синтаксического анализа грамматики. Следовательно, грамматика имеет дерево разбора для $z' = d(hd)^i hg(d)^i dghdghd$, для всех $i \geq 0$, что означает, что z' также принадлежит языку грамматики.

Пример. $Z = UV^iWX^iY$ с наибольшим возможным значением i .

!! Результат представить в виде таблицы

abcdefedcba
 aaaaaaa ddddddd
 a^7 d^7

aaabaaadaaa
aaabaaad a^3

aaab a^3 daaa
 a^3 baaadaaa
 d^3

abbbbc
 $a b^4 c$

abbbccddde
 $a b^3$ cccddde
abbb c^3 ddde
abbbccc d^3 e

abcdabcdabcdabcd
 $abcd^4$

aaabcdabcdabcdab
aa $abcd^4$ daab

aaabcdbcdbcdaaabcdbcdbcdaaabcdbcdbcd
aaabcdbcdbcdaaabcdbcdbcd a^3 bcdbcdbcd
aaabcdbcdbcd a^3 bcdbcdbcdaaabcdbcdbcd
 a^3 bcdbcdbcdaaabcdbc
 d^3 dbcdaaabcdbcdbcd
aaabcdbc d^3
dbcdaaabcdbcdbcdaaa bcd 3
aaabcdbcdbcdaaa bcd 3 aaabcdbcdbcd
aaa bcd 3 aaabcdbcd d^3 dcdaabcdbcdbcd
aaabcdbcdbcd 3

abbbbabbbbabbbbabbbb
abbbbabb d^3 bbabbbba b^4
abbbbabbbba b^4 abbbb
abbbba b^4 abbbbabbbb
 $a b^4$ abbbbab d^3 bbbabbbb
abbbb 4

abcdbcdbcdbcd
 $a bcd^5$
abcbcbcd
 $a bc^3 bd$
 $ab cb^3 d d^3 cd$

3.4. Определение МП-автоматов

Автоматы с магазинной памятью (МП - автоматы) представляют собой модель распознавателей для языков, задаваемых КС-грамматиками. МП - автоматы имеют вспомогательную память, называемую магазином см. 1.10. В магазин можно поместить неограниченное количество символов. В каждый момент времени доступен только верхний символ магазина.

Верхний символом магазина будем считать самый левый символ цепочки.

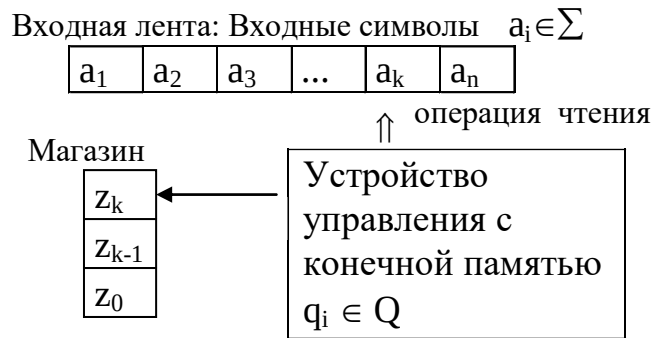


Рис. 1.10. Модель МП - автомата

Определение 9. МП автомат – это семерка объектов

$$\text{МП} = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

Q – конечное множество состояний устройства управления;

Σ – конечный алфавит входных символов;

Γ – конечный алфавит магазинных символов;

δ – функция переходов, отображает множества $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma$ в множество конечных подмножеств множества $Q \times \Gamma^*$, $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow Q \times \Gamma^*$

q_0 – начальное состояние, $q_0 \in Q$;

z_0 – начальный символ магазина, $z_0 \in \Gamma$;

F – множество заключительных состояний, $F \subseteq Q$.

Определение 10. Конфигурацией МП-автомата называется тройка $(q, \omega, z) \in Q \times \Sigma^* \times \Gamma^*$, где

q – текущее состояние управляющего устройства;

ω – необработанная часть входной цепочки (первый символ цепочки ω находится под входной головкой; если $\omega = \varepsilon$, то считается, что вся входная цепочка прочитана) ;

z – содержимое магазина (самый левый символ цепочки z считается верхним символом магазина; если $z = \varepsilon$, то магазин считается пустым).

Такт МП-автомата будем описывать бинарным отношением $\vdash$, определенным на множестве конфигураций. Будем писать:

$$(q, a\omega, \gamma z) \vdash (q', \omega, az), \text{ если } \delta(q, a, z) = (q', a\gamma z), \text{ где} \\ q, q' \in Q, a \in \Sigma \cup \{\varepsilon\}, \omega \in \Sigma^*, z \in \Gamma \text{ и } \gamma \in \Gamma^*$$

Если $a \neq \varepsilon$, то и входная цепочка прочитана не вся, то запись $(q, a\omega, \gamma z) \vdash (q', \omega, a\gamma)$ означает, что МП-автомат в состоянии q , обозревая символ во входной цепочки и имея символ z в вершине магазина, может перейти в новое состояние

q' , сдвинуть входную головку на один символ вправо и заменить верхний символ магазина z символом a и цепочкой магазинных символов γ .

Если $z = a$, то верхний символ удаляется из магазина.

Если $a = \varepsilon$, то текущий входной символ в этом такте называется ε -тактом, не принимается во внимание и входная головка остается неподвижной.

ε -такты могут выполняться также в случае, когда вся входная цепочка прочитана, но если магазин пуст, то такт МП-автомата невозможен по определению.

Так же, как и для конечных автоматов, можно определить транзитивное $\vdash^+$ и рефлексивно-транзитивное $\vdash^*$ замыкания (отношения $\vdash$).

Начальной конфигурацией МП-автомата называется конфигурация вида (q_0, ω, z_0) , где устройство управления находится в начальном состоянии, на входной ленте записана цепочка $\omega \in \Sigma^*$, которую необходимо распознать, а магазин содержит только начальный символ z_0 .

Заключительной конфигурацией МП-автомата называется конфигурация вида (q, ε, γ) , где $q \in F$ – одно из заключительных состояний устройства управления, входная цепочка прочитана до конца, а в магазине записана некоторая, заранее определенная цепочка $\gamma \in \Gamma^*$.

Есть два способа определить язык, допускаемый МП-автоматом:

1. множеством входных цепочек, для которых существует опустошающая магазин последовательность операций;

2. множеством входных цепочек, для которых существует последовательность операций, приводящая автомат в заключительное состояние.

Цепочка $\omega \in \Sigma^*$ допускается МП-автоматом $МП = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$, если $(q_0, \omega, z_0) \vdash^* (q, \varepsilon, \gamma)$ для некоторых $q \in F$ и $\gamma \in \Gamma^*$.

Язык распознаваемый МП-автоматом, называется множество цепочек:

$$L(МП) = \{ \omega \mid \omega \in \Sigma^* \text{ и } (q_0, \omega, z_0) \vdash^* (q, \varepsilon, \gamma) \text{ для некоторых } q \in F \text{ и } \gamma \in \Gamma^* \}$$

Пример. Определим МП-автомат, допускающий язык $L = \{a^n b^n \mid n \geq 0\}$

Цепочка символов языка L : aaabbb

$$МП = (\{q_0, q, q_f\}, \{a, b\}, \{z_0, a\}, \delta, q_0, z_0, \{q_f\})$$

$$1. \delta(q_0, a, z_0) = \{(q_1, az_0)\}$$

$$2. \delta(q, a, a) = \{(q, aa)\}$$

$$3. \delta(q, b, a) = \{(q_2, \varepsilon)\}$$

$$4. \delta(q, \varepsilon, z_0) = \{(q_f, \varepsilon)\}$$

Последовательность тактов:

$$\begin{aligned} (q_0, aaabbb, z_0) &\vdash^1 (q, aabbb, az_0) \vdash^2 (q, abbb, aaz_0) \vdash^3 \\ (q, bbb, aaaz_0) &\vdash^3 (q, bb, aaz_0) \vdash^3 (q, b, az_0) \vdash^3 \\ (q, \varepsilon, z_0) &\vdash^4 (q_f, \varepsilon) \end{aligned}$$

Алгоритм 3.8. Алгоритм построение МП-автомата из КС-грамматики. По КС-грамматике $G = (T, V, P, S)$ можно построить МП автомат, $L(МП) = L(G)$. Пусть $МП = (\{q\}, \Sigma, \Sigma \cup V, \delta, q, S, \{q\})$, где δ определяется следующим образом:

1. Если $A \rightarrow \alpha$ - правило грамматики G , то $\delta(q, \varepsilon, A) = (q, \alpha)$.

2. Если $A \rightarrow a$, $\delta(q, a, a) = \{(q, \varepsilon)\}$ для всех $a \in T$.

Алгоритм 3.9. Алгоритм построение МП-автомата из КС-грамматики с альтернативами:

Вход: $G = (V, T, P, S)$

Выход: МП = $(\{q\}, \Sigma, \Sigma \cup V, \sigma, q, S, \{q\})$

$Q = \{q, qf\}$

$\Sigma = T$

$\Gamma = V \cup T$

$F = \{qf\}$

$Z_0 = S$

$\delta = \emptyset$

foreach($A \rightarrow \alpha | \beta \in P$)

If $((\alpha \in T^* \text{ and } a \in \alpha) \text{ or } (\beta \in T^* \text{ and } b \in \beta))$

$\delta = \delta \cup \{\delta(q, a, a) = \{(q, \varepsilon)\}\}$

else

$\delta = \delta \cup \{\delta(q, \varepsilon, A) = \{(q, \alpha), (q, \beta)\}\}$

end

Рассмотрим на примере. Построим МП-автомат и расширенный МП-автомат по КС-грамматике $G = (T, V, P, S)$, без левой рекурсии, автомат распознает скобочные выражения. Написать последовательность тактов для выделенной цепочки. Определить свойства автомата.

1. А). Построить МП-автомат, распознающий скобочные арифметические выражения заданные КС-грамматикой $G_1 = (\{v, +, *, (,)\}, \{S_0, E, F, L\}, P, S_0)$, где P состоит из правил:

$p_1: S_0 \rightarrow (E)^*L \mid F, p_2: E \rightarrow E+F \mid w, p_3: F \rightarrow F * L \mid L, p_4: L \rightarrow v.$

$S_0 \Rightarrow^1 (E)^*L \Rightarrow^2 (E+F)^*L \Rightarrow^3 (w+F)^*L \Rightarrow^4 (w+L)^*L \Rightarrow^5 (w+v)^*L \Rightarrow^6 (w+v)^*v$

Используя алгоритм 3.8. получим: МП = $(\{q\}, \{w, v, +, *, (,)\}, \{w, v, +, *, (,), S_0, E, F, L\}, \delta, S_0, z_0, \{q\})$, в котором функция переходов δ определяется следующим образом:

1. $\delta(q_0, \varepsilon, S_0) = \{(q, (E)^*L), (q, F)\};$
2. $\delta(q, \varepsilon, E) = \{(q, E+F), (q, w)\};$
3. $\delta(q, \varepsilon, F) = \{(q, F*L), (q, L)\};$
4. $\delta(q, \varepsilon, L) = \{(q, v)\};$
5. $\delta(q, a, a) = \{(q, \varepsilon)\}$ для всех $a \in \Sigma = \{w, v, +, *, (,)\}.$

В). Последовательность тактов МП-автомата для цепочки $(w+v)^*v$:

$(q_0, (w+v)^*v, S_0) \xrightarrow{1} (q, (w+v)^*v, (E)^*L) \xrightarrow{5} (q, w+v)^*v, E)^*L \xrightarrow{2} (q, w+v)^*v, E+F)^*L \xrightarrow{2} (q, (q, w+v)^*v, w+F)^*L \xrightarrow{5} (q, +v)^*v, +F)^*L \xrightarrow{5} (q, v)^*v, F)^*L \xrightarrow{3} (q, v)^*v, L)^*L \xrightarrow{4} (q, v)^*v, v)^*L \xrightarrow{5} (q,)^*v,)^*L \xrightarrow{5} (q, *v, *L) \xrightarrow{5} (q, v, L) \xrightarrow{4} (q, v, v) \xrightarrow{5} (q, \varepsilon)$

Последовательность тактов, которую выполняет МП-автомат, соответствует левому выводу цепочки $v * (v+v)$ в грамматике G_1 .

Тип синтаксических анализаторов, которые можно построить таким образом называют *нисходящим* (предсказывающим) анализатором.

Синтаксические анализаторы, построенные на основе расширенного МП-автомата, называют *восходящими* анализаторами, вывод строится снизу в верх.

Определение 11. Расширенным РМП-автоматом называется семерка объектов $РМП = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$, где верхним элементом магазина является самый правый символ цепочки, δ -функция переходов, которая задает отображение множества $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma$ в множество конечных подмножеств множества $Q \times \Gamma^*$, а все остальные объекты такие же, как и у МП-автомата см. определение 9.

Конфигурация расширенного МП-автомата определяется так же, как и для МП-автомата.

Алгоритм 3.9. По КС-грамматике $G = (T, V, P, S)$ можно построить расширенный РМП автомат, $L(РМП) = L(G)$. Будем обозначать символом $\perp$ - конец цепочки. Пусть $РМП = (\{q, r\}, \Sigma, \Sigma \cup V \cup \perp, \delta, q, \perp, \{r\})$, где δ определяется следующим образом:

1. $\delta(q, a, \epsilon) = \{(q, a)\}$ для всех $a \in \Sigma$ (символы с входной ленты на этих тактах переносятся в магазин).

2. Если $A \rightarrow \alpha$ - правило вывода грамматики G , то $\delta(q, \epsilon, \alpha) = (q, A)$.

3. $\delta(q, \epsilon, \perp S) = \{(r, \epsilon)\}$

РМП-автомат продолжает работу пока магазин не станет пустым.

На практике часто используются детерминированные МП-автоматы.

Пример построения расширенного РМП-автомата

А). Построение расширенного РМП-автомата $(Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$, используем алгоритм 3.9, получим $РМП = (\{q, r\}, \{v, +, *, (,)\}, \{v, +, *, (,), S_0, E, F, L, \perp\}, \delta, q, \perp, \{r\})$, где функция переходов δ определена следующим образом:

1. $\delta(q, a, \epsilon) = \{(q, a)\}$ для всех $a \in \Sigma = \{w, v, +, *, (,)\}$

2. $\delta(q, \epsilon, (E)*L) = \{(q, S_0)\};$

3. $\delta(q, \epsilon, F) = \{(q, S_0)\};$

4. $\delta(q, \epsilon, E+F) = \{(q, E)\};$

5. $\delta(q, \epsilon, w) = \{(q, E)\};$

6. $\delta(q, \epsilon, F*L) = \{(q, F)\};$

7. $\delta(q, \epsilon, L) = \{(q, F)\};$

8. $\delta(q, \epsilon, v) = \{(q, L)\};$

9. $\delta(q, \epsilon, \perp S_0) = \{(r, \epsilon)\};$

В). При анализе входной цепочки $(w+v)*v$ расширенный РМП-автомат выполнит следующую последовательность тактов:

$(q, (w+ v)*v, \perp) \xrightarrow{1} (q, w+ v)*v, \perp()$

$\xrightarrow{1} (q, w+ v)*v, \perp(w)$

$\xrightarrow{5} (q, + v)*v, \perp(E)$

$\xrightarrow{1} (q, v)*v, \perp(E+)$

$\xrightarrow{1} (q,)*v, \perp(E+v)$

$$\begin{aligned}
& \vdash^8 (q,)^*v, \perp (E+L) \\
& \vdash^7 (q,)^*v, \perp (E+F) \\
& \vdash^4 (q,)^*v, \perp (E) \\
& \vdash^1 (q, ^*v, \perp (E)) \\
& \vdash^1 (q, v, \perp (E)^*) \\
& \vdash^1 (q, \varepsilon, \perp (E)^*v) \\
& \vdash^8 (q, \varepsilon, \perp (E)^*L) \\
& \vdash^2 (q, \varepsilon, \perp S_0) \\
& \vdash^9 (r, \varepsilon)
\end{aligned}$$

МП-автомат и расширенный РМП-автоматы – детерминированные автоматы. Обычно синтаксический анализ выполняется путем моделирования МП-автомата, анализирующего входные цепочки.

МП-автомат отображает входные цепочки в соответствующие левые выводы (нисходящий анализ) или правые разборы (восходящий анализ).

Пусть задана КС-грамматика $G = (T, V, P, S)$, правила которой пронумерованы числами $1, 2, \dots, p$, и цепочка $\alpha \in (T \cup V)^*$,

- левым разбором цепочки α называется последовательность правил, примененных при ее левом выводе из S ;
- правым разбором цепочки α называется обращение последовательности правил, примененных при ее левом выводе из S .

Определение 12. Детерминированным МП-автоматом (ДМП-автоматом) с магазинной памятью называется МП-автомат $\text{ДМП} = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$, у которого для каждого $q \in Q$ и $z \in \Gamma$ выполняется одно из условий:

1. $\delta(q, a, z)$ содержит не более одного элемента для каждого $a \in \Sigma$ и $\delta(q, \varepsilon, z) = \emptyset$;
2. $\delta(q, a, z) = \emptyset$ для всех $a \in \Sigma$, и $\delta(q, \varepsilon, z)$ содержит не более одного элемента.

Если каждое правило КС-грамматики начинается с терминального символа, причем альтернативы начинаются с различных символов, то достаточно просто построить *детерминированный* синтаксический анализатор.

3.5. Способы реализации синтаксических анализаторов

Рассмотрим три способа реализации синтаксических анализаторов для заданного языка $L(G)$. Каждый имеет свои достоинства и недостатки. Напомним, что G является приведенной.

1. МП-автомат строим для $L(G) = L(\text{МП})$, правила автомата строятся в соответствии с рассмотренными алгоритмами.

2. Строятся диаграммы Вирта. Для каждого правила грамматики строится диаграмма. Не терминальные символы реализуются вызовом подпрограмм. Таким образом создается неявный стек при их вызове и возможна косвенная рекурсия, что не требует преобразования грамматики в МП автомат.

3. Таблично-управляемый разбор (предварительно строится диаграмма Вирта), диаграмма сопоставляется с узлами и ребрами графа, что реализуется

списочной структуры, над которой выполняется алгоритм обхода графа соответствующий распознаванию входной строки.

Диаграммы Вирта

1. Постановка задачи. Реализовать синтаксический анализатор левым разбором, используя приведенную грамматику без левой рекурсии $G = (\{a, b, c, -, +, (,)\}, \{E, T, F, \text{ор}\}, P, S)$, где $P = \{E \rightarrow T \text{ ор } T, \text{ор} \rightarrow + \mid -, T \rightarrow F \mid (E), F \rightarrow a \mid b \mid c\}$

2. Шаги реализации.

Выделить цепочку, принадлежащую языку задаваемому КС-грамматикой G , например: $L(G) = a + (b - c)$.

Привести вывод выделенной цепочки:

$E \Rightarrow T \text{ ор } T \Rightarrow T \text{ ор } (E) \Rightarrow F \text{ ор } (T \text{ ор } T) \Rightarrow a \text{ ор } (T \text{ ор } T) \Rightarrow a + (T \text{ ор } T) \Rightarrow a + (F \text{ ор } T) \Rightarrow a + (b \text{ ор } T) \Rightarrow a + (b \text{ ор } F) \Rightarrow a + (b \text{ ор } c) \Rightarrow a + (b - c)$

Пронумеровать правила грамматики и представить их, используя метасимволы $\{\dots\}$, $:=$ расширенной формы Бекуса-Наура:

$p_1: E := \{T \text{ ор } T\}$, $p_2: \text{ор} := \{+, -\}$, $p_3: T := \{F, (E)\}$, $p_4: F := \{a, b, c\}$

Составить синтаксический граф, диаграмму Вирта для детерминированного грамматического разбора с просмотром вперед на один символ. Никакие две ветви не должны начинаться с одного и того же символа, если какой-либо граф A можно пройти, не читая вообще никаких входных символов, то такая “нулевая ветвь” должна помечаться всеми символами, которые могут следовать за A (это необходимо для принятия решения о переходе на эту ветвь).

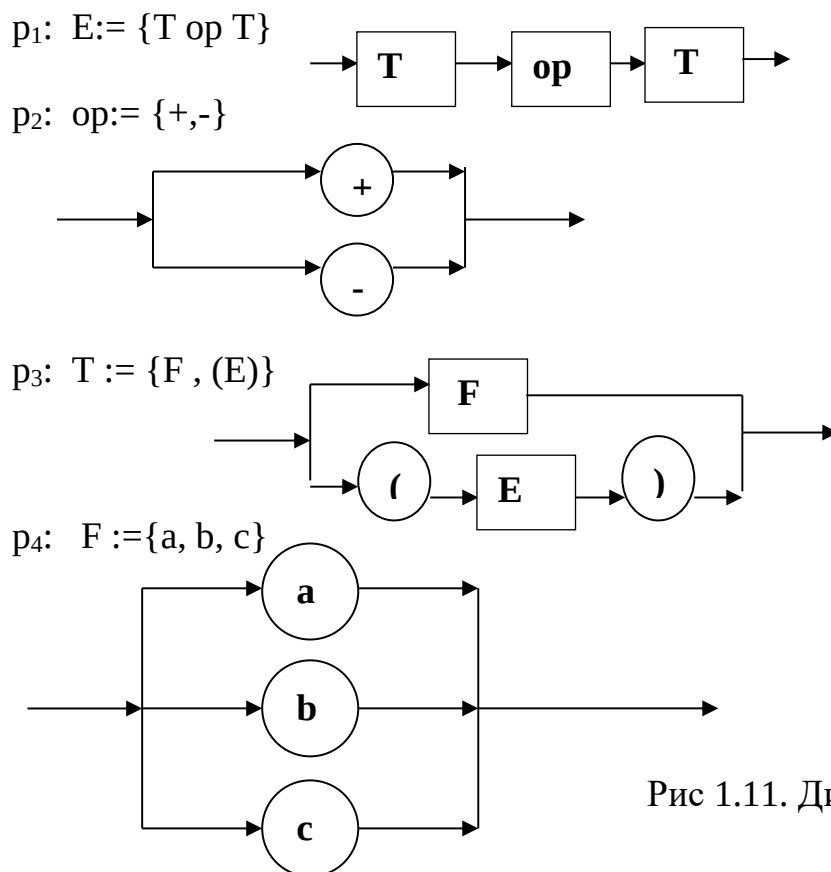
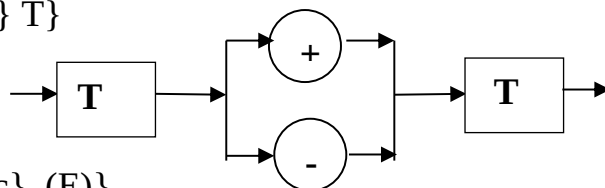


Рис 1.11. Диаграммы Вирта.

Правила преобразования графа в программу:

1. Свести систему графов к возможно меньшему числу отдельных графов с помощью соответствующих подстановок. Сделаем две подстановки, p_2 подставим в p_1 , p_4 подставим в p_3 :

$p_1: E := \{T \{+, -\} T\}$



$p_4: T := \{\{a, b, c\}, (E)\}$

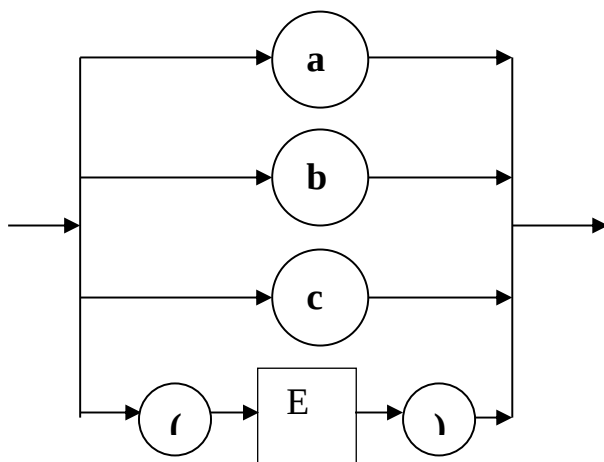


Рис 1.12. Преобразованные диаграммы Вирта.

Полученные правила на графе соответствуют нормальной форме Грейбах.

Преобразовать каждый синтаксический граф в описание процедуры в соответствии с правилами:

1. Каждому графу (диаграмме Вирта) ставится в соответствие процедура.
2. Элемент графа, обозначающий другой граф, переводится в оператор обращения к процедуре.
3. Последовательность элементов в последовательный вызов операторов. Выбор элементов в switch или условный оператор if.

Входные и выходные данные. Пользователь вводит строку в поле редактирования и получает ответ: Да – строка принадлежит языку, порождаемому данной грамматикой, и Нет – строка не принадлежит этому языку. Программа распознает язык $L(G)$ по заданной КС-грамматике G .

Таблично-управляемый разбор

1. Грамматика задается в виде данных, процедура разбора универсально для всех грамматик. Программа работает в строгом соответствии с методом простого нисходящего грамматического разбора и основывается на детерминированном синтаксическом графе (т.е. предложение должно анализироваться просмотром вперед на один символ без возврата).

2. Естественный способ представить граф в виде структуры данных, а не программ – это ввести узел для каждого символа и связать эти узлы с помощью ссылок. Выделим два типа узлов для терминальных (идентифицируется терминальным символом) и нетерминальных символов (ссылка на данные нетерминального символа). Тогда структура узла графически может быть представлена как:

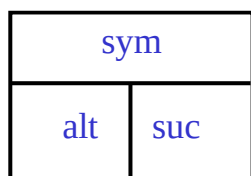


Рис. 1. 14. Структура узла Node

Где “suc” - последователь, “alt” – альтернатива. Пустую последовательность обозначим “empty”. Конструируется класс Node (Узел) для данной структуры.

3. Правила преобразования графов, в структуру данных: получить как можно меньшее число графов с помощью подстановок.

4. Последовательность элементов преобразовать в список узлов (горизонтальный по “suc”), список альтернатив в список узлов (горизонтальный по “alt”), цикл в узел с указателем “suc” на себя, а “alt” на узел с “empty”, где “alt” = NULL, а “suc” выход из цикла.

5. Построим реализацию в виде структуры данных для рассмотренных в п.3 диаграмм.

6. Программа, реализующая структуру на рис. 1.15 приведена ниже. Булевская переменная b управляет алгоритмом разбора. Переменная state определяет состояние цепочки символов.

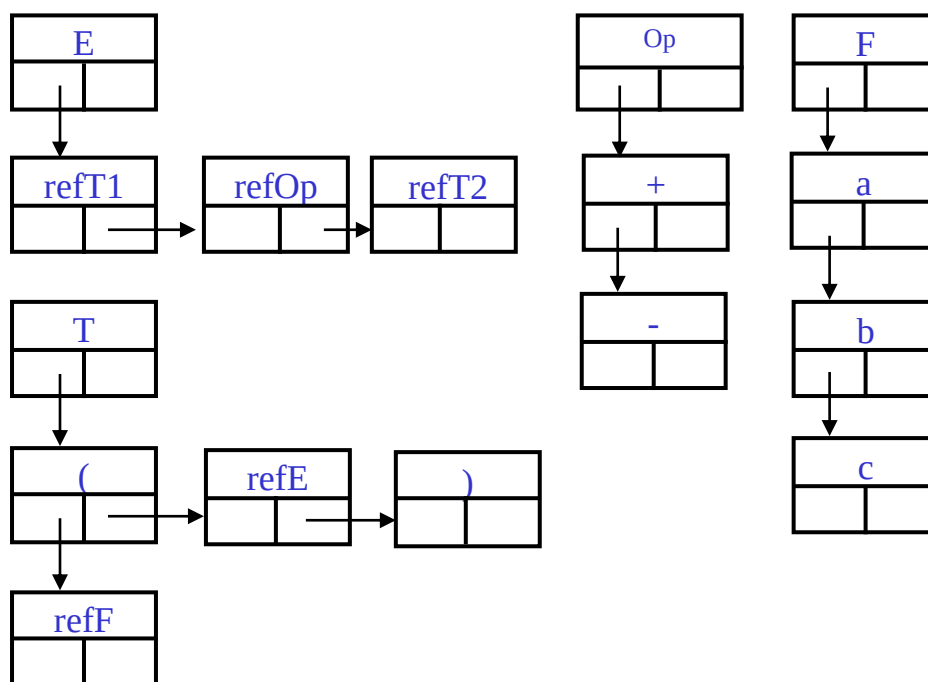


Рис. 1.15. Структура данных для диаграмм п.5

На рис. 1.15. E – начальная вершина Node, а вершины вида refX содержат ссылки на другие вершины. Например, refE содержит ссылку на E, а refT1 и refT2 – на T. см. Приложение А.

3.6. LL(k)-грамматики

Синтаксический LL-анализатор - анализирует цепочку символов входного алфавита на ленте слева (L) направо, и строит левый (L) вывод грамматики.

Определение 13. КС-грамматика $G = (T, V, P, S)$ без ϵ -правил называется простой LL(1) грамматикой (S (простая от англ. simple)-грамматикой, разделенной грамматикой), если для каждого $v \in V$ все его альтернативы начинаются различными терминальными символами. Единица в названии алгоритма означает, что при чтении анализируемой цепочки, находящейся на входной ленте, входная головка может заглядывать вперед на один символ.

Если грамматика не удовлетворяет требованиям, то применяют факторизацию - представляет собой преобразование грамматики в пригодную для предиктивного анализа. Основная идея левой факторизации заключается в том, что когда не ясно, какая из двух альтернативных продукций должна использоваться для нетерминала A, A-продукции можно переписать так, чтобы отложить принятие решения до тех пор, пока из входного потока не будет прочитано достаточно символов для правильного выбора.

Алгоритм. Левая факторизация грамматики

Вход: грамматика G.

Выход: эквивалентная левофакторизованная грамматика.

Метод: для каждого нетерминала A находим самый длинный префикс α , общий для двух или большего числа альтернатив. Если $\alpha \neq \epsilon$, т.е. имеется нетривиальный общий префикс, заменим все продукции $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$, где γ представляет все альтернативы, не начинающиеся с α , продуктами

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

Здесь A' — новый нетерминал. Выполняем это преобразование до тех пор, пока никакие две альтернативы нетерминала не будут иметь общий префикс.

Рассмотрим частный случай функции FIRST:

FIRST(A) – это множество первых терминальных символов, которыми начинаются цепочки, выводимые из нетерминала $A \in V$:

$$\text{FIRST}(A) = \{a \in T \mid A \Rightarrow^+ a\beta, \text{ где } \beta \in (T \cup V)^*\}$$

Обобщим определение множества FIRST так, чтобы его можно было применить для правил произвольного вида. Множество FIRST(α) состоит из множества терминальных символов, которыми начинаются цепочки, выводимые из цепочки α .

$$\text{FIRST}(\alpha) = \{a \in T \mid S \Rightarrow^+ \alpha \Rightarrow^+ a\beta, \text{ где } \alpha \in (T \cup V)^+, \beta \in (T \cup V)^*\}$$

FOLLOW(A) – это множество следующих терминальных символов, которые могут встретиться непосредственно справа от нетерминала в некоторой сентенциальной форме:

$$\text{FOLLOW}(A) = \{a \in T \mid S \Rightarrow^* \alpha A \gamma \text{ и } a = \text{FIRST}(\gamma)\}$$

Пример. $G = (V = \{S, F, L\}, T = \{i, =, *, (,)\}, P, S)$

P:

1. $S \rightarrow F=L$
2. $S \rightarrow L$
3. $F \rightarrow (*L)$
4. $F \rightarrow i$
5. $L \rightarrow F$

Грамматика не принадлежит классу LL(1), так как для правил $S \rightarrow F=L \mid L$:
 $FIRST(F=L) = FIRST(L) = FIRST(F) = \{ (, i \}$

Применив алгоритм факторизации правилам $S \rightarrow F=F \mid F$, заменяем их на правила $S \rightarrow FS'$, $S' \rightarrow =F \mid \varepsilon$. Получим грамматику без неопределённостей:
 $G = (\{S, S', F\}, \{i, =, *, (,)\}, P, S)$

P:

1. $S \rightarrow FS'$
2. $S' \rightarrow =F$
3. $S' \rightarrow \varepsilon$
4. $F \rightarrow (*F)$
5. $F \rightarrow i$

1. Построение функции FIRST

$FIRST(S) = FIRST(F) = \{ (, i \}$

$FIRST(S') = \{ =, \varepsilon \}$

$FIRST(F) = \{ (, i \}$

$FIRST(FS') = FIRST(F) = \{ (, i \}$

$FIRST((*F)) = \{ (\}$

1. Построение функции FOLLOW

$FOLLOW(S) = \{ \perp \}$

$FOLLOW(S') = \{ \perp \}$

$FOLLOW(F) = FIRST(S') \setminus \{ \varepsilon \} \cup FOLLOW(S') \cup \{) \} = \{ =, \perp,) \}$

LL(k) анализатор. Пусть магазин содержит цепочку $Xa\perp$ (см. рис. 1.17), где Xa – цепочка магазинных символов (X - верхний символ магазина), а символ ($\perp$) – специальный символ, называемый *маркером дна* магазина. Если верхним символом магазина является *маркер дна*, то магазин пуст. Выходная лента содержит цепочку номеров правил π , представляющую собой текущее состояние левого разбора.

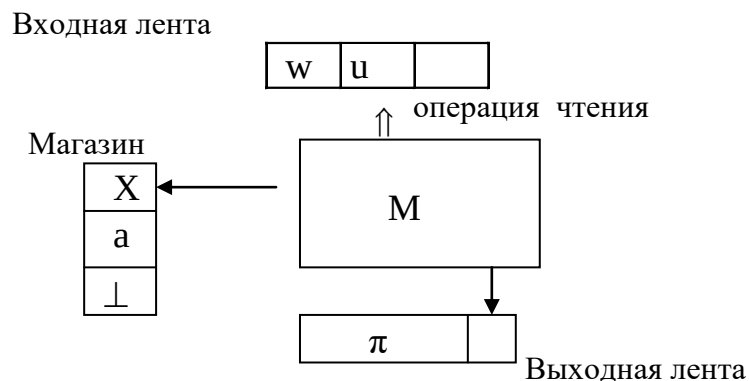


Рис. 1.18. LL(k)-анализатор

Конфигурацию “1-предсказывающего” алгоритма разбора будем представлять в виде $(x, Xa\perp, \pi)$, где x – неиспользуемая часть входной цепочки, Xa – цепочка в магазине, а π – цепочка на выходной ленте, отражающая состояние алгоритма.

Обозначим алфавит магазинных символов (без символа $(\perp)$) как V_p .

M – управляющая таблица управляет работой алгоритма. M задает отображение множества $(V_p \cup T \cup \{\perp\}) \times (T \cup \{\epsilon\})$ в множество, состоящее из следующих элементов:

1. (β, i) , где βV_p^* правая часть правила вывода с номером i .
2. ВЫБРОС.
3. ДОПУСК.
4. ОШИБКА.

Работа алгоритма в зависимости от элемента управляющей таблицы $M(X, a) = (\beta, i)$ следующая:

1. $(x, Xa, \pi) \xrightarrow{i} (x, \beta a, \pi i)$, если $M(X, a) = (\beta, i)$. В этом случае верхний символ магазина X заменяется на цепочку βV_p^* , и в выходную цепочку дописывается номер правила i . Выходная головка при этом не сдвигается.

2. $(ax, a\alpha, \pi) \xrightarrow{} (x, \alpha, \pi)$, если $M(a, a) = \text{ВЫБРОС}$. Это означает, что, если верхний символ магазина совпадает с текущим входным символом, он выбрасывается из магазина, и входная головка сдвигается на один символ вправо.

3. Если алгоритм достигает конфигурации $(\epsilon, \perp, \pi)$, что соответствует элементу управляющей таблицы $M(\perp, \epsilon) = \text{ДОПУСК}$, то его работа прекращается, и выходная цепочка π является левым разбором входной цепочки.

4. Если алгоритм достигает конфигурации (x, Xa, π) и $M(X, a) = \text{ОШИБКА}$, то разбор прекращается и выдается сообщение об ошибке.

Конфигурация $(\omega, S\perp, \epsilon)$, где $S \in V_p$ – начальный символ магазина (начальный символ грамматики), называется *начальной конфигурацией*.

Если $(\omega, S\perp, \epsilon) \xrightarrow{+} (\epsilon, \perp, \pi)$, то π называется выходом алгоритма для входа ω .

Алгоритм 3.10. Построение управляющей таблицы M для LL(1)-грамматики

Вход: LL(1)-грамматика $G = (T, V, P, S)$

Выход: Управляющая таблица M для грамматики G .

Таблица M определяется на множестве $(V \cup T \cup \{\perp\}) \times (T \cup \{\epsilon\})$ по правилам:

1. Для каждого терминала $a \neq \varepsilon$ из $\text{FIRST}(\alpha)$, $A \rightarrow \alpha$ – правило грамматики с номером i , добавляем в $M(A, a) = (\alpha, i)$.

Если $\varepsilon \in \text{FIRST}(\alpha)$, то для каждого терминала b из $\text{FOLLOW}(A)$ добавляем $A \rightarrow \alpha$ в $M(A, b) = (\alpha, i)$. Если $\varepsilon \in \text{FIRST}(\alpha)$ и $\varepsilon = \text{FOLLOW}(A)$, то добавляем $A \rightarrow \alpha$ в $M(A, \varepsilon)$.

2. $M(a, a) = \text{ВЫБРОС}$ для всех $a \in T$.

3. $M(\perp, \varepsilon) = \text{ДОПУСК}$.

4. В остальных случаях $M(X, a) = \text{ОШИБКА}$ для $X(V \cup T \cup \{\perp\})$ и $a \in T \cup \{\varepsilon\}$

Естественным обобщением LL(1)-грамматик являются LL(k)-грамматики. Для КС-грамматики $G = (T, V, P, S)$ определим множество:

$$\text{FIRST}_k(\alpha) = \{x \mid \alpha \Rightarrow_i^* x\beta \text{ и } |x| = k \text{ или } \alpha \Rightarrow_i^* x \text{ и } |x| < k\}$$

Применение алгоритма 3.10. требует использование аванцепочек длиной до k символов, что существенно увеличивает размеры управляющей таблицы.

Кроме того, для некоторых LL(k)-грамматик ($k > 1$) верхний символ магазина и аванцепочка длиной k (или меньше) символов не всегда однозначно определяют правило, которое должно быть определено при разборе.

Если в КС-грамматике $G = (T, V, P, S)$ для двух различных A-правил $A \rightarrow \beta$ и $A \rightarrow \gamma$ выполняется:

$$\text{FIRST}_k(\beta \text{ FOLLOW}_k(A)) \cap \text{FIRST}_k(\gamma \text{ FOLLOW}_k(A)) = \emptyset, \text{ то такая}$$

КС-грамматика называется *сильно* LL(k)-грамматикой.

Сложность построения синтаксических анализаторов и их неэффективность не позволяют практически использовать LL(k)-грамматики.

Проектирование LL(k)-анализатора.

А) Построить управляющую таблицу M для грамматики $G = (T, V, P, S)$, где $P = \{p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8\}$. $p_1: S \rightarrow TE'$, $p_2: E' \rightarrow +TE'$, $p_3: E' \rightarrow \varepsilon$, $p_4: T \rightarrow PT'$, $p_5: T' \rightarrow *PT'$, $p_6: T' \rightarrow \varepsilon$, $p_7: P \rightarrow (S)$, $p_8: P \rightarrow i$

В) Рассмотреть работу алгоритма для выделенной цепочки.

С) Определить является ли LL(k)-грамматика *сильно* LL(k)-грамматикой.

А) Используем алгоритм 3.10. Управляющая таблица должна содержать 11 строк, помеченных символами из множества $(V \cup T \cup \{\perp\})$, и 6 столбцов, помеченных символами из множества $(T \cup \{\varepsilon\})$.

	i	(	)	+	*	ε
S	TE',1	TE', 1				
E'			$\varepsilon, 3$	+TE',2		$\varepsilon,3$
T	PT',4	PT',4				
T'			$\varepsilon, 6$	$\varepsilon, 6$	*PT',5	$\varepsilon,6$
P	i, 8	(S),7				

i	ВЫБРОС					
(		ВЫБРОС				
)			ВЫБРОС			
+				ВЫБРОС		
*					ВЫБРОС	
⊥						ДОПУСК

Шаг 1. Строим таблицу по рядам. Последовательно рассмотрим все нетерминальные символы.

1. Нетерминалу S соответствует правило вывода грамматики $p_1: S \rightarrow TE'$. Так как $FIRST(TE') = \{ (, i \}$, два терминальных символа, то:

$$M(S, () = M(S, i) = TE', 1$$

2. Для нетерминала E' в грамматике имеются два правила вывода p_2 и p_3 :

$p_2: E' \rightarrow +TE'$ множество $FIRST(+TE') = \{ + \}$ и, следовательно, $M(E', +) = +TE', 2;$

$p_3: E' \rightarrow \epsilon$ имеет простую правую часть, вычислим множество символов, следующих за нетерминалом E' в sentential forms. Построив левый вывод $S \Rightarrow TE' \Rightarrow PT'E' \Rightarrow (S)TE' \Rightarrow (TE')TE' \dots$, имеем

$$FOLLOW(E') = \{), \epsilon \}.$$

Таким образом, $M(E',)) = M(E', \epsilon) = (\epsilon, 3).$

Выполняя шаг 1 алгоритма для нетерминалов T, T' и P получим:

Правило грамматики	Множество	Значение M
$p_4: T \rightarrow PT'$ $p_5: T' \rightarrow *PT'$ $p_6: T' \rightarrow \epsilon$	$FIRST(PT') = \{ (, i \}$ $FIRST(*PT') = \{ * \}$ $FOLLOW(T') = \{ +,), \epsilon \}$	$M(T', () = M(T', i) = TE', 4$ $M(T', *) = *PT', 5$ $M(T', +) = M(T',)) = M(T', \epsilon) = \epsilon, 6$
$p_7: P \rightarrow (S)$ $p_8: P \rightarrow i$	$FIRST((S)) = \{ (\}$ $FIRST(i) = \{ i \}$	$M(P, () = (S), 7$ $M(P, i) = i, 8$

Шаг 2. Далее всем элементам таблицы, находящимся на пересечении строки и столбца, отмеченных одним и тем же терминальным символом, присвоим значение ВЫБРОС.

Шаг 3. Элементу таблицы $M(\perp, \epsilon)$ присвоим значение ДОПУСК.

Шаг 4. Остальным элементам таблицы присвоим значение ОШИБКА и представим результат в виде таблицы.

Начальное содержимое магазина - $S\perp$

В) Рассмотрим работу алгоритма для цепочки $i+i*i$.

Шаг 1. Алгоритм находится в начальной конфигурации $(i+i*i, S\perp, \varepsilon)$. Значение управляющей таблицы $M(S, i) = TE', 1$, при этом выполняются следующие действия:

- заменить верхний символ магазина S цепочкой TE' ;
- не сдвигать читающую головку;
- на выходную ленту поместить номер использованного правила 1.

Шаг 2. Выполняя действия, аналогичные описанным для шага 1, получим следующие конфигурации:

Текущая конфигурация	Значение M
$(i+i*i, TE'\perp, 1) \mid$	$M(T, i) = PT', 4$
$(i+i*i, PT'E'\perp, 14) \mid$	$M(P, i) = i, 8$
$(i+i*i, iT'E'\perp, 148) \mid$	$M(i, i) = ВЫБРОС$

Шаг 3. Алгоритм находится в конфигурации $(i+i*i, iT'E'\perp, 148)$. $M(i, i) = ВЫБРОС$. Выполняем следующие действия:

- удаляем верхний символ магазина;
- сдвигаем читающую головку на один символ вправо;
- при этом выходная лента не изменяется.

Алгоритм переходит в конфигурацию $(+i*i, T'E'\perp, 148)$.

Выполняя шаги 1 и 2, алгоритм выполняет следующие действия:

Текущая конфигурация	Значение M
$(+i*i, T'E'\perp, 148) \mid$	$M(T', +) = \varepsilon, 6$
$(+i*i, E'\perp, 1486) \mid$	$M(E', +) = + TE', 2$
$(+i*i, +TE'\perp, 14862) \mid$	$M(+, +) = ВЫБРОС$
$(i*i, TE'\perp, 14862) \mid$	$M(T, i) = PT', 4$
$(i*i, PT'E'\perp, 148624) \mid$	$M(P, i) = i, 8$
$(i*i, iT'E'\perp, 1486248) \mid$	$M(i, i) = ВЫБРОС$
$(*i, T'E'\perp, 1486248) \mid$	$M(T', *) = *PT', 5$
$(*i, *PT'E'\perp, 14862485) \mid$	$M(*, *) = ВЫБРОС$
$(i, PT'E'\perp, 14862485) \mid$	$M(P, i) = (i, 8)$
$(i, iT'E'\perp, 148624858) \mid$	$M(i, i) = ВЫБРОС$
$(\varepsilon, T'E'\perp, 148624858) \mid$	$M(T', \varepsilon) = (\varepsilon, 6)$
$(\varepsilon, E'\perp, 1486248586) \mid$	$M(E', \varepsilon) = (\varepsilon, 3)$
$(\varepsilon, \perp, 14862485863) \mid$	

Шаг 5. Алгоритм находится в конфигурации $(\varepsilon, \perp, 14862485863)$

Так как значение $M(\perp, \varepsilon) = \text{ДОПУСК}$, то цепочка $i+i*i$ принадлежит языку и последовательность номеров правил 14862485863 на выходной ленте – это ее разбор.

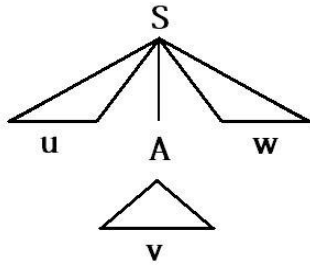
1.7. LR(k)-грамматики

Синтаксический LR-анализатор анализирует входную цепочку слева направо (L), и строит правый (R) вывод грамматики.

Грамматики, для которых можно построить детерминированный восходящий анализатор, называются LR(k)-грамматиками (входная цепочка читается слева (Left) направо, выходом анализатора является правый (Right) разбор, k-число

символов входной цепочки, на которое можно “заглянуть” вперед для выделения основы).

Определение 14.



Дерево вывода цепочки uvw.

Для определения LR(k)-грамматики используются:

1. Множество $FIRST_k(\gamma)$, состоящее из префиксов длины k терминальных цепочек, выводимых из γ .

Если из γ выводятся терминальные цепочки, длина которых меньше k, то эти цепочки также включаются в множество $FIRST_k(\gamma)$. Формально:

$$FIRST_k(\gamma) = \{x \mid \gamma \Rightarrow_i^* xw \text{ и } |x| = k \text{ или } \gamma \Rightarrow_i^* x \text{ и } |x| < k\}$$

2. Пополненной грамматикой, полученной из КС-грамматики $G = (T, V, P, S)$, называется грамматика $G' = (V \in \{S'\}, T, P = \{S' \rightarrow S\}, S')$. Если правила грамматики G' пронумерованы числами 1, 2, ..., p, то, будем считать, что $S' \rightarrow S$ – нулевое правило грамматики G' , а нумерация остальных правил такая же, как в грамматике G .

Утверждение. Если правило грамматики $A \rightarrow \beta$ не зависит от w, то во множестве $FIRST_k(w)$ содержится информация достаточная для определения основы LR(k)-грамматики, и применения операции Свертка.

Определение 15. КС-грамматика $G = (T, V, P, S)$ называется LR(k)-грамматикой для $k \geq 0$, если существуют два правых вывода для *пополненной* грамматики $G' = (T, V', P', S')$ полученной из G :

$$(A \rightarrow \beta \text{ (ab_B_c)}, B \rightarrow \sigma, \sigma x = \beta y$$

$$S' \Rightarrow_r^* \alpha A w \Rightarrow_r \alpha \beta w,$$

$$S' \Rightarrow_r^* \gamma B x \Rightarrow_r \gamma \sigma x \Rightarrow_r \alpha \beta y,$$

для которых $FIRST_k(w) = FIRST_k(y)$ следует, что $\alpha A y = \gamma B x$.

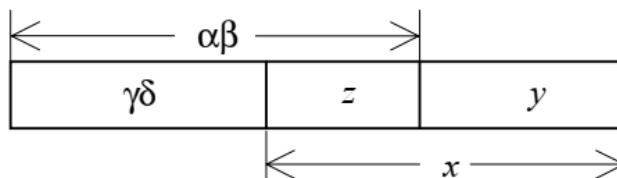


Рис. 3.1

Поскольку $\gamma \sigma x = \alpha \beta y$, то $|\gamma \sigma x| = |\alpha \beta y|$, и, учитывая, что $|\gamma \sigma| < |\alpha \beta|$, заключаем: $|x| > |y|$, т.е. $x = zy$ при некотором $z \in V^+_T$, $|z| > 0$. Заметим, что цепочка **z** является префиксом цепочки x, а не ее окончанием, так как именно цепочка y является окончанием всей сентенциональной формы $\alpha \beta y$. Два разных разбиения одной и той же сентенциональной формы $\gamma \sigma x = \alpha \beta y$ представлены на рис. 3.1.

Условие $\gamma\sigma x = \alpha\beta y$, можно переписать как $\gamma\sigma zy = \alpha\beta y$, и потому $\gamma\sigma z = \alpha\beta$.

Если $\alpha\beta w$ и $\alpha\beta y$ – правовыводимые цепочки пополненной грамматики G' , и $\text{FIRST}_k(w) = \text{FIRST}_k(y)$, $A \rightarrow \beta$ – последнее правило, использованное в правом выводе цепочки $\alpha\beta w$, то правило $A \rightarrow \beta$ должно использоваться также в правом разборе при свертке $\alpha\beta y$ к αAy .

1.7. 1. Верификация LR(k)-грамматики

Для того чтобы провести верификацию LR(k)-грамматики, необходимо определить алгоритмы построения множеств SFIRST_k и SEFF_k .

Определение 16. $\text{FIRST}_k(\alpha) = \{r \mid \exists \alpha \Rightarrow_i^* r\beta \text{ и } |r| = k \text{ или } \alpha \Rightarrow^* r \text{ и } |r| < k\}$ – построение множества префиксов длины k терминальных цепочек, выводимых из цепочки α .

Алгоритм. Построение множества SFIRST_k .

Вход: $G = (T, V, P, S_0)$ – контекстно-свободная грамматика, $k \geq 0$, $\alpha = X_1 X_2 \dots X_n \in (T \cup V)^*$.

Выход: $\text{SFIRST}_{k,\alpha}$ – множество префиксов длины k терминальных цепочек, выводимых из цепочки α .

Общий алгоритм:

Шаги 0a, 0b:

- Инициализация множеств F_k^0 для всех терминалов и нетерминалов.
- Добавление префиксов длины k по правилам грамматики.

Шаги 1, 2, ..., i:

- Добавление множеств F_{i-nk} .
- Повторение шагов до достижения стабильности множеств F_{ik} .

Шаг i + 1:

- Формирование множеств SFIRST_k для всех символов.

Шаг i + 2:

- Получение множества $\text{SFIRST}_k(\alpha)$ для заданной цепочки α .

$$F_{k,Y}^0 \Rightarrow F_{k,Y}^i \Rightarrow \text{SFIRST}_{k,Y} \Rightarrow \text{SFIRST}_{k,\alpha}$$

Рис. 3.2 - Алгоритм построения множества SFIRST_k

Пошаговый алгоритм:

0a. Для каждого терминала $t \in T$ положить $F_{k,t}^i = t$ для любого i .

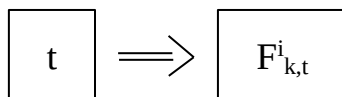


Рис. 3.3 - Шаг 0a

0b. Для каждого нетерминала $Y \in V$, рассмотреть все правила вида $Y \rightarrow x\beta$, где x – слово (может быть пустым), а цепочка β либо начинается с нетерминала, либо

пуста. Если $|x| \geq k$, добавим к множеству $F_{k,Y}^0$ слово $x[1, \dots, k]$. Иначе, если $\beta = \varepsilon$, добавим к множеству $F_{k,Y}^0$ слово x . $\in$

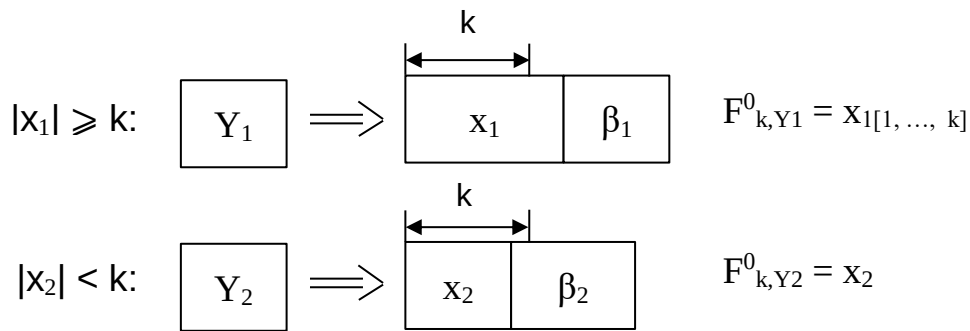


Рис. 3.3 - Шаг 0b

i. Добавить к множеству $F_{k,Y}^i$ множество $F_{k,Y}^{i-1}(Y)$. Для каждого правила $Y \rightarrow \gamma$, то есть $Y_1 \dots Y_n$, добавить к $F_{k,Y}^i$ множество $F_{k,Y_1}^{i-1} \oplus_k \dots \oplus_k F_{k,Y_n}^{i-1}$, далее вычислить F_{k,Y_j}^i для $j = 1 \dots n$.

Условие остановки: $F_{k,Y}^i = F_{k,Y}^{i-1}$ для любого $Y \in V$.

i+1. Положить $SFIRST_{k,Y} = F_{k,Y}^i$ для каждого $Y \in (V \cup T)$.

i+2. Множество $SFIRST_{k,\alpha} = SFIRST_{k,Y_1} \oplus_k \dots \oplus_k SFIRST_{k,Y_n}$.

Реализация алгоритма.

// шаг 0: формирование множеств $F_{k,Y}^0$

foreach ($Y \in (T \cup V)$)

if ($Y \in T$)

$F_{k,Y}^i = \{Y\}$

else

$F_{k,Y}^i = \emptyset$

foreach ($Y \rightarrow x\beta \ \&\& \ x = A_1 \dots A_m \in T^* \ \&\& \ b = B_1 \dots B_s \in (V \cup T)^* \ \&\& \ (B_1 \in V \parallel \beta ==$

$\varepsilon)$)

if ($|x| \geq k$)

$F_{k,Y}^i = F_{k,Y}^i \cup \{A_1 \dots A_k\}$

else if ($\beta == \varepsilon$)

$F_{k,Y}^i = F_{k,Y}^i \cup \{x\}$

end if

end foreach

end if

end foreach

i = 0

// шаг i: пока не выполняется $F_{k,Y}^i = F_{k,Y}^{i-1}$, добавляем к множеству $F_{k,Y}^i$ множество $F_{k,Y}^{i-1}$

do

i = i + 1

success = false

foreach ($Y \in (V \cup T)$)

$F_{k,Y}^i = F_{k,Y}^{i-1}$

if ($Y \in V$)

```

foreach ( $Y \rightarrow \gamma = Y_1 \dots Y_n$ )
   $F_{k,Y}^i = F_{k,Y}^i \cup (F_{k,Y_1}^{i-1} \oplus_k \dots \oplus_k F_{k,Y_n}^{i-1})$ 
end foreach
end if
end foreach
foreach ( $Y \in (V \cup T)$ )
  if ( $F_{k,Y}^i \neq \bar{F}_{k,Y}^{i-1}$ )
    success = true
    break
  end if
end foreach
while (success)

```

// шаг: формируем $SFIRST_{k,Y} = F_{k,Y}^i$

```

foreach ( $Y \in (V \cup T)$ )
   $SFIRST_{k,Y} = F_{k,Y}^i$ 
end foreach

```

// шаг: окончательно получаем $SFIRST_{k,a}$

$SFIRST_{k,a} = SFIRST_{k,X_1} \oplus_k \dots \oplus_k SFIRST_{k,X_n}$

Пример. Пусть $G = (\{+, -, /, *, a, b, (,)\}, \{S, R, T, E, F\}, \{S \rightarrow TR, R \rightarrow +TR \mid -TR \mid \varepsilon, T \rightarrow EF, F \rightarrow *EF \mid /EF \mid \varepsilon, E \rightarrow (S) \mid a \mid b\}, S)$ – контекстно-свободная грамматика. Вычислим $SFIRST_{1,Sab}$, $SFIRST_{1,RFa}$:

Построим множества $SFIRST_{k,X}$ для каждого $X \in V$:

Начало $i = 0$:	$i = 1$:	$i = 2$:
$F_{1,S}^0 = \{\}$	$F_{1,S}^1 = \{\}$	$F_{1,S}^2 = \{(, a, b\}$
$F_{1,R}^0 = \{+, -, \varepsilon\}$	$F_{1,R}^1 = \{+, -, \varepsilon\}$	$F_{1,R}^2 = \{+, -, \varepsilon\}$
$F_{1,T}^0 = \{\}$	$F_{1,T}^1 = \{(, a, b\}$	$F_{1,T}^2 = \{(, a, b\}$
$F_{1,F}^0 = \{*, /, \varepsilon\}$	$F_{1,F}^1 = \{*, /, \varepsilon\}$	$F_{1,F}^2 = \{*, /, \varepsilon\}$
$F_{1,E}^0 = \{(, a, b\}$	$F_{1,E}^1 = \{(, a, b\}$	$F_{1,E}^2 = \{(, a, b\}$

Поскольку на последнем шаге новых нетерминалов ни в одно множество не добавилось, последующие изменения невозможны. В итоге получаем следующие множества:

$SFIRST_{1,S} = \{(, a, b\}$; $SFIRST_{1,R} = \{+, -, \varepsilon\}$; $SFIRST_{1,T} = \{(, a, b\}$; $SFIRST_{1,F} = \{*, /, \varepsilon\}$; $SFIRST_{1,E} = \{(, a, b\}$.

Создаем искомые множества:

$SFIRST_{1,Sab} = SFIRST_{1,S} \oplus_1 FIRST_{1,a} \oplus_1 FIRST_{1,b} = \{(, a, b\} \oplus_1 \{a\} \oplus_1 \{b\} = \{(, a, b\}$;

$SFIRST_{1,Rfa} = SFIRST_{1,R} \oplus_1 SFIRST_{1,F} \oplus_1 SFIRST_{1,a} = \{+, -, \varepsilon\} \oplus_1 \{*, /, \varepsilon\} \oplus_1 \{a\} = \{+, -, *, /, a\}$.

Определение 17. Пусть $G = (T, V, P, S_0)$ – контекстно-свободная грамматика и $\alpha \in (T \cup V)^*$. Положим $EFF_k(\alpha) = FIRST_k(\alpha)$, если α начинается на терминал, а

иначе $\{w \in T^{*k} \mid \exists \alpha \Rightarrow^* \beta \Rightarrow wx, \text{ где } \beta \neq Awx, A \in V, |w| = k \text{ или } |w| < k \text{ и } x = \varepsilon\}$.
EFF расшифровывается как ε -free FIRST.

Алгоритм. Построение множества $SEFF_k$.

Вход: $G = (T, V, P, S_0)$ – контекстно-свободная грамматика, $k \geq 0$, $\alpha = X_1X_2\dots X_n \in (T \cup V)^*$.

Выход: множество активных префиксов длины k цепочек, при правостороннем выводе которых последнее примененное правило не является ε -правилом.

Пошаговый алгоритм:

1. Инициализировать множество $SEFF_k$ как пустое множество.
2. Если X_1 – терминал, то $SEFF_k = FIRST_k(\alpha)$ и алгоритм закончен, иначе перейти к третьему шагу.
3. Для каждой цепочки терминальных символов, выводимой из заданной цепочки, добавить в множество префикс $SEFF_k$ длины k или меньше k (если длина выведенной цепочки меньше k), если последнее примененное правило при правостороннем выводе не является ε -правилом.

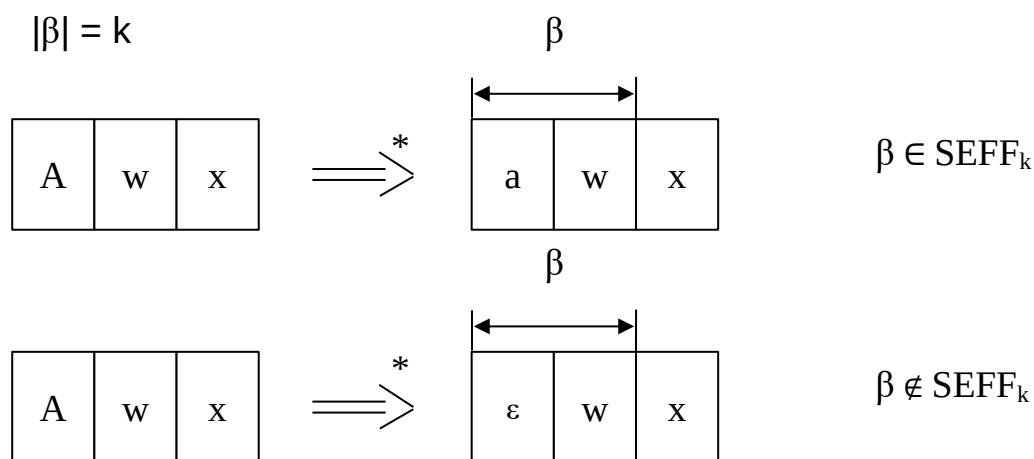


Рис. 3.4 - Алгоритм построения множества $SEFF_k$

// шаг 1: инициализируем $SEFF_k$ как пустое множество
 $SEFF_k = \emptyset$

foreach ($\{X_1, X_2, \dots, X_n\} \in (V \cup T)$)

// шаг 2: если первый символ цепочки - терминал, то $SEFF_k = FIRST_k(\alpha)$

if ($X_1 \in T$)

$SEFF_k = FIRST_k(\alpha)$

// шаг 3: добавляем в множество префиксы длины k , при получении которых последнее правило не было ε -правилом

else

foreach($\alpha \Rightarrow^* \beta \Rightarrow wx \ \&\& \ \beta \neq Awx \ \&\& \ A \in V \ \&\& \ (|w| = k \ \parallel \ (|w| < k \ \&\& \ x = \varepsilon))$
&& $w \in T^{*k}$)

$SEFF_k = SEFF_k \cup \{w\}$

end foreach

end if
end foreach

Пример. Пусть $G = (\{a, b, c\}, \{S, A, B, C\}, \{S \rightarrow AB, A \rightarrow Ba \mid \varepsilon, B \rightarrow Cb \mid C, C \rightarrow c \mid \varepsilon\}, S)$ – контекстно-свободная грамматика. Вычислим $SEFF_2$ для S :

$S \Rightarrow AB = Awx$:

1. $S \Rightarrow AB \Rightarrow ACx \Rightarrow$

$w=B, w=C$

a. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow$

$w=B, w=C, w=c$

i. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Acx \Rightarrow Bax$

$w=B, w=C, w=c, w=a$

1. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Acx \Rightarrow Bax \Rightarrow Cbx$

$w=B, w=C, w=c, w=a, w=b$

a. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Acx \Rightarrow Bax \Rightarrow Cbx \Rightarrow cbx$

$\in SEFF_k$

$w=B, w=C, w=c, w=a, w=b, w=b$

b. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Acx \Rightarrow Bax \Rightarrow Cbx \Rightarrow$

$\varepsilon bx \notin SEFF_k$

$w=B, w=C, w=c, w=a, w=b, w=b$

2. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Acx \Rightarrow Bax \Rightarrow Cax$

$w=B, w=C, w=c, w=a, w=a$

a. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Acx \Rightarrow Bax \Rightarrow Cax \Rightarrow cax$

$\in SEFF_k$

$w=B, w=C, w=c, w=a, w=a, w=a$

b. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Acx \Rightarrow Bax \Rightarrow Cax \Rightarrow \varepsilon ax \notin$

$SEFF_k$

$w=B, w=C, w=c, w=a, w=a, w=a$

ii. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Acx \Rightarrow \varepsilon cx \notin SEFF_k$

$w=B, w=C, w=c, w=a$

b. $S \Rightarrow AB \Rightarrow ACb \Rightarrow Ab \Rightarrow$

$w=B, w=C, w=b$

i. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow Bax$

$w=B, w=C, w=c, w=a$

1. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow Bax \Rightarrow Cbx$

$w=B, w=C, w=c, w=a, w=b$

a. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow Bax \Rightarrow Cbx \Rightarrow cbx \in$

$SEFF_k$

$w=B, w=C, w=c, w=a, w=b, w=b$

b. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow Bax \Rightarrow Cbx \Rightarrow \varepsilon bx \notin$

$SEFF_k$

$w=B, w=C, w=c, w=a, w=b, w=b$

2. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow Bax \Rightarrow Cax$

$w=B, w=C, w=c, w=a, w=a$

a. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow Bax \Rightarrow Cax \Rightarrow cax \in$

$SEFF_k$

$w=B, w=C, w=c, w=a, w=a, w=a$

- b. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow Bax \Rightarrow Cax \Rightarrow \varepsilon ax \notin SEFF_k$
 $w=B, w=C, w=c, w=a, w=a, w=a$
- ii. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Ab \Rightarrow \varepsilon b \notin SEFF_k$
 $w=b, w=c, w=b, w=b$
2. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Acx \Rightarrow \varepsilon cx \notin SEFF_k$
3. $S \Rightarrow AB \Rightarrow ACx \Rightarrow Ab \Rightarrow \varepsilon b \notin SEFF_k$
4. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac$
- a. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac \Rightarrow Bax$
- i. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac \Rightarrow Bax \Rightarrow Cbx$
1. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac \Rightarrow Bax \Rightarrow Cbx \Rightarrow cbx \in SEFF_k$
2. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac \Rightarrow Bax \Rightarrow Cbx \Rightarrow \varepsilon bx \notin SEFF_k$
- ii. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac \Rightarrow Bax \Rightarrow Cax$
1. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac \Rightarrow Bax \Rightarrow Cax \Rightarrow cax \in SEFF_k$
2. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac \Rightarrow Bax \Rightarrow Cax \Rightarrow \varepsilon ax \notin SEFF_k$
- b. $S \Rightarrow AB \Rightarrow AC \Rightarrow Ac \Rightarrow \varepsilon \notin SEFF_k$
5. $S \Rightarrow AB \Rightarrow AC \Rightarrow A$
- a. $S \Rightarrow AB \Rightarrow AC \Rightarrow A \Rightarrow Ba$
- i. $S \Rightarrow AB \Rightarrow AC \Rightarrow A \Rightarrow Ba \Rightarrow Cbx$
1. $S \Rightarrow AB \Rightarrow AC \Rightarrow A \Rightarrow Ba \Rightarrow Cbx \Rightarrow cbx \in SEFF_k$
2. $S \Rightarrow AB \Rightarrow AC \Rightarrow A \Rightarrow Ba \Rightarrow Cbx \Rightarrow \varepsilon bx \notin SEFF_k$
- ii. $S \Rightarrow AB \Rightarrow AC \Rightarrow A \Rightarrow Ba \Rightarrow Ca$
1. $S \Rightarrow AB \Rightarrow AC \Rightarrow A \Rightarrow Ba \Rightarrow Ca \Rightarrow ca \in SEFF_k$
2. $S \Rightarrow AB \Rightarrow AC \Rightarrow A \Rightarrow Ba \Rightarrow Ca \Rightarrow \varepsilon a \notin SEFF_k$
- b. $S \Rightarrow AB \Rightarrow AC \Rightarrow A \Rightarrow \varepsilon \notin SEFF_k$

Множество SEFF: $\{cbx, cax, ca\}$.

Определение 18. Пусть $G = (T, V, P, S_0)$ – контекстно-свободная грамматика и $S_0 \Rightarrow^* \alpha A w \Rightarrow \alpha \beta w$ – некоторый правосторонний вывод в грамматике G , где $\alpha, \beta \in (T \cup V)^*$, $w \in T^*$. Активным префиксом сентенциальной формы $\alpha b w$ называется любая начальная часть (префикс) цепочки $\alpha \beta$, включая, в частности, ε и всю эту цепочку.

$Rp(\alpha) = \{r \in (T \cup V)^* \mid \exists \alpha \Rightarrow_i^* r\beta, \text{ где } \beta \in (T \cup V)^*\}$.

Определение 19. Пусть $G = (T, V, P, S_0)$ – контекстно-свободная грамматика и $S \Rightarrow^* \alpha A w \Rightarrow \alpha \beta w$ – правосторонний вывод в грамматике G , где $\beta = \beta_1 \beta_2$, $\beta_1 \beta_2 \in (T \cup V)^*$. Назовем $[A \rightarrow \beta_1 \cdot \beta_2, u]$, где $u = \text{FIRST}_k(w)$, LR(k)-ситуацией, допустимой для активного префикса $\alpha \beta_1$.

Теорема. Чтобы контекстно-свободная грамматика $G = (T, V, P, S_0)$ была LR(k)-грамматикой, необходимо и достаточно, чтобы выполнялось следующее условие: если $[A \rightarrow \beta \cdot, u]$ – LR(k)-ситуация, допустимая для активного префикса $\alpha \beta$ правосентенциальной формы $\alpha b w$ пополненной грамматики G' , где $u = \text{FIRST}_k(w)$, то не существует никакой другой LR(k)-ситуации $[B \rightarrow \beta_1 \cdot \beta_2, v]$, которая отличается от данной и допустима для активного префикса $\alpha \beta$ при условии, что $u \in \text{EFF}_k(\beta_2 v)$ (в частности, и при $\beta_2 = \varepsilon$).

Определение 20. Множество $I \in \Phi$ назовем непротиворечивым, если оно не содержит двух разных LR(k)-ситуаций вида $[A \rightarrow \beta \cdot, u]$ и $[B \rightarrow \beta_1 \cdot \beta_2, v]$ (цепочка β_2 может быть пустой), таких, что $u \in \text{EFF}_k(\beta_2 v)$.

Алгоритм. LR(k)-верификация:

Вход: $G = (T, V, P, S_0)$ – контекстно-свободная грамматика, $k \geq 0$.

Выход: «Да», если G – LR(k)-грамматика. «Нет» в противном случае.

1. Построить каноническую систему Φ множеств LR(k)-ситуаций, допустимых для грамматики G .
2. Каждое $I \in \Phi$ проверить на непротиворечивость. Если оказывается, что рассматриваемое множество I противоречиво, то закончить алгоритм с ответом «Нет».
3. Если алгоритм не закончился на втором шаге, то завершить алгоритм с ответом «Да».

```
foreach (I ∈ Φ)
    answer = true
    if ([A → β ·, u] ∈ I && [B → β1 · β2, v] ∈ I && u ∈ EFFk(β2 v))
        answer = false
        break
    end if
end foreach
```

Пример. $G = (\{a, b\}, \{S', S\}, \{S' \rightarrow S, S \rightarrow SaSb, S \rightarrow \varepsilon\}, S')$, $k = 1$.

$\Phi = \{$
 $I_0 = \{[S' \rightarrow \cdot S, \varepsilon], [S \rightarrow \cdot SaSb, \varepsilon \mid a], [S \rightarrow \cdot, \varepsilon \mid a]\};$
 $I_1 = \{[S' \rightarrow S \cdot, \varepsilon], [S \rightarrow S \cdot aSb, \varepsilon \mid a]\};$
 $I_2 = \{[S \rightarrow Sa \cdot Sb, \varepsilon \mid a], [S \rightarrow \cdot SaSb, a \mid b], [S \rightarrow \cdot, a \mid b]\};$
 $I_3 = \{[S \rightarrow SaS \cdot b, \varepsilon \mid a], [S \rightarrow S \cdot aSb, a \mid b]\};$
 $I_4 = \{[S \rightarrow Sa \cdot Sb, a \mid b], [S \rightarrow \cdot SaSb, a \mid b], [S \rightarrow \cdot, a \mid b]\};$
 $I_5 = \{[S \rightarrow SaSb \cdot, \varepsilon \mid a]\};$
 $I_6 = \{[S \rightarrow SaS \cdot b, a \mid b], [S \rightarrow S \cdot aSb, a \mid b]\};$
 $I_7 = \{[S \rightarrow SaSb \cdot, a \mid b]\}$
 $\}.$

Достаточно проверить лишь непротиворечивость множеств $I_0, I_1, I_2, I_4, I_5, I_7$.

1) Проверка I_0 : $u \in \{\varepsilon, a\}$, $u \notin \text{EFF}_1(S\{\varepsilon\}) = \emptyset$; $u \notin \text{EFF}_1(SaSb\{\varepsilon, a\}) = \emptyset$.

Вывод: множество I_0 непротиворечиво.

2) Проверка I_1 : $u \in \{\varepsilon\}$, $u \notin \text{EFF}_1(aSb\{\varepsilon, a\}) = \{a\}$.

Вывод: множество I_1 непротиворечиво.

3) Проверка I_2 : $u \in \{a, b\}$, $u \notin \text{EFF}_1(Sb\{\varepsilon, a\}) = \emptyset$; $u \notin \text{EFF}_1(SaSb\{a, b\}) = \emptyset$.

Вывод: множество I_2 непротиворечиво.

4) Проверка I_4 : $u \in \{a, b\}$, $u \notin \text{EFF}_1(Sb\{a, b\}) = \text{EFF}_1(SaSb\{a, b\}) = \emptyset$.

Вывод: множество I_4 непротиворечиво.

5) Проверка $I_5 = \{[S \rightarrow SaSb \cdot, \varepsilon \mid a]\}$ - множество непротиворечиво.

6) Проверка $I_7 = \{[S \rightarrow SaSb \cdot, a \mid b]\}$ - множество непротиворечиво.

Поскольку противоречивых множеств не найдено, то рассматриваемая грамматика – LR(1)-грамматика.

Пример. $G = (\{a\}, \{S', S\}, \{S' \rightarrow S, S \rightarrow aS, S \rightarrow \varepsilon\}, S')$, $k = 1$.

Строим каноническую систему множеств ϕ LR(1)-ситуаций, сразу проверяя множества на непротиворечивость:

- 1) $I_0 = \{[S' \rightarrow \cdot S, \varepsilon], [S \rightarrow \cdot aS, \varepsilon], [S \rightarrow \cdot, \varepsilon]\}$. Проверка I_0 : $u \in \{\varepsilon\}$, $u \notin \text{EFF}_1(S\{\varepsilon\}) = \emptyset$; $u \notin \text{EFF}_1(aS\{\varepsilon\}) = \{a\}$.

Вывод: множество I_0 непротиворечиво.

- 2) $I_1 = \{[S' \rightarrow S \cdot, \varepsilon]\}$ - множество непротиворечиво.

- 3) $I_2 = \{[S \rightarrow a \cdot S, \varepsilon], [S \rightarrow \cdot aS, \varepsilon \mid a]\}$ - множество непротиворечиво.

- 4) $I_3 = \{[S \rightarrow a \cdot S, \varepsilon \mid a], [S \rightarrow \cdot aS, \varepsilon \mid a], [S \rightarrow \cdot, \varepsilon \mid a]\}$. Проверка I_3 : $u \in \{\varepsilon \mid a\}$, $u \notin \text{EFF}_1(S\{\varepsilon \mid a\}) = \emptyset$; $u \in \text{EFF}_1(aS\{\varepsilon \mid a\}) = \{a\}$.

Вывод: множество I_3 противоречиво.

Поскольку противоречивое множество найдено, то рассматриваемая грамматика – не LR(1)-грамматика.

1.7. 2. Алгоритм "Перенос-Свертка"

При LR(k)-анализе применяется алгоритм перенос-свертка (англ. shift-reduce). Суть метода сводится к следующему:

1. Входная цепочка символов считывается до тех пор, пока не накопится в магазине цепочка, совпадающая с правой частью какого-нибудь из правил P . Операция Перенос переносит символ в магазин.

2. Далее все символы совпадающей цепочки извлекаются из стека и на их место помещается нетерминал, находящийся в левой части этого правила (операция Свертка).

Алгоритм "перенос-свертка" λ описывается в терминах конфигураций вида $(\perp X_1 \dots X_m, a_1 \dots a_n, p_1 \dots p_r)$, рис. 1.18. где :

$\perp X_1 \dots X_m$ – содержимое магазина, X_m – верхний символ магазина и $X_i \in V \cup T$, символ $(\perp)$ дна магазина;

$a_1 \dots a_n$ – оставшаяся непрочитанная часть входной цепочки, a_1 – текущий входной символ;

$p_1 \dots p_r$ – разбор правил, полученный к данному моменту времени на ленте π .

Входная лента

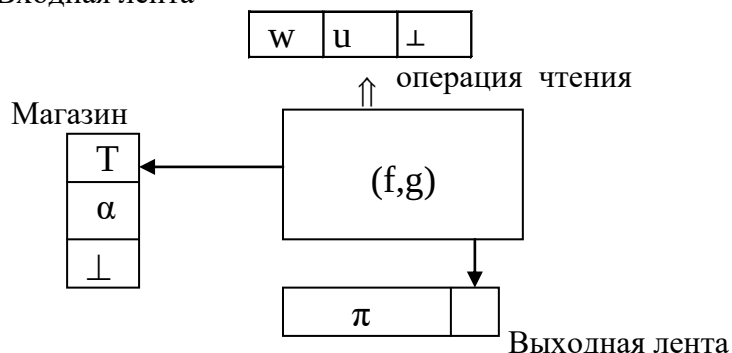


Рис. 1.18. LR(k)-анализатор

Один шаг алгоритма λ можно описать с помощью двух отношений: $\vdash^S$ (shift, Перенос) и $\vdash^r$ (reduce, Свертка), определенных на конфигурациях следующим образом:

1. Если $f(\alpha, aw) = \text{ПЕРЕНОС} (\Pi)$, то входной символ переносится в верхушку магазина и читающая головка сдвигается на один символ вправо. В терминах конфигураций этот процесс описывается так:

$(\alpha, aw, \pi) \vdash^s (\alpha a, w, \pi)$ для $\alpha \in (V \cup T \cup \{\perp\})^*$, $w \in (T \cup \{\varepsilon\})^*$ и $\pi \in \{1, \dots, p\}^*$.

2. Если $f(\alpha\beta, w) = \text{СВЕРТКА} (C)$, $g(\alpha\beta, w) = i$ и $A \rightarrow \beta$ – правило грамматики с номером i , то цепочка β заменяется правой частью правила с номером i , а его номер помещается на выходную ленту, т.е. $(\alpha, aw, \pi) \vdash^s (\alpha A, w, \pi i)$.

3. Если $f(\alpha, w) = \text{ДОПУСК} (D)$, то $(\alpha, w, \pi) \vdash \text{ДОПУСК}$.

В остальных случаях $(\alpha, w, \pi) \vdash \text{ОШИБКА}$ (пустое значение в таблице).

Отношение $\vdash$ определяется как объединение отношений $\vdash^s$ и $\vdash^r$, а транзитивное замыкание отношений $\vdash^+$ и $\vdash^*$ определяется как обычно.

Для практического применения алгоритм мало пригоден, так как для определения функции переноса он требует анализа всей цепочки в магазине и всей необработанной части входной цепочки. Аналогичный недостаток имеет алгоритм и при определении функции свертки.

Для $w \in T^*$ будем записывать $\lambda(w) = \pi$, если $(\perp, w, \varepsilon) \vdash^* (\perp S, \varepsilon, \pi) \vdash \text{ДОПУСК}$, или $\lambda(w) = \text{ОШИБКА}$, в противном случае.

Пример. Применить алгоритм типа “перенос-свертка” для заданной грамматики $G = (T, V, P, S)$, где $P = \{p_1, p_2, p_3, p_4\}$, $p_1: S \rightarrow bAb$, $p_2: A \rightarrow cB$, $p_3: A \rightarrow a$, $p_4: B \rightarrow Aad$

$S \Rightarrow^1 bAb \Rightarrow^2 bcBb \Rightarrow^4 bcAadb \Rightarrow^3 bcaadb$

Разберем с помощью алгоритма λ входную цепочку $bcaadb$.

Начальная конфигурация алгоритма $(\perp, bcaadb, \varepsilon)$.

Выполнение алгоритма определяется значением $f(\perp, bcaadb)$, которое, как видно из определения функции f , имеет значение ПЕРЕНОС. Алгоритм переходит в конфигурацию $(\perp b, caadb, \varepsilon)$, выполняя шаг:

$(\perp, bcaadb, \varepsilon) \vdash^s (\perp b, caadb, \varepsilon)$ и следующие шаги:

$(\perp b, caadb, \varepsilon) \vdash^s (\perp bc, aadb, \varepsilon) \vdash^s (\perp bca, adb, \varepsilon)$

Выполнение алгоритма λ

$\perp$	$bcaadb$	$\vdash^s$
$\perp b$	$caadb$	$\vdash^s$
$\perp bc$	$aadb$	$\vdash^s$
$\perp bca$	adb	$\vdash^r$ по p_3
$\perp bcA$	adb	$\vdash^s$
$\perp bcAa$	db	$\vdash^s$
$\perp bcAad$	b	$\vdash^r$ по p_4
$\perp bcB$	b	$\vdash^r$ по p_2
$\perp bA$	b	$\vdash^s$
$\perp bAb$	ε	$\vdash^r$ по p_1
$\perp S$	ε	ДОПУСК

Очередной шаг выполнения алгоритма определяется значением $f(\perp bca, adb, \varepsilon)$, которое имеет значение СВЕРТКА. Правило, используемое при свертке, определяется значением функции $f(\perp bca, adb) = 3$ и переходим в следующую конфигурацию: $(\perp bca, adb, \varepsilon) \vdash^s (\perp bcA, adb, 3)$, следующие шаги: $(\perp bcA, adb, 3)$

$\vdash^r(\perp bcAa, db, 3) \vdash^r(\perp bcAad, b, 3) \vdash^s(\perp bcB, b, 34) \vdash^s(\perp bA, b, 342) \vdash^r(\perp bAb, \varepsilon, 342) \vdash^r(\perp S, \varepsilon, 3421) \vdash^s \text{ДОПУСК}.$

Таким образом, $\lambda(bcaadb) = 3421$, правый вывод цепочки $bcaadb$ должен быть равен 1243. Для проверки построим цепочку по заданному правому выводу:

$S \Rightarrow^1 bAb \Rightarrow^2 bcBb \Rightarrow^4 bcAadb \Rightarrow^3 bcaadb$

Определение. *Основой называется кодируемая цепочка символов в верхней части магазина.*

Для любой LR(k)-грамматики $G = (T, V, P, S)$ можно построить детерминированный анализатор, с правым разбором входной цепочки. Анализатор состоит из магазина, входной ленты, выходной ленты и управляющего устройства (пара функций f и g) см. рис. 1.18.

Определение. *Грамматическое вхождение* – это символы полного словаря грамматики, снабженные двумя индексами. Первый индекс i задает номер правила грамматики, в правую часть которого входит данный символ, а второй индекс j – номер позиции символа в этой правой части.

Пример. В приведённой грамматике с правилами:

$p_1: S \rightarrow F \& L,$

$p_2: S \rightarrow (S),$

$p_3: F \rightarrow * L,$

$p_4: F \rightarrow i,$

$p_5: L \rightarrow F$

грамматические вхождения $\&_{1,2},)_{2,3}, L_{3,2}$ и т.д.

Каждый символ магазинного алфавита V_p (грамматическое вхождение символа грамматики в правую часть правила вывода) можно интерпретировать как закодированное представление подцепочки из правой части правила, последним символом которой он является. При такой интерпретации магазинный символ вталкивается только тогда, когда символ из верхушки магазина является *кодированным представлением префикса подцепочки, представляемой вталкиваемым в магазин символом*, или когда вталкиваемый символ представляет *новый префикс правой части какого-либо правила*. Перенос символов в магазин осуществляется до тех пор, пока в верхушке магазина не окажется символ, являющийся закодированным представлением основы. В этом случае цепочка магазинных символов, соответствующих основе, выталкивается из магазина, и в магазин вталкивается символ, который исполняет роль подцепочки, совместимой с подцепочкой, представляемой магазинным символом, расположенным под основой.

Магазинный алфавит V_p представляет собой множество специальных символов, соответствующих грамматическим вхождениям или их множествам.

Два способа построения анализаторов для LR(k)-грамматик без ε -правил на основе активных префиксов

Два способа построения анализаторов для LR(k)-грамматики без ε -правил на основе активных префиксов:

I. На основе кодирования активных префиксов и отношения OBLOW (be low входит под) для LR(0) и SLR(1)-грамматика без ϵ -правил. SLR(1)-грамматики (где S - simple) не принадлежат классу LR(0) грамматик.

Алгоритм построения LR(0) анализатора:

Шаг 1. Пронумеровать правила productions. Кодировать активные префиксы.

Шаг В. При кодировании магазинному символу соответствует несколько вхождений, то это SLR(1)-грамматика и таблица действий $f(a)$ задается алгоритмом.

Шаг 2. Построение управляющей таблицы $M = [f(u), g(x)]$;

1. Построить пополненную грамматику G' для исходной грамматики G .

2. Построить отношение OBLOW (be low входит под) грамматических вхождений G .

3. Построить КА $g(x)$ на основе отношения OBLOW. Недетерминированный КА преобразовать в детерминированный. Выполнить Шаг В.

Шаг 4. Применить алгоритм перенос-свёртка.

II. Построение SL(0) анализатора на основе LR(0)-ситуаций, функций замыкания CLOSURE и перехода GOTO.

Шаг 1. Построение управляющей таблицы $M = [f(u), g(x)]$;

1.Пронумеровать правила productions. Построить пополненную грамматику G' для исходной грамматики G .

2.Построить множества ситуаций и построение КА для множества ситуаций каждое множество ситуаций - состояние КА;

3.Построить отношение действий $f(u)$ на основе КА.

Шаг 3. Применить алгоритм перенос-свёртка.

I. Построение анализаторов для LR(k)-грамматики без ϵ -правил на основе активных префиксов и отношения OBLOW

Пусть грамматические вхождения символов X и Z в правую часть i -го правила, а Y_j - грамматическое вхождение символа Y в правую часть j -го правила. Определим множество OFIRST(Y_i) (входит первым), в которое включим Y_j и все грамматические вхождения, с которых могут начинаться цепочки, выводимые и Y :

$$\text{OFIRST}(Y_i) = \{Y_j\} \cup \{X_i \mid Y \Rightarrow^* A\beta \Rightarrow X\alpha\beta \text{ и } X_i \text{ самое левое грамматическое вхождение в правую часть правила } A \rightarrow X\}$$

Если в грамматике есть правила с пустой правой частью, то на последующем шаге вывода они не применяются.

Алгоритм нахождения множества OFIRST(Y_{ij}):

Вход: $G' = (T, V', P', S')$, Y_{ij} – грамматическое вхождение.

Выход: VT – множество грамматических вхождений, находящихся первыми после Y_{ij} .

$VT := \{Y_{ij}\}$

foreach($A_k \rightarrow X_{k1}\alpha \in P' \mid A_k = \text{symbol}(Y_{ij}) \in V'$) // $\text{symbol}(Y_{ij}) = Y$

$VT := VT \cup \text{OFIRST}(X_{k1})$

end

Определим отношение OBLOW (входит под) используя множество OFIRST, следующим образом:

$X_i \text{ OBLOW } Y_j$ - это множество $\{(X_i, Y_j) \mid A \rightarrow \alpha X_i Z_j \beta \in P \text{ и } Y_j \in \text{OFIRST}(Z_i)\}$

$\perp \text{ OBLOW } Y_j$ - это множество $\{(\perp, Y_j) \mid Y_j \in \text{OFIRST}(S_0)\}$, где S_0 - начальное вхождение.

Отношение $X_i \text{ OBLOW } Y_j$ определяет множество Q грамматических вхождений X_i , для которых представляющие их магазинные символы могут встретиться в магазине непосредственно под символом, представляющим Y_j .

Отношение OBLOW будем представлять в виде матрицы, содержащей n столбцов и $(n+1)$ рядов, где n - число грамматических вхождений пополненной грамматики G . Первые n рядов матрицы отмечены грамматическими вхождениями, а последний ряд - маркером дна стека. Если $X_i \text{ OBLOW } Y_j$, то элемент матрицы, расположенный в ряде X_i и столбце Y_j равен 1.

Алгоритм построения матрицы отношения OBLOW:

Вход: $G' = (T, V', P', S')$.

Выход: Γ – множество всех грамматических вхождений (и маркер дна), OBLOW – матрица размера $(n+1) \times n$, где $n = \sum_{i=1}^{|P'|} (|\alpha_i|, A_i \rightarrow \alpha_i \in P')$.

$\Gamma \left\{ \perp \right\}$

foreach ($A_i \rightarrow \{X_{i1}, X_{i2}, \dots, X_{ik}\} \in P'$)

foreach ($X_{ij} \in \{X_{i1}, X_{i2}, \dots, X_{ik}\}$)

$\Gamma := \Gamma \cup \{X_{ij}\}$

foreach ($Y \in \{\text{OFIRST}(X_{ij}) \mid A_i = S'\}$) // см. определение выше
 $\text{OBLOW}[\perp, Y] := 1$

end

foreach ($Y \in \{\text{OFIRST}(X_{i(j+1)}) \mid j \neq k\}$) // см. определение выше
 $\text{OBLOW}[X_{ij}, Y] := 1$

end

end

end

Пример. Построим матрицу отношения OBLOW для грамматики с правилами:

$p_1: S \rightarrow aAb$ $p_2: S \rightarrow c$ $p_3: A \rightarrow bS$

$p_4: A \rightarrow Bb$ $p_5: B \rightarrow aA$ $p_6: B \rightarrow c$

Непосредственно из правил вывода грамматики p_1 и p_4 получим:

$A_1 \text{ OBLOW } b_1, B_4 \text{ OBLOW } b_4$

Из определения отношения OBLOW следует, что $\perp \text{ OBLOW } Y_j$ тогда и только тогда, когда $Y_j \in \text{OFIRST}(S_0)$. Из S можно вывести цепочки $S \Rightarrow a_1 A_1 b_1$ и $S \Rightarrow c_2$. Следовательно $\text{OFIRST}(S_0) = \{a_1, c_2, S_0\}$ и $\perp \text{ OBLOW } a_1, \perp \text{ OBLOW } c_2$ и $\perp \text{ OBLOW } S_0$.

Для правила p_1 : $a_1 \text{ OBLOW } Y_j$, для всех $Y_j \in \text{OFIRST}(A_1)$ из A можно вывести цепочки $A \Rightarrow b_3 S_3$ и $A \Rightarrow b_4 B_4 \Rightarrow b_6 B_4, A \Rightarrow b_4 B_4 \Rightarrow a_5 A_5 b_4$.

Следовательно, $\text{OFIRST}(A_1) = \{a_5, b_3, c_6, A_1, B_4\}$ и $a_1 \text{ OBLOW } a_5, a_1 \text{ OBLOW } b_3, a_1 \text{ OBLOW } c_6, a_1 \text{ OBLOW } A_1, a_1 \text{ OBLOW } B_4$

Вычислим OBLOW для правил p_3 и p_6 , получим матрицу отношения OBLOW.

· Из правила p_1 и p_4 : $A_1 \text{ OBLOW } b_1, B_4 \text{ OBLOW } b_4$

- Из определения отношения OBLOW следует, что $\perp$ OBLOW Y_j тогда и только тогда, когда $Y_j \in \text{OFIRST}(S_0)$. Из S можно вывести цепочки $S \Rightarrow a_1 A_1 b_1$ и $S \Rightarrow c_2$. Следовательно $\text{OFIRST}(S_0) = \{a_1, c_2, S_0\}$ и $\perp$ OBLOW a_1 , $\perp$ OBLOW c_2 , $\perp$ OBLOW S_0
- Для правила p_1 : a_1 OBLOW Y_j , для всех $Y_j \in \text{OFIRST}(A_1)$ из A можно вывести цепочки $A \Rightarrow b_3 S_3$ и $A \Rightarrow B_4 b_4 \Rightarrow c_6 B_4$, $A \Rightarrow b_4 B_4 \Rightarrow a_5 A_5 b_4$. Следовательно, $\text{OFIRST}(A_1) = \{a_5, b_3, c_6, A_1, B_4\}$ и a_1 OBLOW a_5 , a_1 OBLOW b_3 , a_1 OBLOW c_6 , a_1 OBLOW A_1 , a_1 OBLOW B_4
- Правило p_2 – пропуск
- Для правила p_3 : b_3 OBLOW Y_j , для всех $Y_j \in \text{OFIRST}(S_3)$, из S можно вывести цепочки $S \Rightarrow a_1 A_1 b_1$ и $S \Rightarrow c_2$. Следовательно, $\text{OFIRST}(S_3) = \{a_1, c_2, S_3\}$, и b_3 OBLOW a_1 , b_3 OBLOW c_2 , b_3 OBLOW S_3
- Правило p_4 - пропуск
- Для правила p_5 – аналогично для правила p_1 , a_1 OBLOW a_5 , a_1 OBLOW b_3 , a_1 OBLOW c_6 , a_1 OBLOW A_1 , a_1 OBLOW B_4
- Правило p_6 – пропуск

Составим итоговую матрицу отношения OBLOW

	S_0	a_1	A_1	b_1	c_2	b_3	S_3	B_4	b_4	a_5	A_5	c_6	$\perp$
S_0													1
a_1			1			1		1		1		1	1
A_1		1		1									
b_1			1										
c_2						1							1
b_3		1			1		1						
S_3						1							
B_4		1							1				
b_4								1					
a_5		1											
A_5													
c_6		1											

$\perp$ 1 1 1

Шаг 1. Пример кодирования активных префиксов. Для каждого магазинного символа (за исключением S_0 и $\perp$), кодируемая цепочка является *префиксом* правой части некоторого правила грамматики. Например, магазинный символ «&₁», находящийся на вершине магазина, означает что:

1. В верхушке магазина находится терминальный символ грамматики «&»
2. В верхней части магазина находится кодируемая этим символом цепочка символов «F&», то есть префикс *основы* – правой части правила 1.

Индекс каждого символа соответствует номеру правила, префикс правой части которого кодируется этим символом. И наоборот, каждый непустой префикс правой части правила кодируется некоторым магазинным символом. Например, правая часть правила 3 имеет два непустых префикса: «*» и «L», которые соответственно кодируются символами «*₃» и «L₃».

Символ переносится в магазин только в том случае, если он кодирует цепочку, «совместимую» с цепочкой, которая будет находиться в магазине после переноса.

Цепочка, кодируемая данным магазинным символом, *совместима* с цепочкой в магазине, если она является суффиксом магазинной цепочки после переноса данного символа.

$$P = \{ p_1: S \rightarrow F \& L, p_2: S \rightarrow (S), p_3: F \rightarrow * L, p_4: F \rightarrow i, p_5: L \rightarrow F \}$$

Таблица 1. Закодированные символы для заданной грамматики.

Символ грамматики	Магазинный Символ	Кодируемая цепочка	Операции
S	S_0	$\perp S$	Д
	S_2	(S	П
F	F_1	F	П
	F_5	F	C_5
L	L_1	F&L	C_1
	L_3	*L	C_3
i	i_4	i	C_4
(	$(_2$	(	П
*	$*_3$	*	П
&	$\&_1$	F&	П
)	$)_2$	(S)	C_2

Шаг В. Кодируемому магазинному символу соответствует единственное вхождение (детерминированный КА).

Определить функция действий $F(u)$ на множестве $(V_p \cup \{\perp\}) \times (T \cup \{\varepsilon\})$ по правилам:

1. Если $A \rightarrow \beta$ – правило грамматики с номером i , то для конфигурации $(\alpha T, a\alpha, \pi)$, где T кодирует цепочку β , $F(u) = C(i) = (\text{СВЕРТКА}, i)$.

Если магазинному символу $г$ соответствует только одно грамматическое вхождение X_i , являющееся самым правым вхождением в i -е правило вывода грамматики G , то все элементы ряда, помеченной $г$, имеют значение (СВЕРТКА, i).

2. Если $A \rightarrow \beta$ – правило грамматики с номером i , то для конфигурации $(\alpha T, ax, \pi)$, где $г$ кодирует некий префикс цепочки β (но не саму основу), $F(u) = \text{Перенос}$.
3. Для конфигурации $(S_0, \perp, \pi)$, где S_0 кодирует цепочку $\perp S$, $F(u) = \text{ДОПУСК}$.
4. В противном случае, $F(u) = \text{ОШИБКА}$.

Шаг 2. Алгоритм. Построение управляющей таблицы M для LR(0) грамматики. Таблица M состоит из матрицы действий $f(u)$ и переходов $KA\ g(X)$ и, которые строятся отдельно, а затем объединяются.

Вход: LR(0)-грамматика $G = (T, V, P, S)$ не содержащая ε - правил, таблица l закодированных символов.

Выход: Управляющая таблица M для грамматики G .

1. Построить пополненную грамматику G' для исходной грамматики G .
2. Вычислить отношение OBLOW грамматических вхождений грамматики G .
3. Построить отношение переходов $g(x)$.

Отношение переходов $g(x)$ удобно представлять в виде таблицы, содержащей по одному столбцу для каждого символа TUV и одной строке для каждого грамматического вхождения грамматики и маркера дна. Элемент в строке с помеченным грамматическим вхождением X_{ij} и столбце, отмеченном символом грамматики Y , должен содержать все грамматические вхождения, для которых справедливо отношение $X_{ij} \text{ OBLOW } Y_j$.

Аргументами функции переходов g являются символы $X \in (V \cup T)$ и цепочка $u \in (V_p \cup \{\perp\})^*$, её значениями – элементы множества $\{\text{ОШИБКА}\} \cup V_p$, где V_p – множество специальных символов, соответствующих грамматическим вхождениям или их множествам.

Алгоритм построения матрицы $g(X)$ конечного автомата

Вход: $G'=(T, V', P', S')$, Γ - множество грамматических вхождений, матрица OBLOW.

Выход: $g(X)$ - матрица размера $|\Gamma| \times |T \cup V'|$, N - множество грамматических вхождений, приводящих к недетерминированности.

$N := \emptyset$

foreach($X_{ij} \in \Gamma$)

foreach($Y_{kl} \in \Gamma \mid Y_{kl} \neq \perp, \text{OBLOW}[X_{ij}, Y_{kl}] = 1$)

$g[X_{ij}, \text{symbol}(Y_{kl})] := g[X_{ij}, Y] \cup \{Y_{kl}\} \ // \ \text{symbol}(Y_{kl}) = Y$

$N := N \cup g[X_{ij}, \text{symbol}(Y_{kl})] \mid |g[X_{ij}, \text{symbol}(Y_{kl})]| > 1$

end

end

Если некоторые грамматические вхождения содержатся исключительно во множествах с другими грамматическими вхождениями в ячейках, то их не нужно отдельно представлять, как магазинный символ, поскольку нет однозначных переходов к ним. Такие грамматические вхождения можно выделить в отдельное множество N , чтобы исключить из рассмотрения.

Матрица в каждой ячейке должна содержать не более одного грамматического вхождения, иначе проявляется недетерминированность и нужно определить новые состояния так, чтобы можно было задать магазинный алфавит V_p – каждому состоянию конечного автомата соответствует ровно один магазинный символ v_k .

Алгоритм построения матрицы переходов $\Phi(u)$ (примежutoчной) на основе матрицы $g(X)$, может быть построен недетерминированный КА:

Вход: $G'=(V', T, P', S')$, Γ – множество грамматических вхождений, g – матрица, построенная на предыдущем этапе, N – множество грамматических вхождений, приводящих к недетерминированности.

Выход: $F(u)$ - матрица переходов размера $|V_p| \times |T \cup V'|$, V_p - множество магазинных символов, $M:V_p \rightarrow \Gamma$ – словарь.

$V_p := \emptyset$

foreach ($X_{ij} \in \Gamma - N$)

$v_x = \text{GET_VP}(\{X_{ij}\}, M)$

$V_p = V_p \cup \{v_x\}$

foreach ($s \in (T \cup V')$)

$v_d = \text{GET_VP}(g[X_{ij}, s], M) \mid g[X_{ij}, s] \neq \emptyset$

$V_p = V_p \cup \{v_d\}$

$\Phi[v_x, s] = v_d$

// Разрешает недетерминированность

foreach ($Y_{kl} \in g[X_{ij}, s] \mid |g[X_{ij}, s]| > 1$)

foreach ($s' \in (T \cup V')$)

$\Phi[v_d, s'] = \text{GET_VP}(g[Y_{kl}, s'], M) \mid g[Y_{kl}, s'] \neq \emptyset$

end

end

end

end

Алгоритм. GET_VP(θ , M) – преобразовать множество грамматических вхождений θ (недетерминированность) в один магазинный символ v_k .

Вход: θ – множество грамматических вхождений, $M: V_p \rightarrow \Gamma$ – словарь.

Выход: v_θ – магазинный символ, соответствующий множеству θ .

if ($\theta \notin M$)

$M[\text{index}(\theta)] = \theta$ // $\text{index}(\theta)$ – строковое представление множества

end

return $\text{index}(\theta)$

Алгоритм. Построение матрицы функции действий $F(u)$.

Вход: $G'=(V', T, P', S')$, V_p – множество магазинных символов, M – словарь отображения $V_p \rightarrow \Gamma$, Φ – матрица переходов размера $|V_p| \times |T \cup V'|$.

Выход: F - матрица действий размера $|V_p| \times ||T \cup \{\varepsilon\}|$

foreach ($v \in V_p$)

foreach ($t \in \{T \cup \{\varepsilon\}\}$)

if ($M[v] = \{S_0\}$ и $t = \varepsilon$) // $(0) S' \rightarrow S$

```

    F[v,t] = ДОПУСК
  else if (Φ[v,t] ≠ ∅ и t ≠ ε )
    F[v,t] = ПЕРЕНОС
  else if (t ∈ FOLLOW(v) и (Xij ∈ M[v] : j = |a| (Ai → αXij)) !!
    F[v,t] = (СВЕРТКА, i)
  else
    exit // Если ни один из рассмотренных выше случаев не выполнен, то это
    значит, что построить функцию действий нельзя – грамматика не является LR(1)-
    грамматикой.
  end
end
end
end

```

Шаг 4. Алгоритм перенос-свертка. Выполнение алгоритма описывается в терминах конфигураций, представляющих собой тройки вида $(\perp \alpha T, ax, \pi)$, где αT – цепочка магазинных символов (T-верхний символ магазина), ax – необработанная часть входной цепочки, π – выход (строка из номеров правил), построенный к настоящему моменту времени.

Таблица состоит из двух подтаблиц – функции действия и функции переходов. Входным символам с ленты соответствуют столбцы таблицы, символам магазина – строки.

Управление алгоритмом осуществляется двумя функциями, задаваемых в таблице:

1. По верхнему символу магазина и входному символу, определяется функция действия: *ПЕРЕНОС* или *СВЕРТКА*;
2. *ПЕРЕНОС* определяет значение функции перехода, равное магазинному символу, который нужно втолкнуть в магазин;
3. *СВЕРТКА*(i), однозначно определяет правило i для свертки.

Вход: Анализируемая цепочка $z = t_1 t_2 \dots t_j \dots t_n \in T^*$, где j - номер текущего символа входной цепочки, находящегося под читающей головкой. Управляющая таблица M (множество LR(k)-таблиц) для LR(k)-грамматики $G = (T, V, P, S)$

Выход: Если $z \in L(G)$, то правый разбор цепочки z, в противном случае – ошибка.

Алгоритм.

1. $j := 0$.
2. $j := j + 1$. Если $j > n$, то выдать сообщение об ошибке и перейти к шагу 5.
3. Определить цепочку u следующим образом:
 - если $k = 0$, то $u = t_j$;
 - если $k \geq 1$ и $j + k - 1 \leq n$, то $u = t_j t_{j+1} \dots t_{j+k-1}$ – первые k-символов цепочки $t_j t_{j+1} \dots t_n$;
 - если $k \geq 1$ и $j + k - 1 > n$, то $u = t_j t_{j+1} \dots t_n$ – остаток входной цепочки.
4. Применить функцию действия f из строки таблицы M, отмеченной верхним символом магазина г, к цепочке u:

$f(u) = \text{ПЕРЕНОС (П)}$. Определить функцию перехода $g(t_j)$ из строки таблицы M , отмеченной символом T из верхушки магазина. Если $g(t_j) = T'$ и $T' \in V_p \cup \{\perp\}$, **то записать T' в магазин** и перейти к шагу 2. Если $g(t_j) = \text{ОШИБКА}$, то выдать сигнал об ошибке и перейти к шагу 5;

$f(u) = (\text{СВЕРТКА}, i) \text{ (С)}$ и $A \rightarrow \alpha$ – правило вывода с номером i грамматики G . Удалить из верхней части магазина $|\alpha|$ символов, в результате чего в верхушке магазина окажется символ $T' \in V_p \cup \{\perp\}$, и выдать номер правила i на входную ленту. Определить символ $T'' = g(A)$ из строки таблицы M , отмеченной символом T' , записать его в магазин и перейти к шагу 3.

$f(u) = \text{ОШИБКА (О)}$. Выдать сообщение об ошибке и перейти к шагу 5.

$f(u) = \text{ДОПУСК (Д)}$. Объявить цепочку, записанную на входной ленте, правым разбором входной цепочки z .

5. Останов.

Шаг В. При кодировании магазинному символу соответствует несколько вхождений, то это $\text{SLR}(1)$ -грамматика и таблица действий $f(a)$ задается алгоритмом.

Вход: $\text{LR}(0)$ -грамматика $G = (T, V, P, S)$, не содержащая ϵ -правил.

Выход: Множество $\text{TLR}(0)$ -таблиц для грамматики G или сообщение о том, что грамматика G не является $\text{LR}(0)$ грамматикой.

Для магазинного символа Γ , представляющего множество грамматических вхождений Q , и входного символа a значение функции действий $f(a)$ определяется следующим образом:

- если начальное вхождение $S_0 \in Q$ и $a = \epsilon$, то $f(\epsilon) = \text{ДОПУСК}$;
- если $a \neq \epsilon$ и $g(a) \neq \text{ОШИБКА}$, то $f(a) = \text{ПЕРЕНОС}$;
- если $X_j \in Q$ — самое правое грамматическое вхождение i -го правила $A \rightarrow \alpha X_j$ и $a \in \text{FOLLOW}(A)$, то $f(a) = (\text{СВЕРТКА}, i)$. Значение остальных элементов для $f(a)$ — **ОШИБКА**.

Если имеется множество грамматических вхождений, не удовлетворяющих перечисленным в шаге 4 условиям, то грамматика не принадлежит классу $\text{SLR}(1)$.

Пример построения $\text{LR}(0)$ -грамматики. Добавление **продукций изображенных красным цветом** вызывают неоднозначность, что требует построения анализатора для $\text{LR}(1)$ -грамматики.

Построим управляющую таблицу $\text{LR}(0)$ анализатора для следующей грамматики:

$G = \{T, V, P, S\}$, где $T = \{+, (,), I, *\}$, $V = \{S, T, P\}$,

P : (1) $S \rightarrow S + T$

(2) $S \rightarrow T$

(3) $T \rightarrow T * P$ // для этих правил $\text{LR}(0)$ -анализатор не справляется

(4) $T \rightarrow P$

(5) $P \rightarrow (E)$

(6) $P \rightarrow i$

Шаг 1. Определим пополненную грамматику G' для грамматики G' :

$$G' = \{T, V \cup \{S'\}, P \cup \{S' \rightarrow S\}, S\}$$

Шаг 2. Найдем множество грамматических вхождений Γ и матрицу отношения $OBL\!O\!W$ для грамматики G' согласно алгоритму 2:

Вход: $G' = \{T, V', P', S\}$

Выход: Γ – множество грамматических вхождений, $OBL\!O\!W$ – матрица отношения “входит под”.

Будем рассматривать правила пополненной грамматики G' как:

$$P': S'_0 \rightarrow S_{01}$$

$$S_1 \rightarrow S_{11} +_{12} T_{13}$$

$$S_2 \rightarrow T_{21}$$

$$T_3 \rightarrow T_{31} *_{32} P_{33}$$

$$T_4 \rightarrow P_{41}$$

$$P_5 \rightarrow (_{51} S_{52})_{53}$$

$$P_6 \rightarrow i_{61}$$

Шаг 0. $\Gamma := \{\perp\}$

Шаг 1. для цепочки правила $S'_0 \rightarrow S_{01}$:

$$\Gamma := \Gamma \cup \{S_{01}\} = \{\perp, S_{01}\}$$

Это пополненное правило, поэтому в матрицу $OBL\!O\!W$ необходимо записать, что под маркером дна $\perp$ содержатся грамматические вхождения из $OFIRST(S_{01})$.

$$OFIRST(S_{01}) = \{S_{01}, S_{11}, T_{21}, T_{31}, P_{41}, (_{51} i_{61})\}$$

$$OBL\!O\!W[\perp, S_{01}] = 1; OBL\!O\!W[\perp, S_{11}] = 1; OBL\!O\!W[\perp, T_{21}] = 1; OBL\!O\!W[\perp, T_{31}] = 1;$$

$$OBL\!O\!W[\perp, P_{41}] = 1; OBL\!O\!W[\perp, (_{51})] = 1; OBL\!O\!W[\perp, i_{61}] = 1;$$

Шаг 2. для цепочки правила $S_1 \rightarrow S_{11} +_{12} T_{13}$:

$$\Gamma := \Gamma \cup \{S_{11}, +_{12}, T_{13}\} = \{\perp, S_{01}, S_{11}, +_{12}, T_{13}\}$$

$$1. OFIRST(+_{12}) = \{+_{12}\}$$

$$OBL\!O\!W[S_{11}, +_{12}] = 1;$$

$$2. OFIRST(T_{13}) = \{T_{13}, T_{31}, P_{41}, (_{51} i_{61})\}$$

$$OBL\!O\!W[+_{12}, T_{13}] = 1; OBL\!O\!W[+_{12}, T_{31}] = 1; OBL\!O\!W[+_{12}, P_{41}] = 1; OBL\!O\!W[+_{12}, (_{51})] = 1;$$

$$OBL\!O\!W[+_{12}, i_{61}] = 1;$$

Шаг 3. для цепочки правила $S_2 \rightarrow T_{21}$:

$$\Gamma := \Gamma \cup \{T_{21}\} = \{\perp, S_{01}, S_{11}, +_{12}, T_{13}, T_{21}\}$$

Шаг 4. для цепочки правила $T_3 \rightarrow T_{31} *_{32} P_{33}$:

$$\Gamma := \Gamma \cup \{T_{31} *_{32} P_{33}\} = \{\perp, S_{01}, S_{11}, +_{12}, T_{13}, T_{21}, T_{31}, *_{32}, P_{33}\}$$

$$1. OFIRST(*_{32}) = \{*_{32}\}$$

$$OBL\!O\!W[T_{31}, *_{32}] = 1$$

$$2. OFIRST(P_{33}) = \{P_{33}, (_{51} i_{61})\}$$

$$OBL\!O\!W[*_{32}, P_{33}] = 1; OBL\!O\!W[*_{32}, (_{51})] = 1; OBL\!O\!W[*_{32}, i_{61}] = 1;$$

Шаг 5. для цепочки правила $T_4 \rightarrow P_{41}$:

$$\Gamma := \Gamma \cup \{P_{41}\} = \{\perp, S_{01}, S_{11}, +_{12}, T_{13}, T_{21}, T_{31}, *_{32}, P_{33}, P_{41}\}$$

Шаг 6. для цепочки правила $P_5 \rightarrow (_{51} E_{52})_{53}$:

$$\Gamma := \Gamma \cup \{(_{51}, S_{52},)_{53}\} = \{\perp, S_{01}, S_{11}, +_{12}, T_{13}, T_{21}, T_{31}, *_{32}, P_{33}, P_{41}, (_{51}, S_{52},)_{53}\}$$

$$1. OFIRST()_{53} = \{)_{53} \}$$

$$OBL\!O\!W[(_{51}, S_{52},)_{53}] = 1$$

$$2. OFIRST(S_{52}) = \{S_{11}, T_{21}, T_{31}, P_{41}, (_{51}, S_{52}, i_{61})\}$$

$$OBL\!O\!W$$

Шаг 7. для цепочки правила $P_6 \rightarrow i_{61}$:

$$\Gamma := \Gamma \cup \{ \perp, S_{01}, S_{11}, +_{12}, T_{13}, T_{21}, T_{31}, *_{32}, P_{33}, P_{41}, (_{51}, S_{52},)_{53}, i_{61} \}$$

Множество грамматических вхождений:

$$\Gamma := \Gamma \cup \{ \perp, S_{01}, S_{11}, +_{12}, T_{13}, T_{21}, T_{31}, *_{32}, P_{33}, P_{41}, (_{51}, S_{52},)_{53}, i_{61} \}$$

Матрица OBLOW.

Γ	S_{01}	S_{11}	$+_{12}$	T_{13}	T_{21}	T_{31}	$*_{32}$	P_{33}	P_{41}	$(_{51}$	S_{52}	$)_{53}$	i_{61}
$\perp$	1	1			1	1			1	1			1
S_{01}													
S_{11}			1										
$+_{12}$				1		1			1	1			1
T_{13}													
T_{21}													
T_{31}							1						
$*_{32}$								1		1			1
P_{33}													
P_{41}													
$(_{51}$		1			1	1			1	1	1		1
S_{52}													
$)_{53}$												1	
i_{61}													

Алгоритм построения матрицы функции переходов $g(x)$.

Вход: $G' = (V', T, P', S')$, Γ - множество грамматических вхождений, матрица OBLOW.

Выход: g - матрица размера $|\Gamma| \times |T \cup V'|$. N – множество грамматических вхождений, приводящих к недетерминированности.

Шаг 0. $N = \emptyset$

Шаг 1. для $\perp$:

$$g[\perp, S] = \{ S_{01}, S_{11} \}$$

$$g[\perp, T] = \{ T_{21}, T_{31} \}$$

$$g[\perp, P] = \{ P_{41} \}$$

$$g[\perp, (] = \{ (_{51} \}$$

$$g[\perp, i] = \{ i_{61} \}$$

Т.к. $g[\perp, S]$ и $g[\perp, T]$ содержит несколько грамматических вхождений, все их нужно добавить в множество $N = N \cup \{ S_{01}, S_{11} \} \cup \{ T_{21}, T_{31} \} = \{ S_{01}, S_{11}, T_{21}, T_{31} \}$

Шаг 2. для S_{11}

$$g[S_{11}, +] = \{ +_{12} \}$$

Шаг 3. для $+_{12}$

$$g[+_{12}, T] = \{ T_{13}, T_{31} \}$$

$$g[+_{12}, P] = \{ P_{41} \}$$

$$g [+_{12}, () = \{ (_{51} \}$$

$$g [+_{12}, i] = \{ i_{61} \}$$

Т.к. $g [+_{12}, T]$ содержит несколько грамматических вхождений, все их нужно добавить в множество $N = N \cup \{ T_{13}, T_{31} \} = \{ S_{01}, S_{11}, T_{21}, T_{31}, T_{13} \}$

Шаг 4. для T_{31}

$$g [T_{31}, *] = \{ *_{32} \}$$

Шаг 5. для $*_{32}$

$$g [*_{32}, P] = \{ P_{33} \}$$

$$g [*_{32}, () = \{ (_{51} \}$$

$$g [*_{32}, i] = \{ i_{61} \}$$

Шаг 6. для $(_{51}$

$$g [(_{51}, S] = \{ S_{11}, S_{52} \}$$

$$g [(_{51}, T] = \{ T_{21}, T_{31} \}$$

$$g [(_{51}, P] = \{ P_{41} \}$$

$$g [(_{51}, () = \{ (_{51} \}$$

$$g [(_{51}, i] = \{ i_{61} \}$$

$g [(_{51}, S]$ и $g [(_{51}, T]$ содержит несколько грамматических вхождений, все их нужно добавить в множество $N = N \cup \{ S_{11}, S_{52} \} \cup \{ T_{21}, T_{31} \} = \{ S_{01}, S_{11}, T_{21}, T_{31}, T_{13}, S_{52} \}$

Шаг 7. для $)_{53}$ $g [)_{53},)] = \{)_{53} \}$

	S	+	T	*	P	(	)	i
$\perp$	$\{S_{01}, S_{11}\}$		$\{T_{21}, T_{31}\}$		$\{P_{41}\}$	$(_{51}$		i_{61}
S_{01}								
S_{11}		$\{+_{12}\}$						
$+_{12}$			$\{T_{13}, T_{31}\}$		$\{P_{41}\}$	$(_{51}$		i_{61}
T_{13}								
T_{21}								
T_{31}				$\{*_{32}\}$				
$*_{32}$					P_{33}	$(_{51}$		i_{61}
P_{33}								
P_{41}								
$(_{51}$	$\{S_{11}, S_{52}\}$		$\{T_{21}, T_{31}\}$		P_{41}	$(_{51}$		i_{61}
S_{52}								
$)_{53}$							$)_{53}$	
i_{61}								

Полученное множество грамматических вхождений N , приводящих к недетерминированности:

$$N = \{ S_{01}, S_{11}, T_{21}, T_{31}, T_{13}, S_{52} \}$$

Применим матрицу g для построения матрицы Φ функции переходов $g(a)$, зададим множество магазинных символов V_p и словарь отображения $M: V_p \rightarrow \Gamma$.

Алгоритм построения матрицы переходов $g(x)$:

Вход: $G' = (V', T, P', S')$, Γ – множество грамматических вхождений, g – матрица, построенная на предыдущем этапе, N – множество грамматических вхождений, приводящих к недетерминированности.

Выход: Φ – матрица переходов размера $|V_p| \times |TUV|$, V_p – множество магазинных символов, M – словарь, представляющий отображение $V_p \rightarrow \Gamma$.

Шаг 1. Для $\perp$: $M[\perp] := \{\perp\}$

Рассмотрим все $s \in (TUV')$, для которых заданы переходы из $\perp$:

1. $s=S$: $g[\perp, S] = \{S_{01}, S_{11}\}$, $M[v_1] = \{S_{01}, S_{11}\}$ $\Phi[\perp, S] = v_1$

Поскольку магазинный символ v_1 задает несколько грамматических вхождений, необходимо составить таблицу для него:

1) Для S_{01} – нет грамматических вхождений для перехода.

2) Для S_{11} : $g[S_{11}, +] = \{+_{12}\}$, $M[+_{12}] = \{+_{12}\}$ $\Phi[v_1, +] = +_{12}$

2. $s=T$: $g[\perp, T] = \{T_{21}, T_{31}\}$, $M[v_2] = \{T_{21}, T_{31}\}$ $\Phi[\perp, T] = v_2$

Поскольку магазинный символ v_2 задает несколько грамматических вхождений, необходимо составить таблицу для него:

1) Для T_{21} – нет грамматических вхождений для перехода.

2) Для T_{31} : $g[T_{31}, *] = \{*_32\}$, $M[*_32] = \{*_32\}$ $\Phi[v_2, *] = *_32$

3. $s=P$: $g[\perp, P] = \{P_{41}\}$, $M[P_{41}] = \{P_{41}\}$ $\Phi[\perp, P] = P_{41}$

4. $s=($: $g[\perp, (] = \{(5_1)\}$, $M[(5_1)] = \{(5_1)\}$ $\Phi[\perp, (] = (5_1$

5. $s=i$: $g[\perp, i] = \{i_{61}\}$, $M[i_{61}] = \{i_{61}\}$ $\Phi[\perp, i] = i_{61}$

Шаг 2. Для $+_{12}$: $M[+_{12}] = \{+_{12}\}$ (был задан ранее)

Рассмотрим все $s \in (TUV')$, для которых заданы переходы из $+_{12}$:

1. $s=U$: $g[+_{12}, T] = \{T_{13}, T_{31}\}$, $M[v_3] := \{T_{13}, T_{31}\}$ $\Phi[+_{12}, T] = v_3$

Поскольку магазинный символ v_3 задает несколько грамматических вхождений, необходимо составить таблицу для него:

1) Для T_{13} – нет грамматических вхождений для перехода.

2) Для T_{31} : $g[T_{31}, *] = \{*_32\}$, $M[*_32] := \{*_32\}$ (задан ранее) $\Phi[v_3, *] = *_32$

2. $s=($: $g[+_{12}, (] = \{(3_1)\}$, $M[(3_1)] = \{(3_1)\}$ (задан ранее) $\Phi[+_{12}, (] = (3_1$

3. $s=i$: $g[+_{12}, i] = \{i_{61}\}$, $M[i_{61}] = \{i_{61}\}$ (задан ранее) $\Phi[+_{12}, i] = i_{61}$

4. $s=P$: $g[+_{12}, P] = \{P_{41}\}$, $M[P_{41}] = \{P_{41}\}$ (задан ранее) $\Phi[+_{12}, P] = P_{41}$

Шаг 3. Для T_{31} : $M[T_{31}] = \{T_{31}\}$ Рассмотрим все $s \in (TUV')$, для которых заданы переходы из $(3_1$:

1. $s=*$: $g[T_{31}, *] = \{*_32\}$, $M[*_32] := \{*_32\}$ (задан ранее) $\Phi[T_{31}, *] = *_32$

Шаг 4. Для $*_32$: $M[*_32] = \{*_32\}$ (задан ранее) Рассмотрим все $s \in (TUV')$, для которых заданы переходы из $(3_1$:

1. $s=P$: $g[*_32, P] = \{P_{33}\}$, $M[P_{33}] := \{P_{33}\}$ $\Phi[*_32, P] = P_{33}$

2. $s=($: $g[*_32, (] = \{(5_1)\}$, $M[(5_1)] := \{(5_1)\}$ (задан ранее) $\Phi[*_32, (] = (5_1$

3. $s=i$: $g[*_32, i] = \{i_{61}\}$, $M[i_{61}] := \{i_{61}\}$ (задан ранее) $\Phi[*_32, i] = i_{61}$

Шаг 5. Для $(5_1$: $M[(5_1)] = \{(5_1)\}$ (задан ранее). Рассмотрим все $s \in (TUV')$, для которых заданы переходы из $(3_1$:

1. $s=S$: $g[(5_1, S] = \{S_{11}, S_{52}\}$, $M[v_4] := \{S_{11}, S_{52}\}$ $\Phi[(5_1, S] = v_4$ Поскольку магазинный символ v_4 задает несколько грамматических вхождений, необходимо составить таблицу для него:

1) Для S_{52} – нет грамматических вхождений для перехода.

2) Для S_{11} : $g[S_{11}, +] = \{+_{12}\}$, $M[+_{12}] := \{+_{12}\}$ (задан ранее) $\Phi[v_4, +] = +_{12}$

2. $s=T$: $g[(5_1, T] = \{T_{21}, T_{31}\}$, $M[v_2] := \{T_{21}, T_{31}\}$ $\Phi[(5_1, T] = v_2$

3. $s=P$: $g[(5_1, P] = \{P_{41}\}$, $M[P_{41}] := \{P_{41}\}$ $\Phi[(5_1, P] = P_{41}$

3. $s=($: $g[(5_1, (] = \{(5_1)\}$, $M[(5_1)] := \{(5_1)\}$ $\Phi[(5_1, (] = (5_1$

3. $s=i$: $g[(5_1, i] = \{i_{61}\}$, $M[i_{61}] := \{i_{61}\}$ $\Phi[(5_1, i] = i_{61}$

Шаг 6. Для $)_{53}$: $M[()_{53}] = \{)_{53} \}$. Рассмотрим все $s \in (T \cup V')$, для которых заданы переходы из $(_{31}$:

1. $s =)$: $g[T_{31},)] = \{)_{53} \}$, $M[()_{53}] := \{)_{53} \}$ (задан ранее) $\Phi [T_{31},)] =)_{53}$

Полученное множество магазинных символов V_p :

$$V_p = \{ \perp, v_1, +_{12}, v_2, *_{32}, P_{41}, (_{51}, i_{61}, v_3, T_{31}, P_{33}, v_4,)_{53} \}$$

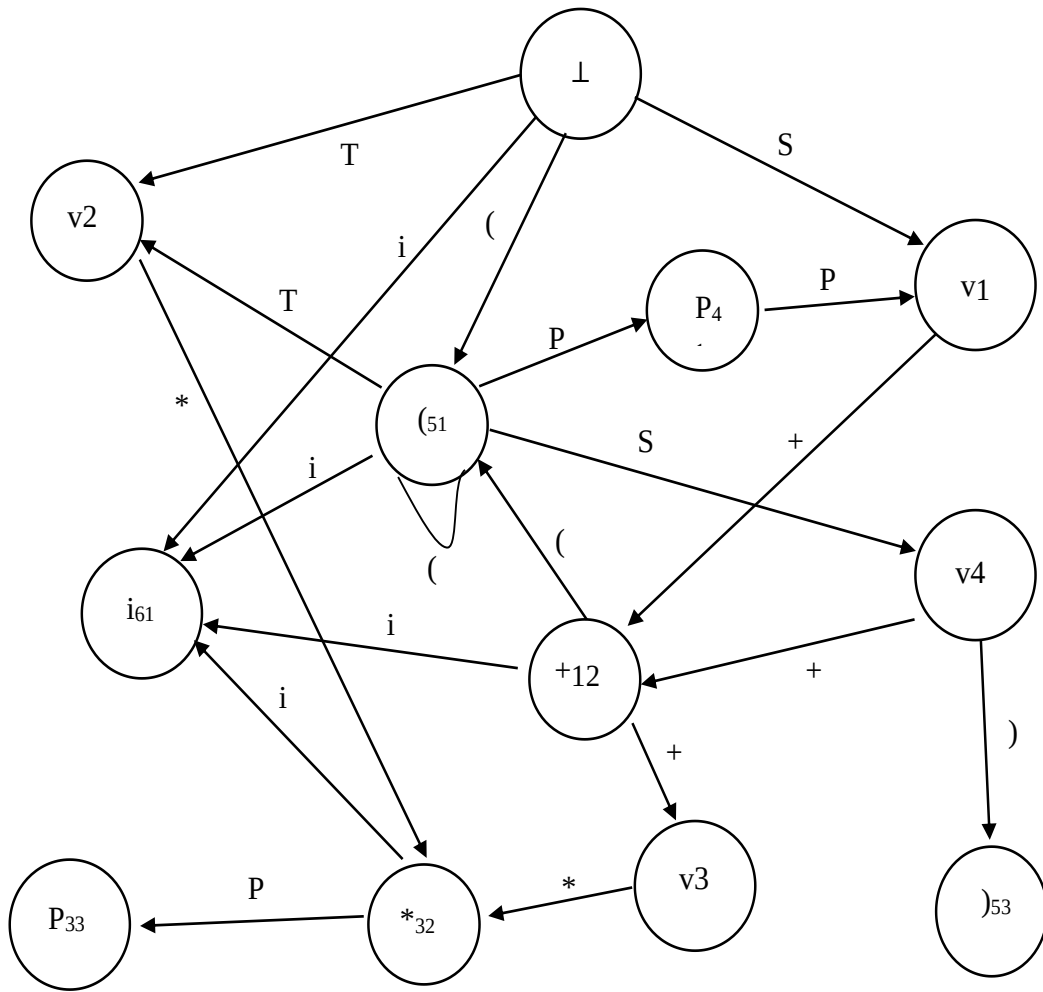
Полученный словарь отображения $M: V_p \rightarrow \Gamma$:

V_p	$\perp$	v_1	$+_{12}$	v_2	$*_{32}$	P_{41}	$(_{51}$	i_{61}	v_3	T_{31}	P_{33}	V_4	$)_{53}$
Γ	$\{\perp\}$	$\{S_{01}, S_{11}\}$	$\{+_{12}\}$	$\{T_{21}, T_{31}\}$	$\{*_{32}\}$	$\{P_{41}\}$	$\{(_{51}\}$	$\{i_{61}\}$	$\{T_{13}, T_{31}\}$	$\{T_{31}\}$	$\{P_{33}\}$	$\{S_{11}, S_{52}\}$	$\{)_{53} \}$

Матрица переходов Φ .

	S	+	T	*	P	(	)	i
$\perp$	v_1		v_2		P_{41}	$(_{51}$		i_{61}
v_1		$+_{12}$						
$+_{12}$			v_3			$(_{51}$		i_{61}
v_2				$*_{32}$				
$*_{32}$					P_{33}	$(_{51}$		i_{61}
P_{41}								
$(_{51}$	v_4		v_2		P_{41}	$(_{51}$		i_{61}
i_{61}								
v_3				$*_{32}$				
P_{33}								
v_4		$+_{12}$					$)_{53}$	
$)_{53}$								

Матрица переходов Φ задает конечный автомат, с диаграммой, где магазинные символы - вершины, ребра - операции переноса терминального или нетерминального символов:



Шаг 4. Построить матрицу действия $F(u)$ для всех магазинных символов V_p , каждому из которых соответствует один ряд таблицы, согласно **алгоритму**:

Вход: $G' = (T, V', P', S')$, V_p – множество магазинных символов, M – словарь отображения $V_p \rightarrow \Gamma$, Z – матрица переходов размера $|V_p| \times |T \cup V'|$.

Выход: F – матрица действий размера $|V_p| \times |T \cup \{\epsilon\}|$

Рассмотрим все $v \in V_p = \{\perp, v_1, +_{12}, v_2, *_{32}, P_{41}, (_{51}, i_{61}, v_3, T_{31}, P_{33}, v_4,)_{53}\}$

1. $v = \perp$, $M[\perp] = \{\perp\} \Rightarrow \forall t \in \Phi[v, t], F[\perp, t] = \text{ПЕРЕНОС}$
2. $v = v_1$, $M[v_1] = \{S_{01}, S_{11}\} \Rightarrow \forall t \in \Phi[v, t], F[v_1, t] = \text{ПЕРЕНОС}$
3. $v = +_{12}$, $M[+_{12}] = \{+_{12}\} \Rightarrow \forall t \in \Phi[v, t], F[+_{12}, t] = \text{ПЕРЕНОС}$
4. $v = v_2$, $M[v_2] = \{T_{21}, T_{31}\} \Rightarrow \forall t \in \Phi[v, t], F[v_2, t] = \text{ПЕРЕНОС}$

$\text{FOLLOW}(T_{21}) = \{+, \epsilon\}$ т.к. T_{21} является правым вхождением правила 2, то $\forall t \in \text{FOLLOW}(v_2), F[v_2, t] = \text{СВЕРТКА}, 2$

5. $v = *_{32}$, $M[*_{32}] = \{*_{32}\} \Rightarrow \forall t \in \Phi[v, t], F[*_{32}, t] = \text{ПЕРЕНОС}$
6. $v = P_{41}$, $M[P_{41}] = \{P_{41}\}$

$\text{FOLLOW}(P_{41}) = \{+, *, \epsilon\}$ т.к. P_{41} является правым вхождением правила 4, то $\forall t \in \text{FOLLOW}(P_{41}), F[P_{41}, t] = \text{СВЕРТКА}, 4$

7. $v = (_{51}$, $M[(_{51}] = \{(_{51}\} \Rightarrow \forall t \in \Phi[v, t], F[(_{51}, t] = \text{ПЕРЕНОС}$

8. $v = i_{61}$, $M[i_{61}] = \{i_{61}\}$
 $FOLLOW(i_{61}) = \{+, *,), \varepsilon\}$ т.к. i_{61} является правым вхождением правила 6, то $\forall t \in FOLLOW(i_{61})$, $F[P_{41}, t] = СВЕРТКА$, 6
9. $v = v_3$, $M[v_3] = \{T_{13}, T_{31}\} \Rightarrow \forall t \in \Phi[v, t]$, $H[v_3, t] = ПЕРЕНОС$
 $FOLLOW(T_{13}) = \{+,), \varepsilon\}$ т.к. T_{13} является правым вхождением правила 1, то $\forall t \in FOLLOW(T_{13})$, $F[v_3, t] = СВЕРТКА$, 2
10. $v = T_{31}$, $M[T_{31}] = \{T_{31}\} \Rightarrow \forall t \in \Phi[v, t]$, $F[T_{31}, t] = ПЕРЕНОС$
11. $v = P_{33}$, $M[P_{33}] = \{P_{33}\}$
 $FOLLOW(P_{33}) = \{+, *,), \varepsilon\}$ т.к. P_{33} является правым вхождением правила 3, то $\forall t \in FOLLOW(P_{33})$, $F[P_{33}, t] = СВЕРТКА$, 3
12. $v = v_4$, $M[v_4] = \{S_{11}, S_{52}\} \Rightarrow \forall t \in \Phi[v, t]$, $F[v_4, t] = ПЕРЕНОС$
 $F[v_4, \varepsilon] = ДОПУСК$
13. $v =)_{53}$, $M[)]_{53}] = \{)_{53} \}$
 $FOLLOW()_{53}) = \{+, *,), \varepsilon\}$ т.к. $)_{53}$ является правым вхождением правила 5, то $\forall t \in FOLLOW()_{53})$, $F[)]_{53}, t] = СВЕРТКА$, 5

Матрица действий $F(u)$:

	+	*	(	)	i	ε
$\perp$			П		П	
v_1	П					Д
$+_{12}$			П		П	
v_2	C,2	П		C,2		C,2
$*_{32}$			П		П	
P_{41}	C,4	C,4		C,4		C,4
$(_{51}$			П		П	
i_{61}	C,6	C,6		C,6		C,6
v_3	C,1	П		C,1		C,1
P_{33}	C,3	C,3		C,3		C,3
V_4	П			П		
$)_{53}$	C,5	C,5		C,5		C,5

Управляющая таблица М.

Γ	$F(u)$						$g(x)$						
	+	*	(	)	i	ε	S	+	T	*	P	(	)
$\perp$			П		П		v_1		v_2		P_{41}	$(_{51}$	
v_1	П					Д		$+_{12}$					
12			П		П				v_3			$(_{51}$	
v_2	C,2	П		C,2		C,2				$*_{32}$			
$*_{32}$			П		П						P_{33}	$(_{51}$	
P_{41}	C,4	C,4		C,4		C,4							
$(_{51}$			П		П		v_4		v_2		P_{41}	$(_{51}$	
i_{61}	C,6	C,6		C,6		C,6							
v_3	C,1	П		C,1		C,1				$*_{32}$			
P_{33}	C,3	C,3		C,3		C,3							
v_4	П			П				$+_{12}$					$)_{53}$

) ₅₃	C,5	C,5		C,5		C,5								
-----------------	-----	-----	--	-----	--	-----	--	--	--	--	--	--	--	--

Шаг 4. Применение алгоритма перенос-свертка.

Разница между LR(0) и LR(1) анализаторами

LR(0) требует только канонических элементов для построения таблицы синтаксического анализа, LR(1) требует символ просмотра вперед.

LR(1) позволяет выбирать корректные операции свертки и использовать грамматику, которая не является однозначно обратимой, а LR(0) - нет.

LR(0) операции свертки для всех элементов, тогда как LR(1) только для символов просмотра вперед.

LR(0) = канонические элементы и LR(1) = LR(0) + символ просмотра вперед.

В канонической коллекции LR(0) состояние/элемент имеет вид:

Например. $I_2 = \{C \rightarrow AB\bullet\}$

тогда как в канонической форме LR(1) состояние/элемент имеет вид:

Например. $I_2 = \{C \rightarrow d\bullet, e/d\}$, где e и d - символ просмотра вперед.

Построение канонической формы LR(0) начинается с $C = \{ \text{CLOSURE} (\{S' \rightarrow S\}) \}$, тогда как построение канонической коллекции LR(1) начинается с $C = \{ \text{CLOSURE} (\{S' \rightarrow S, \$\}) \}$ где S — начальный символ.

В LR(0) может возникнуть конфликт в таблице синтаксического анализа, в LR(1) используется просмотр вперед, чтобы избежать ненужных конфликтов в таблице синтаксического анализа.

Алгоритм LR(0) проще, чем алгоритм LR(1).

II. Построение анализатора LR(0)-грамматик на основе LR(0)-ситуаций: функции замыкания CLOSURE и перехода GOTO

Определение. LR(0) ситуация - это правило грамматики с точкой в некоторой позиции правой части, например $[A \rightarrow w_1 \bullet w_2]$, если $A \rightarrow w_1 w_2$ — правило КС-грамматики.

Пример. Для правила $S \rightarrow (S)$ можно получить 4 ситуации:

$[S \rightarrow \bullet (S)]$

$[S \rightarrow (\bullet S)]$

$[S \rightarrow (S \bullet)]$

$[S \rightarrow (S) \bullet]$

Замечание. LR(0)-ситуация не содержит аванцепочку ϵ , поэтому при ее записи можно опускать квадратные скобки.

Утверждение. Для каждого активного префикса существует непустое множество ситуаций.

Для простого LR(0)-анализатора вначале на основе КС-грамматики строится детерминированный конечный автомат (КА) для распознавания активных префиксов: ситуации группируются в множества, изменяющие состояния анализатора.

Шаг 1. Определение ситуаций и построение конечного автомата

Пусть I – множество LR(0)-ситуаций КС-грамматики G . Тогда назовем замыканием множества I множество ситуаций $CLOSURE(I)$, построенное по следующим правилам:

1. Включить в $CLOSURE(I)$ все ситуации из I .
2. Если ситуация $A \rightarrow \alpha \bullet B \beta$ уже включена в $CLOSURE(I)$ и $B \rightarrow \gamma$ – правило грамматики, то добавить в множество $CLOSURE(I)$ ситуацию $B \rightarrow \bullet \gamma$ при условии, что там ее еще нет.
 - Наличие ситуации $A \rightarrow \alpha \bullet B \beta$ в множестве $CLOSURE(I)$ говорит о том, что в некоторый момент разбора может встретиться во входном потоке анализатора подстрока, выводимая из $B \beta$.
 - Если в грамматике имеется правило $B \rightarrow \gamma$, то также может встретиться во входном потоке анализатора подстрока, выводимая из γ , следовательно, в $CLOSURE(I)$ нужно включить ситуацию $B \rightarrow \bullet \gamma$.
3. Повторять правило 2, до тех пор, пока в $CLOSURE(I)$ нельзя будет включить новую ситуацию.

Пример 1. Исходная КС-грамматика : $G(\{V = \{S, L, F\}, T = \{i, =, (*), P, S\}$
 $P = \{S \rightarrow F=L, S \rightarrow L, F \rightarrow (*L), F \rightarrow i, L \rightarrow F\}$
 Пополненная грамматика G_0 содержит еще одно правило: $S' \rightarrow S$

Замыкание множества ситуаций

Пусть вначале множество ситуаций $CLOSURE(I_0)$ содержит одну ситуацию $S' \rightarrow \bullet S$, т.е. $CLOSURE(I_0) = \{S' \rightarrow \bullet S\}$

Ситуация $S' \rightarrow \bullet S$ содержит точку перед символом S , поэтому по второму правилу в $CLOSURE(I_0)$ необходимо включить S -правила с точкой слева. Окончательно получим:

$$I_0 = CLOSURE(\{S' \rightarrow \bullet S\})$$

$$\text{Шаг 1: } I_0 = \{S' \rightarrow \bullet S\}$$

$$\text{Шаг 2: } I_0 = I_0 \cup \{S \rightarrow \bullet F=L, S \rightarrow \bullet L\}$$

$$\text{Шаг 3: } I_0 = I_0 \cup \{F \rightarrow \bullet (*L), F \rightarrow \bullet i\}$$

$$\text{Шаг 4: } I_0 = I_0 \cup \{L \rightarrow \bullet F\}$$

$$I_0 = \{S' \rightarrow \bullet S, S \rightarrow \bullet F=L, S \rightarrow \bullet L, F \rightarrow \bullet (*L), F \rightarrow \bullet i, L \rightarrow \bullet F\}$$

Если I -множество ситуаций, допустимых для некоторого активного префикса γ , то $GOTO(I, X)$ – это множество ситуаций, допустимых для активного префикса γX .

Аргументами функции $GOTO(I, X)$ являются множество ситуаций I и символ грамматики X .

Определение. Функция $GOTO(I, X)$ определяется как замыкание множества всех ситуаций $[A \rightarrow \alpha X \bullet \beta]$, таких что $[A \rightarrow \alpha \bullet X \beta] \in I$

Пусть для грамматики G_0 некоторое множество $I = \{[S' \rightarrow S \bullet], [S \rightarrow F \bullet =L]\}$.

Для вычисления значения $GOTO(I_0, =)$ необходимо рассмотреть ситуации, в которых непосредственно сразу за точкой следует символ $=$. Это ситуация $[S \rightarrow F \bullet =L]$. Перемещая точку за символ $=$, построим множество ситуаций $[S \rightarrow F = \bullet L]$ и замыкание этого множества: $\{L \rightarrow \bullet F, F \rightarrow \bullet (*L), F \rightarrow \bullet i\}$. Тогда $GOTO(I_2, =) = \{S \rightarrow F = \bullet L, L \rightarrow \bullet F, F \rightarrow \bullet (*L), F \rightarrow \bullet i\} = I_6$

Функция переходов GOTO

$I_0 = \text{CLOSURE}(\{S' \rightarrow \bullet S\}) = \{S' \rightarrow \bullet S, S \rightarrow \bullet F=L, S \rightarrow \bullet L, F \rightarrow \bullet (*L), F \rightarrow \bullet i, L \rightarrow \bullet F\}$

$\text{GOTO}(I_0, S) = \{S' \rightarrow S \bullet\} = I_1$

$\text{GOTO}(I_0, F) = \{S \rightarrow F \bullet=L, L \rightarrow F \bullet\} = I_2$

$\text{GOTO}(I_0, L) = \{S \rightarrow L \bullet\} = I_3$

$\text{GOTO}(I_0, i) = \{F \rightarrow i \bullet\} = I_4$

$\text{GOTO}(I_0, () = \{F \rightarrow (\bullet *L)\} = I_5$

$\text{GOTO}(I_2, =) = \{S \rightarrow F= \bullet L, L \rightarrow \bullet F, F \rightarrow \bullet (*L), F \rightarrow \bullet i\} = I_6$

$\text{GOTO}(I_5, *) = \{F \rightarrow (* \bullet L), L \rightarrow \bullet F, F \rightarrow \bullet (*L), F \rightarrow \bullet i\} = I_7$

$\text{GOTO}(I_6, L) = \{S \rightarrow F=L \bullet\} = I_8$

$\text{GOTO}(I_6, F) = \{L \rightarrow F \bullet\} = I_9$

$\text{GOTO}(I_6, () = \{F \rightarrow (\bullet *L)\} = I_5$

$\text{GOTO}(I_6, i) = \{F \rightarrow i \bullet\} = I_4$

$\text{GOTO}(I_7, L) = \{F \rightarrow (*L \bullet)\} = I_{10}$

$\text{GOTO}(I_7, F) = \{L \rightarrow F \bullet\} = I_9$

$\text{GOTO}(I_7, () = \{F \rightarrow (\bullet *L)\} = I_5$

$\text{GOTO}(I_7, i) = \{F \rightarrow i \bullet\} = I_4$

$\text{GOTO}(I_{10},) = \{F \rightarrow (*L) \bullet\} = I_{11}$

Каноническая форма множества ситуаций

Построение канонической системы множеств $LR(0)$ – ситуаций:

- 1 $\varphi = \emptyset$
2. Включить в φ множество $I_0 = \text{CLOSURE}([S' \rightarrow \bullet S])$, которое в начале «не отмечено».
3. Если множество ситуаций I , входящее в систему, «не отмечено», то:
 - отметить множество I ;
 - вычислить для каждого символа $X \in (V \cup \Sigma)$ значение $I' = \text{GOTO}(I, X)$;
 - если множество $I' \neq \emptyset$ и еще не включено в φ , то включить его в систему множеств как «неотмеченное» множество.
4. Повторять шаг 3, пока все множества ситуаций системы φ не будут отмечены.

В начале построения система множеств $\varphi = \emptyset$.

Определив множество $I_0 = \text{CLOSURE}([S' \rightarrow \bullet S]) = \{S' \rightarrow \bullet S, S \rightarrow \bullet F=L, S \rightarrow \bullet L, F \rightarrow \bullet (*L), F \rightarrow \bullet i, L \rightarrow \bullet F\}$, включим его в систему φ в качестве «неотмеченного» множества.

$\varphi = \{ I_0 = \{S' \rightarrow \bullet S, S \rightarrow \bullet F=L, S \rightarrow \bullet L, F \rightarrow \bullet (*L), F \rightarrow \bullet i, L \rightarrow \bullet F\} \}$

Отметим множество I_0 и определим множества $\text{GOTO}(I_0, X)$ для всех символов X грамматики G_0 :

$I_1 = \text{GOTO}(I_0, S) = \text{CLOSURE}([S' \rightarrow S \bullet]) = \{S' \rightarrow S \bullet\}$

Множество I_1 непустое и отсутствует в системе φ . Включим I_1 в φ как «неотмеченное» множество.

$\varphi = \{I_0 = \{S' \rightarrow \bullet S, S \rightarrow \bullet F=L, S \rightarrow \bullet L, F \rightarrow \bullet (*L), F \rightarrow \bullet i, L \rightarrow \bullet F\},$

$I_1 = \{S' \rightarrow S \bullet\}\}$

Продолжая строить $GOTO(I_0, X)$, получаем:

$$I_2 = GOTO(I_0, F) = CLOSURE([S \rightarrow \bullet F=L]) = \{ S \rightarrow F\bullet=L, L \rightarrow F\bullet \}$$

$$I_3 = GOTO(I_0, L) = \{ S \rightarrow L\bullet \}$$

$$I_4 = GOTO(I_0, i) = \{ F \rightarrow i\bullet \}$$

$$I_5 = GOTO(I_0, () = \{ F \rightarrow (\bullet *L) \}$$

Остальные множества $GOTO(I_0, X)$, где $X \in \{S, *,), =\}$, пусты, поэтому система φ принимает вид:

$$\varphi = \{ I_0 = \{ S' \rightarrow \bullet S, S \rightarrow \bullet F=L, S \rightarrow \bullet L, F \rightarrow \bullet (*L), F \rightarrow \bullet i, L \rightarrow \bullet F \},$$

$$I_1 = \{ S' \rightarrow S\bullet \}$$

$$I_2 = GOTO(I_0, F) = CLOSURE([S \rightarrow \bullet F=L]) = \{ S \rightarrow F\bullet=L, L \rightarrow F\bullet \}$$

$$I_3 = GOTO(I_0, L) = \{ S \rightarrow L\bullet \}$$

$$I_4 = GOTO(I_0, i) = \{ F \rightarrow i\bullet \}$$

$$I_5 = GOTO(I_0, () = \{ F \rightarrow (\bullet *L) \}$$

}

Продолжая выполнять шаг 3 построения системы множеств, «отмечаем» множество I_1 . Для всех $X \in (V \cup \Sigma)$ множества $GOTO(I_1, X)$ пусты.

Теперь отмечаем множество I_2 . $GOTO(I_2, =) = \{ S \rightarrow F=\bullet L, L \rightarrow \bullet F, F \rightarrow \bullet (*L), F \rightarrow \bullet i \} = I_6$

отмечаем множество I_5 . $GOTO(I_5, *) = \{ F \rightarrow (\bullet *L), L \rightarrow \bullet F, F \rightarrow \bullet (*L), F \rightarrow \bullet i \} = I_7$

Добавляем множество I_6 в φ .

$$GOTO(I_6, L) = \{ S \rightarrow F=L\bullet \} = I_8$$

$$GOTO(I_6, F) = \{ L \rightarrow F\bullet \} = I_9$$

$$GOTO(I_6, () = \{ F \rightarrow (\bullet *L) \} = I_5$$

$$GOTO(I_6, i) = \{ F \rightarrow i\bullet \} = I_4$$

Добавляем в φ только те множества, которые в φ отсутствуют. Продолжая аналогичным образом, включаем в систему φ множества:

$$GOTO(I_7, L) = \{ F \rightarrow (*L\bullet) \} = I_{10}$$

$$GOTO(I_{10},) = \{ F \rightarrow (*L)\bullet \} = I_{11}$$

В результате получаем окончательную систему $LR(0)$ – множеств:

$\varphi = \{$

$$I_0 = \{ S' \rightarrow \bullet S, S \rightarrow \bullet F=L, S \rightarrow \bullet L, F \rightarrow \bullet (*L), F \rightarrow \bullet i, L \rightarrow \bullet F \},$$

$$I_1 = \{ S' \rightarrow S\bullet \},$$

$$I_2 = \{ S \rightarrow F\bullet=L, L \rightarrow F\bullet \},$$

$$I_3 = \{ S \rightarrow L\bullet \},$$

$$I_4 = \{ F \rightarrow i\bullet \},$$

$$I_5 = \{ F \rightarrow (\bullet *L) \},$$

$$I_6 = \{ S \rightarrow F=\bullet L, L \rightarrow \bullet F, F \rightarrow \bullet (*L), F \rightarrow \bullet i \},$$

$$I_7 = \{ F \rightarrow (\bullet *L), L \rightarrow \bullet F, F \rightarrow \bullet (*L), F \rightarrow \bullet i \},$$

$$I_8 = \{ S \rightarrow F=L\bullet \},$$

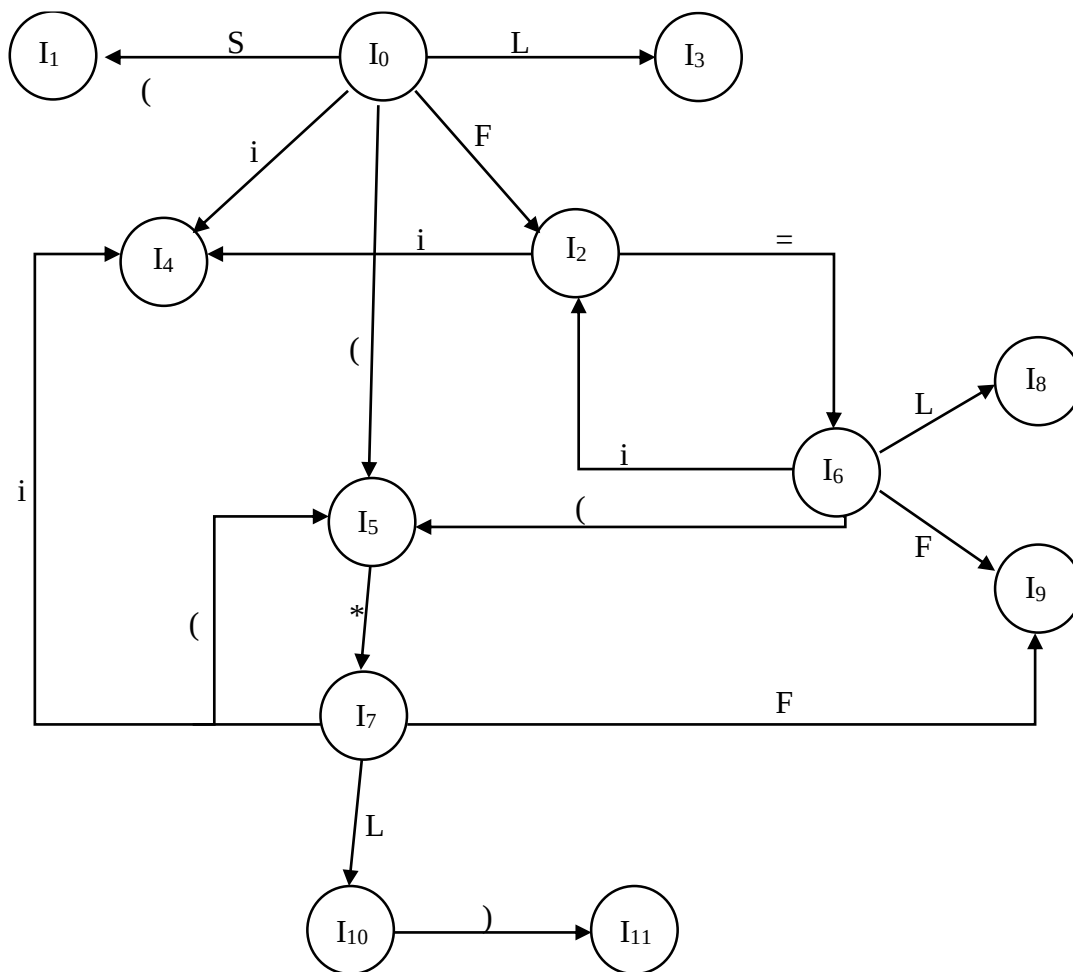
$$I_9 = \{ L \rightarrow F\bullet \},$$

$$I_{10} = \{ F \rightarrow (*L\bullet) \},$$

$$I_{11} = \{ F \rightarrow (*L)\bullet \}$$

}

Используя каноническую систему LR(0)-множеств, можно представить функцию GOTO в виде диаграммы детерминированного конечного автомата. Диаграмма переходов ДКА для активных префиксов грамматики G_0 :



Переход в состояние, которому на диаграмме соответствует лист (вершина, не имеющая исходящих дуг) однозначно определяет операцию Свёртки (i) по правилу i с переходом в новое состояние определяемое переходом на КА левым нетерминалом правила i , все остальные переходы операцию Переноса.

Шаг 2. Построение управляющей таблицы. Алгоритм построения управляющей таблицы M для $LR(0)$ -грамматик основывается на рассмотрении пар грамматических вхождений, которые могут быть представлены соседними магазинными символами в процессе разбора допустимых цепочек.

1. Если операция $[s_m, a_i] = \text{Перенос}(s)$, синтаксический анализатор выполняет перенос в стек очередного состояния s и его конфигурация становится

$(s_0s_1... s_ms, a_{i+1}... a_n\$)$

Символ a_i хранить в стеке не нужно (он может быть восстановлен из s). Текущим входным символом становится a_{i+1} .

2. Если операция $[s_m, a_i] = \text{Свертка}(i)$ правила $p_i: A \rightarrow \beta$, синтаксический анализатор выполняет свертку в два шага и его конфигурация становится

$(s_0s_1... s_{m-r}s, a_ia_{i+1}... a_n\$)$

здесь r - длина β , а $s = \text{GOTO}[s_{m-r}, A]$.

2.1. Определяется правило для свертки i и левый нетерминал: $p_i: A \rightarrow \alpha$.

2.2. Синтаксический анализатор снимает r символов состояний с вершины стека, что переносит на вершину стека состояние s_{m-r} . При свертке текущий входной символ не изменяется. (Удаляется из верхней части магазина $|\alpha|$ символов в соответствии с правилом $C(i)$, где i - номер правила, $A \rightarrow \alpha$, $|\alpha|$ - длина правой части правила).

2.3. После чего на вершину стека помещается s , запись из $\text{GOTO}[s_{m-r}, A]$. (По правилу для свертки i и левый нетерминал: $p_i: A \rightarrow \alpha$, определяется по таблице переходов состояние, которое должно быть занесено в стек)

Последовательность символов грамматики $X_{m-r+1}...X_m$ всегда соответствует α , правой части продукции свертки.

3. Если операция $[s_m, a_i] = \text{допуск}$ синтаксический анализ завершается.

4. Если операция $[s_m, a_i] = \text{ошибка}$ синтаксический анализатор вызывает программу восстановления после ошибки.

Управляющая таблица

I	f(u)						g(x)		
	i	=	*	(	)	$\perp$	S	F	L
0	П, 4			П, 5			1	2	3
1						Д			
2		П, 6			С, 5	С, 5			
3						С, 2			
4		С, 4			С, 4	С, 4			

5			П, 7						
6	П, 4			П, 5				9	8
7	П, 4			П, 5				9	10
8						С, 1			
9					С, 5	С, 5			
10					П, 11				
11		С, 3			С, 3	С, 3			

Шаг 3. Применение алгоритма «перенос-свёртка» для разбора цепочки символов на ленте.

Работа алгоритма описывается в терминах конфигураций, представляющих собой тройки вида $(\alpha T, ax, \pi)$, где αT – цепочка магазинных символов, T – верхний символ магазина, T кодирует некий префикс цепочки (символ состояния), ax – необработанная часть входной цепочки, π – выход, построенный к настоящему моменту времени.

Рассмотрим последовательность тактов LR(k)-алгоритма при анализе входной цепочки $abcb$. В магазине МП-автомата вместе с помещенным туда символами показаны и номера строк управляющей таблицы, в формате «символ»«номер».

Распознавание цепочек:

1. $(0, (*i)) = (*i) \perp, \varepsilon) \vdash^{\Pi} (0\ 5, (*i)) = (*i) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7, (*i)) = (*i) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7\ 5, (*i)) = (*i) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7\ 5\ 7, i) = (*i) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7\ 5\ 7\ 4,)) = (*i) \perp, \varepsilon) \vdash^C (0\ 5\ 7\ 5\ 7\ 9,)) = (*i) \perp, 4) \vdash^C (0\ 5\ 7\ 5\ 7\ 10,)) = (*i) \perp, 4\ 5) \vdash^{\Pi} (0\ 5\ 7\ 5\ 7\ 10\ 11,)) = (*i) \perp, 4\ 5) \vdash^C (0\ 5\ 7\ 9,)) = (*i) \perp, 4\ 5\ 3) \vdash^C (0\ 5\ 7\ 10,)) = (*i) \perp, 4\ 5\ 3\ 5) \vdash^{\Pi} (0\ 5\ 7\ 10\ 11, = (*i) \perp, 4\ 5\ 3\ 5) \vdash^C (0\ 2, = (*i) \perp, 4\ 5\ 3\ 5\ 3) \vdash^{\Pi} (0\ 2\ 6, (*i) \perp, 4\ 5\ 3\ 5\ 3) \vdash^{\Pi} (0\ 2\ 6\ 5, (*i) \perp, 4\ 5\ 3\ 5\ 3) \vdash^{\Pi} (0\ 2\ 6\ 5\ 7, i) \perp, 4\ 5\ 3\ 5\ 3) \vdash^{\Pi} (0\ 2\ 6\ 5\ 7\ 4,) \perp, 4\ 5\ 3\ 5\ 3) \vdash^C (0\ 2\ 6\ 5\ 7\ 9,) \perp, 4\ 5\ 3\ 5\ 3\ 4) \vdash^C (0\ 2\ 6\ 5\ 7\ 10,) \perp, 4\ 5\ 3\ 5\ 3\ 4\ 5) \vdash^{\Pi} (0\ 2\ 6\ 5\ 7\ 10\ 11, \perp, 4\ 5\ 3\ 5\ 3\ 4\ 5) \vdash^C (0\ 2\ 6\ 9, \perp, 4\ 5\ 3\ 5\ 3\ 4\ 5\ 3) \vdash^C (0\ 2\ 6\ 8, \perp, 4\ 5\ 3\ 5\ 3\ 4\ 5\ 3) \vdash^C (0\ 1, \perp, 4\ 5\ 3\ 5\ 3\ 4\ 5\ 3\ 1) \vdash^D$
2. $(0, i = i \perp, \varepsilon) \vdash^{\Pi} (0\ 4, = i \perp, \varepsilon) \vdash^C (0\ 2, = i \perp, 4) \vdash^{\Pi} (0\ 2\ 6, i \perp, 4) \vdash^{\Pi} (0\ 2\ 6\ 4, \perp, 4) \vdash^C (0\ 2\ 6\ 9, \perp, 4\ 4) \vdash^C (0\ 2\ 6\ 8, \perp, 4\ 4\ 5) \vdash^C (0\ 1, \perp, 4\ 4\ 5\ 1) \vdash^D$
3. $(0, ((*i))) \perp, \varepsilon) \vdash^{\Pi} (0\ 5, ((*i))) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7, ((*i))) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7\ 5, ((*i))) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7\ 5\ 7, ((*i))) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7\ 5\ 7\ 5, (*i)) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7\ 5\ 7\ 5\ 7, i)) \perp, \varepsilon) \vdash^{\Pi} (0\ 5\ 7\ 5\ 7\ 5\ 7\ 4,))) \perp, \varepsilon) \vdash^C (0\ 5\ 7\ 5\ 7\ 5\ 7\ 9,))) \perp, 4) \vdash^C (0\ 5\ 7\ 5\ 7\ 5\ 7\ 10,))) \perp, 4\ 5) \vdash^{\Pi} (0\ 5\ 7\ 5\ 7\ 5\ 7\ 10\ 11,))) \perp, 4\ 5) \vdash^C (0\ 5\ 7\ 5\ 7\ 9,))) \perp, 4\ 5\ 3)$

$$\begin{aligned}
& \vdash^C (0\ 5\ 7\ 5\ 7\ 10,))\perp, 4\ 5\ 3\ 5) \vdash^\Pi (0\ 5\ 7\ 5\ 7\ 10\ 11,)\perp, 4\ 5\ 3\ 5) \vdash^C (0\ 5\ 7\ 9,)\perp, \\
& 4\ 5\ 3\ 5\ 3) \vdash^C (0\ 5\ 7\ 10,)\perp, 4\ 5\ 3\ 5\ 3\ 5) \vdash^\Pi (0\ 5\ 7\ 10\ 11, \perp, 4\ 5\ 3\ 5\ 3\ 5) \vdash^C (0\ 4, \\
& \perp, 4\ 5\ 3\ 5\ 3\ 5\ 3) \vdash^C (0\ 2, \perp, 4\ 5\ 3\ 5\ 3\ 5\ 3\ 4) \vdash^C (0\ 3, \perp, 4\ 5\ 3\ 5\ 3\ 5\ 3\ 4\ 5) \vdash^C (0 \\
& 1, \perp, 4\ 5\ 3\ 5\ 3\ 5\ 3\ 4\ 5\ 2) \vdash^\Delta
\end{aligned}$$

3.8. Грамматики предшествования

Существует подкласс LR(k)-грамматик, который называется грамматиками предшествования для них легко строится алгоритм типа “перенос-свертка”. При восходящих методах синтаксического анализа в текущей сентенциальной форме выполняется поиск основы α , которая в соответствии с правилом грамматики $A \rightarrow \alpha$ сворачивается к нетерминальному символу A.

Основная проблема восходящего синтаксического анализа заключается в поиске основы и нетерминального символа, к которому ее нужно приводить (сворачивать).

Принцип организации распознавателя входных цепочек языка, заданного грамматикой предшествования, основывается на том, что для каждой упорядоченной пары символов в грамматике устанавливается некоторое отношение, называемое отношением предшествования. В процессе разбора входной цепочки расширенный МП-автомат сравнивает текущий символ входной цепочки с одним из символов, находящихся на верхушке стека автомата. В процессе сравнения проверяется, какое из возможных отношений предшествования существует между этими двумя символами. В зависимости от найденного отношения выполняется либо сдвиг (перенос), либо свертка. При отсутствии отношения предшествования между символами алгоритм сигнализирует об ошибке.

Задача заключается в том, чтобы иметь возможность непротиворечивым образом *определить отношения предшествования между символами грамматики*. Если это возможно, то грамматика может быть отнесена к одному из классов грамматик предшествования.

Существует несколько видов грамматик предшествования. Они различаются по тому, какие отношения предшествования в них определены и между какими типами символов (терминальными или нетерминальными) могут быть установлены эти отношения. Кроме того, возможны незначительные модификации функционирования самого алгоритма «сдвиг-свертка» в распознавателях для таких грамматик (в основном на этапе выбора правила для выполнения свертки, когда возможны неоднозначности) [5, 6, 23, 65].

Выделяют следующие виды грамматик предшествования:

- простого предшествования;
- расширенного предшествования;
- слабого предшествования;
- смешанной стратегии предшествования;
- операторного предшествования.

Рассмотрим два наиболее простых и распространенных типа — грамматики простого и операторного предшествования.

Граматики простого предшествования

Грамматикой простого предшествования называют такую приведенную (без циклов, бесполезных и недостижимых символов и λ -правил) КС-грамматику $G(VN, VT, P, S)$, $V = VT \cup VN$, в которой:

1. Для каждой упорядоченной пары терминальных и нетерминальных символов выполняется не более чем одно из трех отношений предшествования:

- $B_i =\bullet B_j$ ($\forall B_i, B_j \in V$), если и только если $\exists$ правило $A \rightarrow xB_iB_jy \in P$, где $A \in VN$, $x, y \in V^*$;
- $B_i <\bullet B_j$ ($\forall B_i, B_j \in V$), если и только если $\exists$ правило $A \rightarrow xB_iDy \in P$ и вывод $D \Rightarrow^* S_jz$, где $A, D \in VN$, $x, y, z \in V^*$;
- $B_i \bullet> B_j$ ($\forall B_i, B_j \in V$), если и только если $\exists$ правило $A \rightarrow xCB_jy \in P$ и вывод $C \Rightarrow^* zB_i$ или $\exists$ правило $A \rightarrow xCDy \in P$ и выводы $C \Rightarrow^* zB_i$ и $D \Rightarrow^* B_jw$, где $A, C, D \in VN$, $x, y, z, w \in V^*$.

2. Различные правила в грамматике имеют разные правые части (то есть в грамматике не должно быть двух различных правил с одной и той же правой частью).

Отношения $=\bullet$, $<\bullet$ и $\bullet>$ называют отношениями предшествования для символов. Отношение предшествования единственно для каждой упорядоченной пары символов. При этом между какими-либо двумя символами может и не быть отношения предшествования — это значит, что они не могут находиться рядом ни в одном элементе разбора синтаксически правильной цепочки. Отношения предшествования зависят от порядка, в котором стоят символы, и в этом смысле их нельзя путать со знаками математических операций — они не обладают ни свойством коммутативности, ни свойством ассоциативности. Например, если известно, что $B_i \bullet> B_j$, то не обязательно выполняется $B_j <\bullet B_i$ (поэтому знаки предшествования иногда помечают специальной точкой: $=\bullet$, $<\bullet$, $\bullet>$).

Для грамматик простого предшествования известны следующие полезные свойства:

- всякая грамматика простого предшествования является однозначной;
- легко проверить, является или нет произвольная КС-грамматика грамматикой простого предшествования (для этого достаточно проверить рассмотренные выше свойства грамматик простого предшествования или воспользоваться алгоритмом построения матрицы предшествования, который рассмотрен далее).

Как и для многих других классов грамматик, для грамматик простого предшествования не существует алгоритма, который бы мог преобразовать произвольную КС-грамматику в грамматику простого предшествования (или доказать, что преобразование невозможно).

Метод предшествования основан на том факте, что отношения предшествования между двумя соседними символами распознаваемой строки соответствуют трем следующим вариантам:

- $B_i <\bullet B_{i+1}$, если символ B_{i+1} — крайний левый символ некоторой основы (это отношение между символами можно назвать «предшествует основе» или просто «предшествует»);
- $B_i \bullet> B_{i+1}$, если символ B_i — крайний правый символ некоторой основы (это отношение между символами можно назвать «следует за основой» или просто «следует»);

- $V_i = \bullet V_{i+1}$, если символы V_i и V_{i+1} принадлежат одной основе (это отношение между символами можно назвать «составляют основу»),

Исходя из этих соотношений, выполняется разбор строки для грамматики предшествования.

Суть принципа такого разбора можно пояснить на рис. 12.5. На нем изображена входная цепочка символов $\alpha\beta\gamma\delta$ в тот момент, когда выполняется свертка цепочки γ . Символ a является последним символом подцепочки α , а символ b — первым символом подцепочки β . Тогда, если в грамматике удастся установить непротиворечивые отношения предшествования, то в процессе выполнения разбора по алгоритму «сдвиг-свертка» можно всегда выполнять сдвиг до тех пор, пока между символом на верхушке стека и текущим символом входной цепочки существует отношение $<\bullet$ или $=\bullet$. А как только между этими символами будет обнаружено отношение $\bullet>$, так сразу надо выполнять свертку. Причем для выполнения свертки из стека надо выбирать все символы, связанные отношением $=\bullet$. То, что все различные правила в грамматике предшествования имеют различные правые части, гарантирует непротиворечивость выбора правила при выполнении свертки.

Таким образом, установление непротиворечивых отношений предшествования между символами грамматики в комплексе с несовпадающими правыми частями различных правил дает ответы на все вопросы, которые надо решить для организации работы алгоритма «сдвиг-свертка» без возвратов.

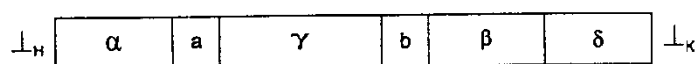


Рис. 12.5. Отношения между символами входной цепочки в грамматике предшествования

На основании отношений предшествования строят матрицу предшествования грамматики. Строки матрицы предшествования помечаются первыми (левыми) символами, столбцы — вторыми (правыми) символами отношений предшествования. В клетки матрицы на пересечении соответствующих столбца и строки помещаются знаки отношений. При этом пустые клетки матрицы говорят о том, что между данными символами нет ни одного отношения предшествования.

Для построения матрицы предшествования используются функции **LHS(X)** и **RHS(X)**.

Определим функции **LHS(X)** и **RHS(X)**:

- **LHS(X)** - задает множество крайних левых символов, с которых могут начинаться цепочки, выводимые из X :

$$\text{LHS}(X) = \{Y \mid X \Rightarrow^* Y\beta \mid Y \in (VUT), \beta \in (VUT)^*, X \in V\}$$

- **RHS(X)** - задает множество крайних правых символов, с которых могут начинаться цепочки, выводимые из X :

$$\text{RHS}(X) = \{Y \mid X \Rightarrow^* \beta Y \mid Y \in (VUT), \beta \in (VUT)^*, X \in V\}$$

Определим отношения предшествования используя функции *LHS* и *RHS*

- $X <\cdot Y \Leftrightarrow \{(X,Y) \mid A \rightarrow \alpha XZ\beta \text{ и } Y \in \text{LHS}(Z), Z \in V, X,Y \in (V \cup T)\}$
- $X \cdot> Y \Leftrightarrow \{(X,Y) \mid A \rightarrow \alpha ZY\beta \text{ и } X \in \text{RHS}(Z), Z \in V, X,Y \in (V \cup T)\}$
- $X \cdot> Y \Leftrightarrow \{(X,Y) \mid A \rightarrow \alpha ZA\beta \text{ и } X \in \text{RHS}(Z) \text{ и } Y \in \text{LHS}(A), Z, A \in V, X,Y \in (V \cup T)\}$
- $X \cdot Y \Leftrightarrow \{(X,Y) \mid A \rightarrow \alpha XY\beta, X,Y \in (V \cup T)\}$
- $\perp <\cdot X \Leftrightarrow \{(\perp_h, X) \mid X \in \text{LHS}(S_0), X \in (V \cup T)\}$
- $\$ \cdot> X \Leftrightarrow \{(\perp_k, X) \mid X \in \text{RHS}(S_0), X \in (V \cup T)\}$

Алгоритм нахождения множества, задаваемого функцией LHS(X).

Вход: $G = (V, T, P, S_0)$, X , где $X \in V$.

Выход: LHS_x - множество крайних левых символов, выводимых из X

$\text{LHS}_x := \emptyset$

foreach($X \rightarrow Y\alpha \in P \mid Y \in (V \cup T), \alpha \in (V \cup T)^*$)

$\text{LHS}_x := \text{LHS}_x \cup Y$

if($Y \in V$)

$\text{LHS}_x := \text{LHS}_x \cup \text{LHS}(Y)$

end

Алгоритм нахождения множества, задаваемого функцией RHS(X).

Вход: $G = (V, T, P, S_0)$, X , где $X \in V$.

Выход: RHS_x - множество крайних правых символов, выводимых из X

$\text{RHS}_x := \emptyset$

foreach($X \rightarrow \alpha Y \in P \mid Y \in (V \cup T), \alpha \in (V \cup T)^*$)

$\text{RHS}_x := \text{RHS}_x \cup Y$

if($Y \in V$)

$\text{RHS}_x := \text{RHS}_x \cup \text{RHS}(Y)$

end

Алгоритм построения матрицы предшествования M.

Вход: $G = (V, T, P, S_0)$.

Выход:

матрица предшествования **M** - размерами $(V \cup T \cup \perp) \times (V \cup T \cup \$)$

Шаг 1.

foreach($X \mid X \in \text{LHS}(S_0)$)

$M[\perp, X] := <\cdot$

end

foreach($X \mid X \in \text{RHS}(S_0)$)

$M[X, \$] := \cdot>$

end

Шаг 2.

foreach ($A \rightarrow \{X_1, X_2, \dots, X_k\}$)

foreach((X_{i-1}, X_i) $\in \{X_1, X_2, \dots, X_k\}$) $\forall$ Для каждой пары


```

M[Xi-1, Xi] := =·
if(Xi-1 ∈ V)
    foreach(Y ∈ RHS(Xi-1))
        M[Y, Xi] := ·>
    if(Xi ∈ V)
        foreach(X ∈ LHS(Xi))
            M[Y, X] := ·>
        end
    end
end

if(Xi ∈ V)
    foreach(Y ∈ LHS(Xi))
        M[Xi-1, Y] := <·
    end
end

end
end

```

Описывается в тройках вида $(\perp t_1 t_2 \dots t_K, a\gamma \$, \pi)$, где $t_1 t_2 \dots t_K$ - цепочка магазинных символов с верхним символом t_K , $a\gamma$ - входная цепочка с символом a под читающей головкой, π - выход.

Алгоритм. Перенос-свертка.

Вход: Анализируемая цепочка, матрица предшествования M ,
 $G = (V, T, P, S_0)$ - грамматика простого предшествования с пронумерованными правилами.

Выход: $t \in L(G)$, в противном случае – ошибка.

Шаг 1. Перенести символа $\perp$ в магазин.

Добавить $\$$ в входную цепочку.

Шаг 2.

```

\\ tK- верхний символ магазина
\\ a - обозреваемый символ
if (M[tK, a] = =· or M[tK, a] = <·)
    \\ Перенос.
    (⊥t1t2...tK, aγ$, π) ⊢ Π (⊥t1t2...tKa, γ$, π)

else if(M[tK, a] = ·>)
    \\ Свертка.
    i := k
    while(M[ti-1, ti] = =·) \\ ti-1 ti - соседние символы в магазине
        i-- \\ выделение основы ti...tK (P.S. A → ti...tK)
    end while
    if(pj : A → ti...tK ∈ P | A ∈ V)

```

$(\perp t_1 t_2 \dots t_K, a\gamma \$, \pi) \vdash^C (\perp_{n+1} t_1 t_2 \dots t_{i-1} A, a\gamma \$, \pi_j)$
 Перейти к шагу 2.

else

$(\perp t_1 t_2 \dots t_K, a\gamma \$, \pi) \vdash$ **ОШИБКА**
 Перейти к шагу 3.

else if $(t_K = S_0 \text{ and } a = \$)$

$(\perp S_0, \$, \pi) \vdash$ **ДОПУСК**
 Перейти к шагу 3.

else

$(\perp t_1 t_2 \dots t_K, a\gamma \$, \pi) \vdash$ **ОШИБКА**
 Перейти к шагу 3.

Шаг 3. Останов.

Пример.

$G = (T, V, P, S_0)$, где

$T = \{a, c, g, d, f, x, b, m\}$, $V = \{S, A, B, D, C, C', N\}$, $S_0 = S$

P:

$p_1: S \rightarrow aAb$

$p_2: A \rightarrow cDCD$

$p_3: A \rightarrow xN$

$p_4: B \rightarrow cD$

$p_5: D \rightarrow g$

$p_6: C \rightarrow dC'$

$p_7: C' \rightarrow fB$

$p_8: N \rightarrow mN$

$p_9: N \rightarrow m$

Построение матрицы предшествования

0. $LHS_S = \{a\}$ $RHS_S = \{b\}$ $M[\perp_n, a] = <\cdot$ $M[b, \perp_K] = \cdot>$	1. $S \rightarrow aAb$ $M[a, A] = =\cdot$ $M[A, b] = =\cdot$ $LHS_A = \{c, x\}$	2. $A \rightarrow cDCD$ $M[c, D] = =\cdot$ $M[D, C] = =\cdot$ $M[C, D] = =\cdot$
---	--	---

	$M[a, c] = <\cdot$ $M[a, x] = <\cdot$ $RHS_A = \{D, N, m\}$ $M[D, b] = \cdot>$ $M[N, b] = \cdot>$ $M[m, b] = \cdot>$	$LHS_D = \{g\}$ $M[C, g] = <\cdot$ $RHS_D = \{g\}$ $M[g, C] = \cdot>$ $LHS_C = \{d\}$ $M[D, d] = <\cdot$ $M[g, d] = \cdot> \setminus RHS_D$ LHS_C $RHS_C = \{C', B, D, g\}$ $M[C', D] = \cdot>$ $M[B, D] = \cdot>$ $M[D, D] = \cdot>$ $M[g, D] = \cdot>$ $M[C', g] = \cdot>$ $M[B, g] = \cdot>$ $M[D, g] = \cdot>$ $M[g, g] = \cdot>$
3. $A \rightarrow xN$ $M[x, N] = =\cdot$ $LHS_N = \{m\}$ $M[x, m] = <\cdot$	4. $B \rightarrow cD$ $M[c, D] = =\cdot$ $LHS_D = \{g\}$ $M[c, g] = <\cdot$	5. $D \rightarrow g$
6. $C \rightarrow dC'$ $M[d, C'] = =\cdot$ $LHS_{C'} = \{f\}$ $M[d, f] = <\cdot$	7. $C' \rightarrow fB$ $M[f, B] = =\cdot$ $LHS_B = \{c\}$ $M[f, c] = <\cdot$	8. $N \rightarrow mN$ $M[m, N] = =\cdot$ $LHS_N = \{m\}$ $M[m, m] = <\cdot$

	S	A	D	N	B	C	C'	a	b	c	d	f	g	x	m	\$
S																
A									=.							
D			.>			=.			.>		<.		.>			
N									.>							
B			.>										.>			
C			=.										<.			
C'			.>										.>			
a		=.								<.				<.		
b																.>
c			=.										<.			
d							=.					<.				
f					=.						<.					
g			.>			.>					.>		.>			
x				=.											<.	
m				=.					.>						<.	
⊥								<.								

$(\perp, axmmb$, ε), $\vdash^{\Pi}(\perp a, xmmb$, ε) $\vdash^{\Pi}(\perp ax, mmb$, ε) $\vdash^{\Pi}(\perp axm, mb$, ε)$
 $\vdash^{\Pi}(\perp axmm, b\perp_K, \varepsilon)$ $\vdash^C(\perp axmN, b\perp_K, 9)$ $\vdash^C(\perp axN, b$, $98)$ $\vdash^C(\perp aA, b$, $983)$ $\vdash^{\Pi}(\perp aAb, $, $983)$ $\vdash^C(\perp S, $, $9831)$ $\vdash$ ДОПУСК$$$$$$$

Грамматики операторного предшествования

Операторной грамматикой называется КС-грамматика без λ -правил, в которой правые части всех правил не содержат смежных нетерминальных символов. Для операторной грамматики отношения предшествования можно задать на множестве терминальных символов (включая символы $\perp_n$ и $\perp_k$).

Грамматикой операторного предшествования называется операторная КС-грамматика $G(VN, VT, P, S)$, $V = VT \cup VN$, для которой выполняются следующие условия:

1. Для каждой упорядоченной пары терминальных символов выполняется не более чем одно из трех отношений предшествования:

- $a = \bullet b$, если и только если существует правило $A \rightarrow xaby \in P$ или правило $A \rightarrow xACby$, где $a, b \in VT$, $A, C \in VN$, $x, y \in V^*$;
- $a < \bullet b$, если и только если существует правило $A \rightarrow xACy \in P$ и вывод $C \Rightarrow^* bz$ или вывод $C \Rightarrow^* Dbz$, где $a, b \in VT$, $A, C, D \in VN$, $x, y, z \in V^*$;
- $a \bullet > b$, если и только если существует правило $A \rightarrow xCby \in P$ и вывод $C \Rightarrow^* za$ или вывод $C \Rightarrow^* zaD$, где $a, b \in VT$, $A, C, D \in VN$, $x, y, z \in V^*$.

2. Различные порождающие правила имеют разные правые части, λ -правила отсутствуют.

Отношения предшествования для грамматик операторного предшествования определены таким образом, что для них выполняется еще одна особенность — правила грамматики операторного предшествования не могут содержать двух смежных нетерминальных символов в правой части. То есть в грамматике операторного предшествования $G(VN, VT, P, S)$, $V = VT \cup VN$ не может быть ни одного правила вида: $A \rightarrow xBCy$, где $A, B, C \in VN$, $x, y \in V^*$ (здесь x и y — это произвольные цепочки символов, могут быть и пустыми).

Для грамматик операторного предшествования также известны следующие свойства:

- всякая грамматика операторного предшествования задает детерминированный КС-язык (но не всякая грамматика операторного предшествования при этом является однозначной!);
- легко проверить, является или нет произвольная КС-грамматика грамматикой операторного предшествования (точно так же, как и для простого предшествования).

Как и для многих других классов грамматик, для грамматик операторного предшествования не существует алгоритма, который бы мог преобразовать произвольную КС-грамматику в грамматику операторного предшествования (или доказать, что преобразование невозможно).

Принцип работы распознавателя для грамматики операторного предшествования аналогичен грамматике простого предшествования, но

отношения предшествования проверяются в процессе разбора только между терминальными символами.

Для построения матрицы операторного предшествования используются функции $LHS^t(X)$ и $RHS^t(X)$.

Определим функции $LHS^t(X)$ и $RHS^t(X)$:

- $LHS^t(X)$ - задает множество крайних левых терминальных символов относительно нетерминального символа X :

$$LHS^t(X) = \{t \mid X \Rightarrow^* t\beta \text{ или } X \Rightarrow^* Yt\beta \mid t \in \mathbf{T}, X, Y \in \mathbf{V}, \beta \in (\mathbf{V} \cup \mathbf{T})^*\}$$

- $RHS^t(X)$ - задает множество крайних правых терминальных символов относительно нетерминального символа X :

$$RHS^t(X) = \{t \mid X \Rightarrow^* \beta t \text{ или } X \Rightarrow^* \beta tY \mid t \in \mathbf{T}, X, Y \in \mathbf{V}, \beta \in (\mathbf{V} \cup \mathbf{T})^*\}$$

Определим отношения предшествования используя функции LHS^t и RHS^t

- $a \prec b \Leftrightarrow \{(a, b) \mid A \rightarrow \alpha aX\beta \text{ и } b \in LHS^t(X), a, b \in \mathbf{T}, X \in \mathbf{V}, \alpha, \beta \in (\mathbf{V} \cup \mathbf{T})^*\}$
- $a \succ b \Leftrightarrow \{(a, b) \mid A \rightarrow \alpha Xb\beta \text{ и } b \in RHS^t(X), a, b \in \mathbf{T}, X \in \mathbf{V}, \alpha, \beta \in (\mathbf{V} \cup \mathbf{T})^*\}$
- $a = \cdot b \Leftrightarrow \{(a, b) \mid A \rightarrow \alpha ab\beta \text{ или } A \rightarrow \alpha aXb\beta, a, b \in \mathbf{T}, X \in \mathbf{V}, \alpha, \beta \in (\mathbf{V} \cup \mathbf{T})^*\}$
- $\perp \prec \cdot a \Leftrightarrow \{(\perp, a) \mid a \in LHS^t(S_0), a \in \mathbf{T}, S_0 \in \mathbf{V}\}$
- $\$ \succ \cdot a \Leftrightarrow \{(\$, a) \mid a \in RHS^t(S_0), a \in \mathbf{T}, S_0 \in \mathbf{V}\}$

Алгоритм нахождения множества, задаваемого функцией $LHS^t(X)$.

Вход: $G = (V, T, P, S_0)$, $LHS(X)$.

Выход: $LHS^t(X)$ - множество крайних левых терминальных символов относительно нетерминального символа X

foreach($X \in V$)

$$LHS_0^t(X) = \{t \mid X \rightarrow t\beta \text{ или } X \rightarrow Yt\beta \mid t \in \mathbf{T}, X, Y \in \mathbf{V}, \beta \in (\mathbf{V} \cup \mathbf{T})^*\}$$

end

$i = 0$

foreach($X \in V$)

do:

$$i = i + 1$$

$$LHS_i^t(X) = LHS_{i-1}^t(X)$$

foreach($Y \in (LHS(X) \cap V)$)

$$LHS_i^t(X) = LHS_{i-1}^t(X) \cup LHS_{i-1}^t(Y)$$

end

while ($LHS_i^t(X) \neq LHS_{i-1}^t(X)$)

$$LHS^t(X) = LHS_i^t(X)$$

end

Алгоритм нахождения множества, задаваемого функцией $RHS^t(X)$.

Вход: $G = (V, T, P, S_0)$, $RHS(X)$.

Выход: $RHS^t(X)$ - множество крайних правых терминальных символов относительно нетерминального символа X

foreach($X \in V$)

$RHS^t_0(X) = \{t \mid X \rightarrow \beta t \text{ или } X \rightarrow \beta t Y \mid t \in T, X, Y \in V, \beta \in (V \cup T)^*\}$

end

$i = 0$

foreach($X \in V$)

do:

$i = i + 1$

$RHS^t_i(X) = RHS^t_{i-1}(X)$

foreach($Y \in (RHS(X) \cap V)$)

$RHS^t_i(X) = RHS^t_{i-1}(X) \cup RHS^t_{i-1}(Y)$

end

while ($RHS^t_i(X) \neq RHS^t_{i-1}(X)$)

$RHS^t(X) = RHS^t_i(X)$

end

Алгоритм построения матрицы предшествования M .

Вход: $G = (V, T, P, S_0)$.

Выход:

матрица предшествования M - размерами $(T \cup \perp) \times (T \cup \$)$

Шаг 1.

foreach($X \mid X \in LHS^t(S_0)$)

$M[\perp, X] = \prec$

end

foreach($X \mid X \in RHS^t(S_0)$)

$M[X, \$] = \succ$

end

Шаг 2.

foreach ($A \rightarrow \{X_1, X_2, \dots, X_k\}$)

foreach(($X_{i-1}, X_i \in \{X_1, X_2, \dots, X_k\}$) $\setminus$ Для каждой пары

if ($X_{i-1}, X_i \in T$)

$M[X_{i-1}, X_i] = =$

if($X_{i-1} \in V, X_i \in T$)

foreach($b \in RHS^t(X_{i-1})$)

$M[b, X_i] = \succ$

end

if($X_{i-1} \in T, X_i \in V$)

foreach($b \in LHS^t(X_i)$)

```

                                 $M[X_{i-1}, b] = <$ 
                                end
                        end
                foreach  $((X_{i-1}, X_i, X_{i+1}) \in \{X_1, X_2, \dots, X_k\})$   $\forall$  Для каждой тройки
                        if  $(X_{i-1}, X_{i+1} \in T, X_i \in V)$ 
                                 $M[X_{i-1}, X_i] = =$ 
                        end
                end
end

```

Описывается в тройках вида $(\perp t_1 t_2 \dots t_k, a\gamma \$, \pi)$, где $t_1 t_2 \dots t_k$ - цепочка магазинных символов с верхним символом t_k , $a\gamma$ - входная цепочка с символом a под читающей головкой, π - выход.

Алгоритм «перенос-свертка» для грамматики операторного предшествования

Алгоритм выполняется расширенным МП-автоматом и имеет те же условия завершения и обнаружения ошибок. Основное отличие состоит в том, что при определении отношения предшествования этот алгоритм не принимает во внимание находящиеся в стеке нетерминальные символы и при сравнении ищет ближайший к верхушке стека терминальный символ. Однако после выполнения сравнения и определения границ основы при поиске правила в грамматике нетерминальные символы следует, безусловно, принимать во внимание.

Алгоритм состоит из следующих шагов.

Шаг 1. Поместить в верхушку стека символ $\perp_H$, считывающую головку - в начало входной цепочки символов.

Шаг 2. Сравнить с помощью отношения предшествования терминальный символ, ближайший к вершине стека (левый символ отношения), с текущим символом входной цепочки, обозреваемым считывающей головкой (правый символ отношения). При этом из стека надо выбрать самый верхний терминальный символ, игнорируя все возможные нетерминальные символы.

Шаг 3. Если имеет место отношение $<\bullet$ или $=\bullet$, то произвести сдвиг (перенос текущего символа из входной цепочки в стек и сдвиг считывающей головки на один шаг вправо) и вернуться к шагу 2. Иначе перейти к шагу 4.

Шаг 4. Если имеет место отношение $\bullet>$, то произвести свертку. Для этого надо найти на вершине стека все терминальные символы, связанные отношением $=\bullet$ («основу»), а также все соседствующие с ними нетерминальные символы (при определении отношения нетерминальные символы игнорируются). Если терминальных символов, связанных отношением $=\bullet$, на верхушке стека нет, то в качестве основы используется один, самый верхний в стеке терминальный символ стека. Все (и терминальные, и нетерминальные) символы, составляющие основу, надо удалить из стека, а затем выбрать из грамматики правило, имеющее правую часть, совпадающую с основой, и поместить в стек левую часть выбранного правила. Если правило, совпадающее с основой, найти не удалось, то необходимо прервать выполнение алгоритма и сообщить об ошибке, иначе, если разбор не закончен, то вернуться к шагу 2.

Шаг 5. Если не установлено ни одно отношение предшествования между текущим символом входной цепочки и самым верхним терминальным символом в стеке, то надо прервать выполнение алгоритма и сообщить об ошибке.

Алгоритм. Перенос-свертка.

Вход: Анализируемая цепочка γ , матрица предшествования $\mathbf{M}$,
 $G = (V, T, P, S_0)$ - грамматика операторного предшествования с пронумерованными правилами.

Выход: $\gamma \in L(G)$, в противном случае – ошибка.

Шаг 1. Перенести символа $\perp$ в магазин. Добавить $\$$ в конец входной цепочки. Заменить все нетерминальные символы на один.

Шаг 2. $(\perp t_1 t_2 \dots t_k, a\gamma\$, \pi)$ – \textit{конфигурация в начале шага 2}

\textit{// } t_k - верхний символ магазина, a - обозреваемый символ цепочки

$k = x$

foreach $(t_i \in \{t_k \dots t_2 t_1 \perp\} \text{ и } t_i \in V)$ \textit{// идем по стеку, ищем индекс первого терминала}

$k = i$

end

\textit{// наша начальная конфигурация примет вид: } $(\perp t_1 t_2 \dots t_k \alpha, a\gamma\$, \pi)$ где, t_k - верхний терминальный символ магазина, $\alpha = t_k \dots t_x$ - цепочка нетерминальных символов, которые могут находиться сверху в магазине

if $(M[t_k, a] = = \cdot \text{ or } M[t_k, a] = < \cdot)$ \textit{// Перенос.}

$(\perp t_1 t_2 \dots t_k \alpha, a\gamma\$, \pi) \vdash^\Pi (\perp t_1 t_2 \dots t_k \alpha a, \gamma\$, \pi)$

else if $(M[t_k, a] = \cdot >)$ \textit{// Свертка.}

$i := k - 1$

foreach $(t_p \in \{t_k \dots t_2 t_1 \perp\} \text{ и } t_p \in V)$ \textit{// идем по стеку, ищем индекс следующего терминала}

$i = p$

end

if $(M[t_i, t_k] = = \cdot)$

$p = k$

while $(M[t_i, t_p] = = \cdot \text{ или } t_i \in V)$ \textit{// } $t_i t_p$ - соседние терминальные символы в магазине (игнорируя нетерминальные)

$p = i$

$i--$ \textit{// выделение основы } $t_i \dots t_k$ (P.S. $A \rightarrow t_i \dots t_k \alpha$)

end while

else

$i = k$

if $(t_{i-1} \in V)$ $i--$

if $(p_m : A \rightarrow t_i \dots t_k \alpha \in P \mid A \in V)$

$(\perp t_1 t_2 \dots t_k \alpha, a\gamma\$, \pi) \vdash^C (\perp t_1 t_2 \dots t_{i-1} A, a\gamma\$, \pi m)$

Перейти к шагу 2.

else

$(\perp t_1 t_2 \dots t_k \alpha, a\gamma\$, \pi) \vdash$ **ОШИБКА**

Перейти к шагу 3.

else if ($\alpha = S_0$ and $a = \$$)
 $(\perp S_0, \$, \pi) \vdash \text{ДОПУСК}$
 Перейти к шагу 3.

else
 $(\perp t_1 t_2 \dots t_k \alpha, a \gamma \$, \pi) \vdash \text{ОШИБКА}$
 Перейти к шагу 3.

Шаг 3. Останов.

Пример. $G = (\mathbf{T}, \mathbf{V}, \mathbf{P}, S_0)$, где $\mathbf{T} = \{ (,), \wedge, \&, \sim, a \}$, $\mathbf{V} = \{ S, T, E, F \}$, $S_0 = S$; $\mathbf{P} = \{ p_1: S \rightarrow S \wedge T; p_2: S \rightarrow T; p_3: T \rightarrow T \& E; p_4: T \rightarrow E; p_5: E \rightarrow \sim E; p_6: E \rightarrow F; p_7: F \rightarrow (E); p_8: F \rightarrow a \}$

Построение множеств $LHS^t(X)$ и $RHS^t(X)$

$LHS^t_0(S) = \{ \wedge \}$	$RHS^t_0(S) = \{ \wedge \}$
$LHS^t_0(T) = \{ \& \}$	$RHS^t_0(T) = \{ \& \}$
$LHS^t_0(E) = \{ \sim \}$	$RHS^t_0(E) = \{ \sim \}$
$LHS^t_0(F) = \{ (, a \}$	$RHS^t_0(F) = \{), a \}$
$LHS^t_1(S) = \{ \wedge, \&, \sim, (, a \}$	$RHS^t_1(S) = \{ \wedge, \&, \sim,), a \}$
$LHS^t_1(T) = \{ \&, \sim, (, a \}$	$RHS^t_1(T) = \{ \&, \sim,), a \}$
$LHS^t_1(E) = \{ \sim, (, a \}$	$RHS^t_1(E) = \{ \sim,), a \}$
$LHS^t_1(F) = \{ (, a \}$	$RHS^t_1(F) = \{), a \}$
$LHS^t_2(S) = \{ \wedge, \&, \sim, (, a \}$	$RHS^t_2(S) = \{ \wedge, \&, \sim,), a \}$
$LHS^t_2(T) = \{ \&, \sim, (, a \}$	$RHS^t_2(T) = \{ \&, \sim,), a \}$
$LHS^t_2(E) = \{ \sim, (, a \}$	$RHS^t_2(E) = \{ \sim,), a \}$
$LHS^t_2(F) = \{ (, a \}$	$RHS^t_2(F) = \{), a \}$

$LHS^t(X)$: $LHS^t(S) = \{ \wedge, \&, \sim, (, a \}$, $LHS^t(T) = \{ \&, \sim, (, a \}$, $LHS^t(E) = \{ \sim, (, a \}$, $LHS^t(F) = \{ (, a \}$.

$RHS^t(X)$: $RHS^t(S) = \{ \wedge, \&, \sim,), a \}$, $RHS^t(T) = \{ \&, \sim,), a \}$, $RHS^t(E) = \{ \sim,), a \}$, $RHS^t(F) = \{), a \}$.

Построение матрицы операторного предшествования:

0. $\perp, \$$	
$LHS^t(S) = \{ \wedge, \&, \sim, (, a \}$	$RHS^t(S) = \{ \wedge, \&, \sim,), a \}$
$M[\perp, \wedge] = <\cdot$	$M[\wedge, \$] = \cdot>$
$M[\perp, \&] = <\cdot$	$M[\&, \$] = \cdot>$
$M[\perp, \sim] = <\cdot$	$M[\sim, \$] = \cdot>$
$M[\perp, (] = <\cdot$	$M[(, \$] = \cdot>$
$M[\perp, a] = <\cdot$	$M[a, \$] = \cdot>$
1. $S \rightarrow S \wedge T$	
$LHS^t(T) = \{ \&, \sim, (, a \}$	$RHS^t(S) = \{ \wedge, \&, \sim,), a \}$

$$M[\wedge, \&] = <\cdot$$

$$M[\wedge, \sim] = <\cdot$$

$$M[\wedge, (] = <\cdot$$

$$M[\wedge, a] = <\cdot$$

$$M[\wedge, \wedge] = \cdot>$$

$$M[\&, \wedge] = \cdot>$$

$$M[\sim, \wedge] = \cdot>$$

$$M[), \wedge] = \cdot>$$

$$M[a, \wedge] = \cdot>$$

2. $S \rightarrow T$ //нет терминалов, в правиле только 1 символ

3. $T \rightarrow T\&E$

$$LHS^t(E) = \{\sim, (, a\}$$

$$M[\&, \sim] = <\cdot$$

$$M[\&, (] = <\cdot$$

$$M[\&, a] = <\cdot$$

$$RHS^t(T) = \{\&, \sim,), a\}$$

$$M[\&, \&] = \cdot>$$

$$M[\sim, \&] = \cdot>$$

$$M[), \&] = \cdot>$$

$$M[a, \&] = \cdot>$$

4. $T \rightarrow E$ //нет терминалов, в правиле только 1 символ

5. $E \rightarrow \sim E$

$$LHS^t(E) = \{\sim, (, a\}$$

$$M[\sim, \sim] = <\cdot$$

$$M[\sim, (] = <\cdot$$

$$M[\sim, a] = <\cdot$$

6. $E \rightarrow F$ //нет терминалов, в правиле только 1 символ

7. $F \rightarrow (E)$

$$LHS^t(E) = \{\sim, (, a\}$$

$$M[(, \sim] = <\cdot$$

$$M[(, (] = <\cdot$$

$$M[(, a] = <\cdot$$

$$RHS^t(E) = \{\sim,), a\}$$

$$M[\sim,)] = \cdot>$$

$$M[),)] = \cdot>$$

$$M[a,)] = \cdot>$$

$$M[(,)] = =\cdot$$

8. $F \rightarrow a$ //в правиле только 1 символ

Матрица операторного предшествования символов грамматики:

	$\wedge$	$\&$	$\sim$	$($	$)$	a	$\$$
$\wedge$	$\cdot>$	$<\cdot$	$<\cdot$	$<\cdot$		$<\cdot$	$\cdot>$
$\&$	$\cdot>$	$\cdot>$	$<\cdot$	$<\cdot$		$<\cdot$	$\cdot>$
$\sim$	$\cdot>$	$\cdot>$	$<\cdot$	$<\cdot$	$\cdot>$	$<\cdot$	$\cdot>$
$($			$<\cdot$	$<\cdot$	$=\cdot$	$<\cdot$	
$)$	$\cdot>$	$\cdot>$			$\cdot>$		$\cdot>$
a	$\cdot>$	$\cdot>$			$\cdot>$		$\cdot>$
$\perp$	$<\cdot$	$<\cdot$	$<\cdot$	$<\cdot$		$<\cdot$	

Распознавание входной цепочки символов:

Алгоритм разбора цепочек грамматики операторного предшествования игнорирует нетерминальные символы. Поэтому имеет смысл преобразовать исходную грамматику таким образом, чтобы оставить в ней только один нетерминальный символ. Тогда получим следующий вид правил:

$$P = \{p_1: S \rightarrow S^{\wedge}S; p_2: S \rightarrow S; p_3: S \rightarrow S \& S; p_4: S \rightarrow S; p_5: S \rightarrow \sim S; p_6: S \rightarrow S; p_7: S \rightarrow (S); p_8: S \rightarrow a\}$$

Это преобразование не ведет к созданию эквивалентной грамматики и выполняется только после построения всех множеств и матрицы предшествования. Само преобразование выполняется только с целью более эффективного выполнения алгоритма разбора, в который уже заложены необходимые данные о порядке применения правил при создании матрицы предшествования. Поэтому, в отличие от грамматик простого предшествования, при разборе выходная цепочка не содержит правил, которые не содержат терминальных символов или может содержать альтернативу, которая появилась после замены всех нетерминальных символов одним символом.

Распознавание цепочки $\sim(a)$:

$$\begin{aligned} (\perp, \sim(a)\$, \epsilon) \vdash^{\Pi} (\perp \sim, (a)\$, \epsilon) \vdash^{\Pi} (\perp \sim(, a)\$, \epsilon) \vdash^{\Pi} (\perp \sim(a,)\$, \epsilon) \vdash^{C,8} (\perp \sim(S,)\$, 8) \\ \vdash^{\Pi} (\perp \sim(S), \$, 8) \vdash^{C,7} (\perp \sim S, \$, 87) \vdash^{C,5} (\perp S, \$, 875) \vdash \text{ДОПУСК} \end{aligned}$$

Для всех классов грамматик предшествования общим является способ поиска правого конца основы (хвоста основы). При разборе алгоритмом “перенос- свертка”, всякий раз, когда между верхним символом магазина и первым из необработанных входных символов выполняется отношение $\bullet>$, принимается решение о свертке, в противном случае делается перенос.

Таким образом, с помощью отношения $\bullet>$ локализуется правый конец основы правовыводимой цепочки. Определение левого конца основы (головой основы) и нужной свертки осуществляется в зависимости от используемого типа отношения предшествования. см. Алгоритм "Перенос-Свертка". В литературе описано несколько вариантов грамматик предшествования.

Глава 4. Грамматики общего вида и машины Тьюринга

4.1. Грамматики общего вида и машина Тьюринга

Определение 15. Грамматика $G = (T, V, P, S)$ называется *грамматикой типа 0*, если на правила вывода не накладывается никаких ограничений (кроме тех, которые указаны в определении грамматики).

Граматики типа 0 - это самые общие грамматики.

Языки высокого уровня не являются рекурсивными. Проблема выявления принадлежности выражения w языку, порожденному грамматикой типа 0, где $w \in T^*$, в общем случае неразрешима.

Определение 16. Машина Тьюринга (служит для проверки разрешимости алгоритма), в иерархии Хомского занимает самый верхний уровень:

$M = (Q, \Sigma, R, L, B, \delta, q_0,)$

Q - множество состояний

Σ - входной алфавит

R, L - правые и левые символы, не входящие в $Q \cup \Sigma$, B - пробел ($B \in \Sigma$)

δ - множество правил **переписывания**

q_0 - исходное состояние

$\delta = q_i a_i \rightarrow q_k a_l, a_i$ - входные символы $q_i a_i \rightarrow q_k \{B, L, R\}$,

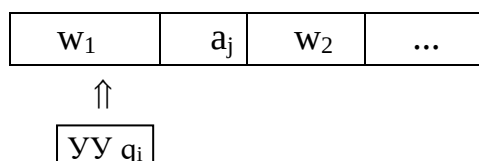
$\delta = Q \times \Sigma \text{ в } Q \times (\Sigma \cup \{B, L, R\}) \quad q_i a_j \rightarrow q_k \{B, L, R\}$

Машина Тьюринга - детерминированный преобразователь. Конфигурация машины - множество элементов следующего вида:

$\Sigma^* \times Q \times \Sigma^+$, то есть последовательности $w_1 q_1 a_j w_2$, где w_1 и $w_2 \in \Sigma^*$, $a_j \in \Sigma$, $q_i \in Q$.

Правила из δ задают последовательные переходы от одной конфигурации (до тех пор, пока не будет достигнута заключительная конфигурация).

Конфигурация $w_1 q_1 a_j w_2$ называется **заключительной**, если к ней неприменимо ни одно из правил δ (т.е. правил с заголовком $q_1 a_j$).



Работа машины сводится к следующему, управляющее устройство (УУ):

1. Читает содержимое ячейки, обозреваемой головкой в текущий момент времени.

2. Пишет в обозреваемую ячейку соответствующий символ из входного словаря.

3. Перемещается в соседнюю ячейку, находящуюся слева или справа от обозреваемой ячейки.

Находясь в q_i читает символ a_i и переходит в состояние q_k и либо печатает в этой ячейке символ a_j , либо перемещает головку влево или вправо.

Если к каждой цепочке применить правило, то получится новая конфигурация. Воспринимаемый машиной Тьюринга язык $L = \{w \in \Sigma^* \mid q_0 \text{ и } w \Rightarrow^* C_f, \text{ где } C_f - \text{заключительная конфигурация.}\}$

Если w - цепочка языка, то машина за конечный период времени находит заключительную конфигурацию C_f . Если же $w \notin L$, то машина не

останавливается, что приводит к неразрешимости проблемы распознавания принадлежности данного выражения языку.

Машины Тьюринга с операционными символами

Определение 1. Машина Тьюринга (служит для проверки разрешимости алгоритма), в иерархии Хомского занимает самый верхний уровень:

$M = (Q, \Sigma, R, L, B, \delta, D, q_0)$

Q - множество состояний

Σ - входной алфавит

R, L - правые и левые символы, не входящие в $Q \cup \Sigma$, B - пробел ($B \in \Sigma$)

δ - множество правил переписывания

q_0 - исходное состояние

$\delta = (q_i, a_i, D) \rightarrow (q_k)$,

a_i - входные символы

D - отношение "операционный символ" - "операция"

Машина Тьюринга - детерминированный преобразователь. Правила из δ задают последовательные переходы от одной конфигурации (до тех пор, пока не будет достигнута заключительная конфигурация). Конфигурация $q_i a_j D$ называется заключительной, если к ней неприменимо ни одно из правил δ (т.е. правил с заголовком $q_l a_j$). Работа машины сводится к следующему, управляющее устройство (УУ): 1. Читает содержимое ячейки, обозреваемой головкой в текущий момент времени. 2. Выполняет операцию по операционному символу 3. Переходит в следующее состояние. Находясь в q_i читает символ a_i и переходит в состояние q_k . Если к каждой цепочке применить правило, то получится новая конфигурация.

Псевдокод:

```
currState = q0
tape = x1x2...
position = 0
while(currState)
    foreach( $\delta(s, t, oper)$ :  $\in \delta$ )
        if ( $t = x_{pos}$ )
            tape, position = Oper(tape, position)
            currState =  $\delta(s, t, oper)$ 
        break
```

Пример:

$M = (Q, \Sigma, R, L, B, \delta, D, q_0)$

Q : I, C1, C0, F

$q_0 = I$

Σ : 1 0

$\delta_1(I, 1, D\{C,0\}) = (C1);$
 $\delta_2(I, 0, D\{C,1\}) = (C0);$
 $\delta_3(C1, 0, D\{R,\}) = (I);$
 $\delta_4(C0, 1, D\{R,\}) = (I);$
 $\delta_5("I", "", "", "",) = (F);$

}

1010->0101

Такты срабатывания:

Пример конфигурации - (State, A,D, δ)

$(I, \underline{1}010, \{C,0\}, C1) \xrightarrow{1} (C1, 0010, \{R,\}I) \xrightarrow{3} (I, 0010, \{C,1\}, C0) \xrightarrow{1} (C0, 0110, \{R,\}I)$
 $\xrightarrow{4} (I, 0110, \{C,0\}, C1) \xrightarrow{1} (C1, 0100, \{R,\}I) \xrightarrow{3} (I, 0100, \{C,1\}, C0)$
 $\xrightarrow{1} (C0, 0101, \{R,\}I) \xrightarrow{5} (I, 0101, \{\}, F)$

4.2. Контекстно-зависимые грамматики и ленточные автоматы

Определение 17. Грамматика $G = (T, V, P, S)$ называется *неукорачивающей грамматикой*, если каждое правило из P имеет вид $\alpha \rightarrow \beta$, где $\alpha \subset (T \cup V)^+$, $\beta \subset (T \cup V)^+$ и $|\alpha| \leq |\beta|$.

Определение 18. Грамматика $G = (T, V, P, S)$ называется *контекстно-зависимой (КЗ)*, если каждое правило из P имеет вид $\alpha \rightarrow \beta$, где $\alpha = \xi_1 A \xi_2$; $\beta = \xi_1 \gamma \xi_2$; $A \subset V$; $\gamma \subset (T \cup V)^+$; $\xi_1, \xi_2 \subset (T \cup V)^*$.

Граматику типа 1 можно определить как неукорачивающую либо как контекстно-зависимую.

Выбор определения не влияет на множество языков, порождаемых грамматиками этого класса, поскольку доказано, что множество языков, порождаемых неукорачивающими грамматиками, совпадает с множеством языков, порождаемых КЗ-грамматиками.

5. Соотношения между классами грамматик и языками

Отношения между классами КС-грамматик

В общем случае можно выделить правоанализируемые и левоанализируемые КС-грамматики. Об этих двух принципиально разных классах грамматик уже говорилось выше: первые предполагают построение левостороннего (восходящего) распознавателя, вторые — правостороннего (нисходящего). Это вовсе не значит, что для КС-языка, заданного, например, некоторой левоанализируемой грамматикой, невозможно построить расширенный МП-автомат, который порождает правосторонний вывод. Указанное разделение грамматик относится только к построению на их основе детерминированных МП-автоматов и детерминированных расширенных МП-автоматов. Только эти типы автоматов представляют интерес при создании компиляторов и анализе входных цепочек языков программирования. Недетерминированные автоматы,

порождающие как левосторонние, так и правосторонние выводы, можно построить в любом случае для языка заданного любой КС-грамматикой, но для создания компилятора такие автоматы интереса не представляют (см. раздел «Распознаватели КС-языков. Автоматы с магазинной памятью»).

Вспомним, что:

Утверждение 1. Существует подкласс LR(k)-грамматик, который называется грамматиками предшествования.

Утверждение 2. Грамматикой операторного предшествования называется операторная КС-грамматика $G(VN, VT, P, S)$, $V = VT \cup VN$, для которой выполняются следующие условия:

1. Для каждой упорядоченной пары терминальных символов выполняется не более чем одно из трех отношений предшествования:
 - $a = \bullet b$, если и только если существует правило $A \rightarrow xayby \in P$ или правило $A \rightarrow xACby$, где $a, b \in VT$, $A, C \in VN$, $x, y \in V^*$;
 - $a < \bullet b$, если и только если существует правило $A \rightarrow xCby \in P$ и вывод $C \Rightarrow^* bz$ или вывод $C \Rightarrow^* bzD$, где $a, b \in VT$, $A, C, D \in VN$, $x, y, z \in V^*$.
 - $a < \bullet b$, если и только если существует правило $A \rightarrow xCby \in P$ и вывод $C \Rightarrow^* za$ или вывод $C \Rightarrow^* zaD$, где $a, b \in VT$, $A, C, D \in VN$, $x, y, z \in V^*$.
2. Различные порождающие правила имеют разные правые части, λ -правила отсутствуют.

Определение 1. Грамматика называется однозначной, если для каждой цепочки символов языка, заданного этой грамматикой, можно построить единственный левосторонний (и единственный правосторонний) вывод.

На рис. 12.10 изображена условная схема, дающая представление о соотношении классов КС-грамматик и языков.

Интересно, что классы левоанализируемых и правоанализируемых грамматик являются несопоставимыми. То есть существуют левоанализируемые КС-грамматики, на основе которых нельзя построить детерминированный расширенный МП-автомат, порождающий правосторонний вывод; и наоборот — существуют правоанализируемые КС-грамматики, не допускающие построение МП-автомата, порождающего левосторонний вывод. Конечно, существуют грамматик, подпадающие под оба класса и допускающие построение детерминированных автоматов как с правосторонним, так и с левосторонним выводом.

Следует помнить также, что все упомянутые классы КС-грамматик — это счетные, но бесконечные множества. Нельзя построить и рассмотреть все возможные левоанализируемые грамматик или даже все возможные LL(1)-грамматик. Сопоставление классов КС-грамматик производится исключительно на основе анализа структуры их правил. Только на основании такого рода анализа произвольная КС-грамматика может быть отнесена в тот или иной класс (или несколько классов).

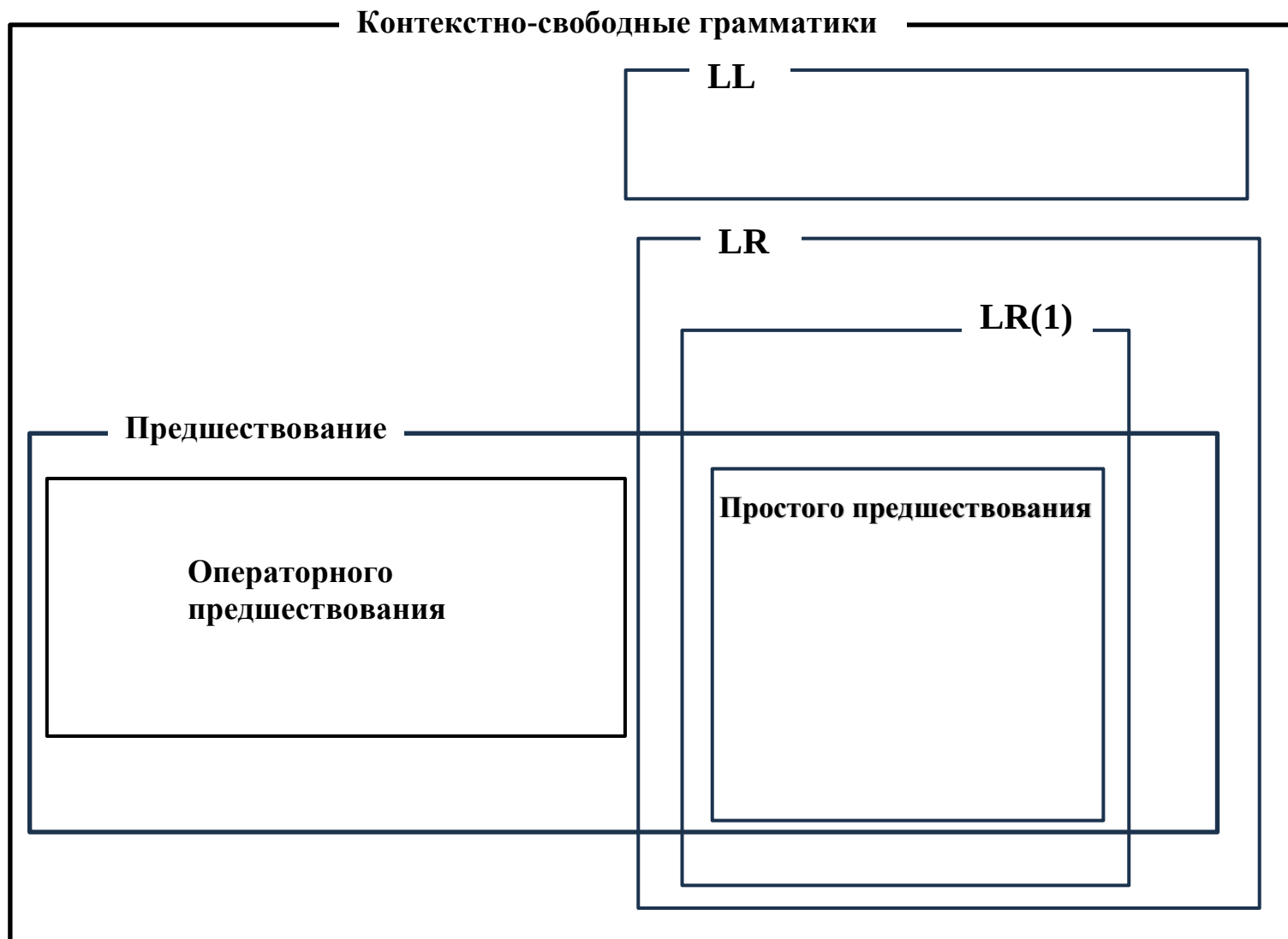


Рис. 12.10. Соотношение классов КС-грамматик и языков

Все это тем более интересно, если вспомнить, что рассмотренный в данном курсе класс левоанализируемых LL-грамматик является собственным подмножеством класса LR-грамматик: любая LL-грамматика является LR-грамматикой, но не наоборот - существуют LR-грамматики, которые не являются LL-грамматиками. Этот факт также нашел свое отражение в схеме на рис. 12.10. Значит, любая LL-грамматика является правоанализируемой, но существуют также и другие левоанализируемые грамматики, не попадающие в класс правоанализируемых грамматик.

Для LL(k)-грамматик, составляющих класс LL-грамматик, интересна еще одна особенность: доказано, что всегда существует язык, который может быть задан LL(k)-грамматикой для некоторого $k > 0$, но не может быть задан LL(k-1)-грамматикой. Таким образом, все LL(k)-грамматики для всех k представляют определенный интерес (другое дело, что распознаватели для них при больших значениях k будут слишком сложны). Интересно, что проблема эквивалентности для двух LL(k)-грамматик разрешима.

С другой стороны, для LR(k)-грамматик, составляющих класс LR-грамматик, доказано, что любой язык, заданный LR(k)-грамматикой с $k > 1$, может быть задан LR(1)-грамматикой. То есть LR(k)-грамматики с $k > 1$ интереса не представляют. Однако доказательство существования LR(1)-грамматики вовсе не означает, что такая грамматика всегда может быть построена (проблема преобразования КС-грамматик неразрешима).

На рис. 12.10 условно показана связь между некоторыми классами КС-грамматик, упомянутых в данном пособии. Из этой схемы видно, например, что любая LL-грамматика является LR-грамматикой, но не всякая LL-грамматика является LR(1)-грамматикой.

Если вспомнить, что любой детерминированный КС-язык может быть задан, например, LR(1)-грамматикой, но в то же время, классы левоанализируемых и правоанализируемых грамматик несопоставимы, то напрашивается вывод: один и тот же детерминированный КС-язык может быть задан двумя или более несопоставимыми между собой грамматики. Таким образом, можно вернуться к мысли о том, что проблема преобразования КС-грамматик неразрешима (на самом деле, конечно, наоборот: из неразрешимости проблемы преобразования КС-грамматик следует возможность задать один и тот же КС-язык двумя несопоставимыми грамматики). Это, наверное, самый интересный вывод, который можно сделать из сопоставления разных классов КС-грамматик.

Отношения между классами КС-языков

КС-язык называется языком некоторого класса КС-языков, если он может быть задан КС-грамматикой из данного класса КС-грамматик. Например, класс LL-языков составляют все языки, которые могут быть заданы с помощью LL-грамматик.

Соотношение классов КС-языков представляет определенный интерес, оно не совпадает с соотношением классов КС-грамматик. Это связано с многократно уже упоминавшейся проблемой преобразования грамматик. Также LL-языки являются собственным подмножеством LR-языков: всякий LL-язык является

одновременно LR-языком, но существуют LR-языки, которые не являются LL-языками. Поэтому LL-языки образуют более узкий класс, чем LR-языки.

На рис. 12.10 приведено соотношение между некоторыми известными классами КС-языков.

Следует обратить внимание прежде всего на то, что интересующий разработчиков компиляторов в первую очередь класс детерминированных КС-языков полностью совпадает с классом LR-языков и, более того, совпадает с классом LR(1)-языков. То есть доказано, что для любого детерминированного КС-языка существует задающая его LR(1)-грамматика. Этот факт уже упоминался выше. Проблема состоит в том, что не всегда возможно найти такую грамматику и нет формализованного алгоритма, как ее построить в общем случае.

Языки простого предшествования, в свою очередь, также являются собственным подмножеством LR-языков, а языки операторного предшествования — собственным подмножеством языков простого предшествования. Интересно, что языки операторного предшествования представляют собой более узкий класс, чем языки простого предшествования.

В то же время языки простого предшествования и LL-языки несопоставимы между собой: существуют языки простого предшествования, которые не являются LL-языками, и в то же время существуют LL-языки, которые не являются языками простого предшествования. Однако существуют языки, которые одновременно являются и языками простого предшествования, и LL-языками. Аналогичное замечание относится также к соотношению между собой языков операторного предшествования и LL-языков.

Можно еще отметить, что язык арифметических выражений над символами a и b , заданный грамматикой $G(\{+,-,/,*,a,b\},\{S,T,E\},P,S)$, $P = \{S \rightarrow S+T | S-T | T, T \rightarrow T*E | T/E | E, E \rightarrow (S) | a | b\}$, который многократно использовался в примерах в данном учебном пособии, подпадает под все указанные выше классы языков. Из приведенных ранее примеров можно заключить, что этот язык является и LL-языком, и языком операторного предшествования, а следовательно, и языком простого предшествования и, конечно, LR(1)-языком. В то же время этот язык по мере изложения материала пособия описывался различными грамматиками, не все из которых могут быть отнесены в указанные классы. Более того, он может быть задан с помощью грамматики, которая не являлась даже однозначной.

Таким образом, соотношение классов КС-языков не совпадает с соотношением задающих их классов КС-грамматик. Это связано с неразрешимостью проблем преобразования и эквивалентности грамматик, которые не имеют строго формализованного решения.

Соотношения между типами грамматик:

- 1) любая регулярная грамматика является КС-грамматикой;
- 2) любая регулярная грамматика является укорачивающей КС-грамматикой (УКС). Отметим, что УКС-грамматика, содержащая правила вида $A \rightarrow \epsilon$, не является КЗ-грамматикой и не является неукорачивающей грамматикой;
- 3) любая (приведенная) КС-грамматика является КЗ-грамматикой;
- 4) любая (приведенная) КС-грамматика является неукорачивающей грамматикой;

- 5) любая КЗ-грамматика является грамматикой типа 0.
- 6) любая неукорачивающая грамматика является грамматикой типа 0.

Определение 19. Язык $L(G)$ является языком типа k , если его можно описать грамматикой типа k .

Соотношения между типами языков:

- 1) каждый регулярный язык является КС-языком, но существуют КС-языки, которые не являются регулярными (например, $L = \{a^n b^n \mid n > 0\}$);
- 2) каждый КС-язык является КЗ-языком, но существуют КЗ-языки, которые не являются КС-языками (например, $L = \{a^n b^n c^n \mid n > 0\}$);
- 3) каждый КЗ-язык является языком типа 0. УКС-язык, содержащий пустую цепочку, не является КЗ-языком. Если язык задан грамматикой типа k , то это не значит, что не существует грамматики типа k' ($k' > k$), описывающей тот же язык. Поэтому, когда говорят о языке типа k , обычно имеют в виду максимально возможный номер k .

Например, КЗ-грамматика $G_1 = (\{0,1\}, \{A,S\}, P_1, S)$ и КС-грамматика $G_2 = (\{0,1\}, \{S\}, P_2, S)$, где $P_1 = \{S \rightarrow 0A1, 0A \rightarrow 00A1, A \rightarrow \varepsilon\}$ и $P_2 = \{S \rightarrow 0S1 \mid 01\}$

описывают один и тот же язык $L = L(G_1) = L(G_2) = \{0^n 1^n \mid n > 0\}$. Язык L называют КС-языком, т.к. существует КС-грамматика, его описывающая. Но он не является регулярным языком, т.к. не существует регулярной грамматики, описывающей этот язык.